

2023-2024

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : INFORMATIQUE

VERMEG

**Développement d'une Solution Digitale de
gestion des clauses bénéficiaires :Solife**

Réalisé par : Askri Hiba

Encadré par :MaaloulMejdi

Encadrant ESPRIT : MellitiDhouha

Encadrant Entreprise :MejdiMaaloul

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I
validate the submission of the student's report:

- Nom & Prénom /Name & Surname : **Askri Hiba**

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : **Maaloul Mejdi**

Cachet & Signature / Stamp & Signature



Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : **Melliti Dhouha**

Signature / Signature

Dédicace

À mon cher père **Riadh**, dont la sagesse, l'amour et les sacrifices continuent de guider mon chemin, même en son absence. Tu as toujours été une source d'inspiration, et ton souvenir est le moteur de ma détermination. À ma chère mère **Raja**, pour son amour inébranlable et son soutien constant. Tu as été mon pilier, et chaque réussite est une ode à ton dévouement sans faille.

À mon frère **Mortadha**, pour ton soutien constant, ton amour fraternel et ta présence toujours réconfortante.

Tu es à mes côtés à chaque étape de mon parcours et tu représentes une force précieuse dans ma vie.

À mon amie **Chaima**, pour ton amitié précieuse, ta bienveillance et ton soutien inestimable. Je te souhaite un avenir merveilleux, rempli de succès, de bonheur et de réalisations.

À mes amis **Mariam, Radhwene, Fatma et Mohamed**, pour votre amitié sincère, vos encouragements et votre soutien moral. Vous avez su illuminer ce voyage académique par vos sourires, vos conseils et votre présence réconfortante. Merci d'avoir été là dans les moments de joie comme dans les moments de doute.

Remerciements

Avant tout, je tiens à rendre grâce à **Dieu** pour Ses bénédictions infinies, Sa guidance et Sa protection tout au long de mon parcours académique. C'est grâce à Sa volonté que j'ai pu atteindre ce stade.

Je souhaite exprimer ma sincère reconnaissance à Monsieur **Mejdi Maaloul**, mon encadrant professionnel, pour ses conseils éclairés et sa patience tout au long de ce projet. Ses orientations ont grandement contribué à l'enrichissement de ce travail.

Mes remerciements vont également à Madame **Melliti Dhouha**, mon encadrante académique, dont l'expertise et les remarques judicieuses ont été d'une aide précieuse dans l'orientation de mes recherches et l'amélioration de la qualité de ce rapport.

Je tiens à remercier chaleureusement les membres du jury pour le temps qu'ils ont consacré à l'évaluation de ce travail et pour leur présence lors de cette étape importante.

Enfin, je ne saurais oublier ceux qui ont, de près ou de loin, participé à la réalisation de ce projet. À mes collègues, enseignants, amis, proches et membres de ma famille, je vous remercie pour vos encouragements, votre soutien constant et vos conseils avisés. Votre aide a été une source de motivation précieuse tout au long de ce voyage académique.

Table des matières

Introduction Générale	1
Chapitre 1 : Contexte et cadre du projet	3
Introduction.....	3
1.1 Présentation de la société Vermeg	3
1.1.1 Vue d'ensemble.....	3
1.1.2 Activités principales et produits.....	4
1.1.3 Portefeuille clients.....	5
1.1.4 Service concerné par le projet	5
1.2 Contexte et problématique.....	5
1.3.Analyse de l'existant.....	6
1.3.1. Analyse technique	6
1.3.2. Limites identifiées.....	6
1.4 Objectifs et solution envisagée.....	7
1.4.1.Approche adoptée	7
1.4.2. Résultats attendus.....	7
1.5 Méthode choisie	8
1.5.1 Adoption de la méthode SCRUM	8
1.5.2 Organisation de l'équipe et des rôles	9
1.5.3 Rôles dans l'équipe SCRUM	10
1.6 Exploration préalable : Secteur et processus métier	10
1.6.1 Domaine de l'assurance	10
1.6.1.1 Principaux concepts de l'assurance.....	10
1.6.1.2 Différents type d'assurances	11
1.6.2 Processus lié aux avenants	12
1.6.3 Gestion des clauses bénéficiaires à travers un avenant administratif	13
Conclusion	13
Chapitre 2 : Analyse des besoins et conception.....	14
Introduction.....	14
2.1 Acteurs et besoins identifiés	14
2.1.1 Identification des parties prenantes.....	14
2.1.2 Identification des besoins fonctionnels	15
2.1.3 Identification des besoins non fonctionnels	15
2.2 Conception initiale	16
2.2.1 Utilisation UML.....	16
2.2.2 Diagramme de cas d'utilisation global.....	16
2.3 Définition des fonctionnalités	17
2.4 Planification et organisation des Sprints	19
2.5 Architecture de la solution	19
2.6 Automatisation des tests.....	20

2.7 Configuration de l'environnement de travail	21
Conclusion	23
Chapitre 3 : Sprint 1 - Étude et réalisation.....	24
Introduction.....	24
3.1 Définition des tâches.....	24
3.2 Cas d'utilisation détaillés	24
3.2.1 Diagrammes de cas d'utilisation du sprint 1	25
3.2.2 Détails des cas d'utilisation.....	25
3.2.2.1 Consulter une police d'assurance.....	25
3.2.2.2 Consulter l'historique des avenants d'une police d'assurance.....	26
3.2.2.3 Consulter les types d'avenants de modification éligibles sur une police.....	27
3.3 Conception technique.....	28
3.3.1 Séquences principales	28
3.3.2 Diagramme de séquences.....	29
3.4 Réalisation et tests.....	31
3.4.1 Configuration des outils	32
3.4.1.1 Configuration de l'outil Keycloak	32
3.4.1.2 Configuration de l'outil Postman.....	33
3.4.1.3 Authentification via interface digitale.....	33
3.4.2 Tests manuels de sprint 1	34
3.4.2.1 Consulter une police d'assurance.....	34
3.4.2.2 Consulter l'historique des avenants sur une police	35
3.4.2.3 Consulter les types d'avenants de modification éligibles sur une police.....	37
3.4.2.4 Consulter les clauses bénéficiaires sur une police	38
Conclusion	39
Introduction.....	40
4.1 Définition des tâches	40
4.2 Cas d'utilisation détaillés	40
4.2.1 Diagramme de cas d'utilisation du sprint 2	41
4.2.2 Détails des cas d'utilisation.....	41
4.2.2.1 Encoder un avenant administratif de subtypeadd clauses	41
4.2.2.2 Encoder un avenant administratif de subtypeModify clauses.....	42
4.2.2.3 Encoder un avenant administratif de subtyperemove clauses.....	44
4.2.2.4 Valider un avenant administratif de gestion des clauses bénéficiaires	45
4.3 Conception technique.....	45
4.3.1 Diagrammes de séquences spécifiques	45
4.3.1.1 Diagramme de séquences « Encoder un avenant administratif de add clauses ».....	46
4.3.1.2 Diagramme de séquences « Valider un avenant administratif de gestion des clauses ».....	47
4.4 Réalisation et tests.....	47
4.4.1 Tests manuels de sprint 2	48
4.4.1.1 Encoder un avenant administratif de type add clauses	48

4.4.1.2 Encoder un avenant administratif de type modify clauses	50
4.4.1.3 Encoder un avenant administratif de type remove clauses	53
4.4.1.4 Valider un avenant administratif de type gestion des clauses.....	55
Conclusion	56
Chapitre 5 : Sprint 3 - Étude et Réalisation	57
Introduction.....	57
5.1 Définition des tâches	57
5.2 Cas d'utilisation détaillés	57
5.2.1 Diagramme de cas d'utilisation du sprint 3	57
5.2.2 Détails des cas d'utilisation.....	57
5.2.2.1 Ajouter un tier	58
5.2.2.2 Modifier un tier	59
5.2.2.3 Consulter un tier.....	59
5.3 Conception technique.....	60
5.3.3 Diagramme de séquences spécifiques.....	60
5.3.3.1 Diagramme de séquences de cas utilisation « Ajouter un tier »	60
5.4 Réalisation et tests.....	61
5.4.1 Tests manuels de sprint 3	61
5.4.1.1 Ajouter un tier	61
5.4.2.1 Mettre à jour un tier	63
5.4.3.1 Consulter un tier.....	64
Conclusion	65
Chapitre 6 : Sprint 4 - Automatisation et déploiement	66
Introduction.....	66
6.1 Définition des tâches	66
6.2 Détails des cas d'utilisations	66
6.2.1 Spécification de cas d'utilisation « Automatisation des tests et intégration continue» ..	66
6.2.1.1 Valider tous les scenarios de tests.....	67
6.2.1.2 Consulter les pipelines Jenkins et la réalisation de la release	68
6.3 Réalisation des tests automatisés	68
6.3.1 Tests automatisés sprint 1	68
6.3.2 Génération des rapports des tests automatisés du sprint 1	71
6.3.3 Tests automatisés Sprint 2.....	72
6.3.4 Génération des rapports des tests automatiques sprint 2.....	75
6.3.5 Tests automatisés Sprint 3.....	76
6.3.6 Génération des rapports des tests automatiques sprint 3.....	78
6.4 Intégration Continue CI et déploiement CD	79
6.4.1 Pipelines Jenkins et configuration CI/CD.....	79
6.4.2 Livraison continue et déploiement continu CD	80
6.4.3 Réalisation de CI/CD	81
6.4.3.1 Création et Exécution d'un Pipeline Jenkins	81

6.4.3.2 Configuration de la chaîne de déploiement.....	83
Conclusion	85
Conclusion générale.....	86
Webographie	87

Liste des tableaux

Tableau 1. Le backlog global du projet	18
Tableau 2. L'organisation des sprints.....	19
Tableau 3. Lebacklog du sprint 1	24
Tableau 4.Scénario de cas d'utilisation Consulter une police	25
Tableau 5.Scénario de cas d'utilisation Consulter l'historique des avenants sur une police ...	26
Tableau 6.Scénario de cas d'utilisation Consulter les types d'avenants de modification éligibles sur une police.....	27
Tableau 7.Scénario de cas d'utilisation Consulter les clauses bénéficiaires sur une police.....	28
Tableau 8.Backlog du sprint 2.....	40
Tableau 9.Le scénario de cas d'utilisation « Encoder un avenant administratif Add clauses »	42
Tableau 10. Le scénario de cas d'utilisation « Encoder un avenant administratif Modify clauses »	43
Tableau 11. Le scénario de cas d'utilisation « Encoder un avenant administratif Remove clauses »	44
Tableau 12. Le scénario de cas d'utilisation « Valider un avenant administratif de gestion clauses bénéficiaires »	45
Tableau 13.Lebacklog du sprint 3	57
Tableau 14. Le scénario de cas d'utilisation « Ajouter un tier »	58
Tableau 15. Le scénario de cas d'utilisation « Mettre à jour un tier »	59
Tableau 16. Le scénario de cas d'utilisation de « consulter un tier ».....	60
Tableau 17. Lebacklog du sprint 4	66
Tableau 18. Le scénario de cas d'utilisation valider les scénarios de tests	67
Tableau 19.Le scénario de cas d'utilisation consulter les pipelines et la génération de release	68

Liste des figures

Figure 1. Composants du Produit Solife.....	5
Figure 2. Méthode de Scrum.....	9
Figure 3.Processus global de création d'un avenant	12
Figure 4.Diagramme de cas d'utilisation global du projet.....	17
Figure 5.Vue Architecture REST.....	20
Figure 6. Le diagramme de cas d'utilisation de sprint 1 de projet.....	25
Figure 7.Le diagramme de séquence de consulter une police.....	29
Figure 8.Le diagramme de séquence de consulter l'historique des avenants sur une police ...	30
Figure 9.Le diagramme de séquence de consulter les types d'avenants de modification éligibles sur une police.....	31
Figure 10. Configuration de l'outil Keycloak.....	32
Figure 11.Génération du token via Postman	33
Figure 12.Interface digitale authentication	34
Figure 13.Test API consultation police avec user_dig via Postman.....	34
Figure 14.Interface digitale consulter police.....	35
Figure 15. Test API consultation historique avenants sur police avec user_dig via Postman .	36
Figure 16.Test consultation historique avenants sur police via Interface digitale	36
Figure 17.Test API consultation types d'avenants de modification éligibles sur police via Postman.....	37
Figure 18.Test consultation types de modification éligibles sur police via Postman	38
Figure 19.Test API consultation des clauses bénéficiaires sur police via Postman.....	38
Figure 20.Test consultation des clauses bénéficiaires sur police via Interface digitale.....	39
Figure 21. Le diagramme de cas d'utilisation du sprint 2 du projet	41
Figure 22. Le diagramme de séquence de « add clauses »	46
Figure 23. Le diagramme de séquence de « valider un avenant »	47
Figure 24.Test API « Encoder un avenant administratif add clauses » via Postman.....	49
Figure 25. Interface digitale « Encoder un avenant administratif add clauses »	49
Figure 26.Interface digitale « Encoder un avenant administratif add clauses »	50
Figure 27.Test API « Encoder un avenant administratif modify clauses via Postman »	51
Figure 28.Interface digitale « Encoder un avenant administratif modify clauses »	52
Figure 29.Interface digitale « Encoder un avenant administratif modify clauses »	52
Figure 30.Test API « Encoder un avenant administratif remove clauses » via postman.....	54
Figure 31.Interface digitale « Encoder un avenant administratif remove clauses »	54
Figure 32.Interface digitale « Encoder un avenant administratif remove clauses »	55
Figure 33.Test API « valider un avenant administratif de gestion des clauses » via postman	55
Figure 34.Interface digitale « valider un avenant administratif de gestion des clauses »	56
Figure 35.Le Diagramme raffiné du cas d'utilisation Sprint3	58
Figure 36. Le diagramme de séquence de cas d'utilisation « Ajouter un tier ».....	61
Figure 37.Test Api Ajouter un tier avec user_dig via Postman	62
Figure 38. Interface digitale d'ajout un tier	62
Figure 39.Test Api Mettre à jour un tier avec user_dig via Postman	63

Figure 40.Interface digitale Mettre à jour un tier.....	63
Figure 41.Test Api Consulter un tier avec user_dig via Postman.....	64
Figure 42.Interface digitale consulter un tier	64
Figure 43.Le diagramme raffiné de cas d'utilisation Automatisation des tests et intégration continue.....	67
Figure 44.Test automatisé API consulter une police via ReadyApi	69
Figure 45.Test automatisé API consulter historique avenant une police via ReadyApi	70
Figure 46.Test automatisé API consulter des avenants de modification éligibles sur une police via ReadyApi	71
Figure 47.Génération rapport consulter une police.....	71
Figure 48.Génération rapport consulter historique des avenants d'une police	72
Figure 49.Génération rapport consulter des avenants de modification éligibles sur une police.....	72
Figure 50.Test automatisé API add clauses via ReadyApi	73
Figure 51.Test automatisé API Modify clauses via ReadyApi	74
Figure 52.Test automatisé API remove clauses via ReadyApi	75
Figure 53.Génération rapport des apis de gestion clauses bénéficiaires sur une police	75
Figure 54.Test automatisé API consulter un tier via ReadyApi	76
Figure 55.Test automatisé API ajout un tier via ReadyApi	77
Figure 56.Test automatisé API mettre à jour un tier via ReadyApi	78
Figure 57.Génération rapport de test consulter un tier.....	78
Figure 58.Génération rapport de test ajout un tier	79
Figure 59.Génération rapport de test mettre à jour d'un tier	79
Figure 60.Processus d'intégration continue et déploiement	80
Figure 61.Exécution du Pipeline	83
Figure 62. Vue pipeline.....	84
Figure 63.Processus déploiement.....	85

Liste des abréviations

API : Application Programming Interface

HTTPS : HyperText Transfer Protocol Secure

IDE : Integrated Development Environment

SSO : Single Sign-On

CI/CD : Continuous Integration / Continuous Deployment

REST : Representational State Transfer

JSON : JavaScript Object Notation

XML : Extensible Markup Language

SQL : Structured Query Language

UML : Unified Modeling Language

Introduction Générale

Le secteur de l'assurance joue un rôle clé dans le développement économique, contribuant à la stabilité financière d'un pays tout en enregistrant une croissance continue. En raison de son importance stratégique, ce domaine se distingue par des caractéristiques uniques, notamment dans le domaine des investissements en actions, où les compagnies d'assurance se retrouvent souvent parmi les plus grands investisseurs, comme le montrent diverses études.

Avec cette croissance soutenue et les spécificités de leurs activités, les entreprises du secteur sont confrontées à des défis de taille, dont l'un des principaux est l'adaptation de leurs systèmes d'information à une stratégie de développement ambitieuse. Ces systèmes doivent non seulement gérer efficacement les flux de données avec les clients, mais aussi répondre aux évolutions des réglementations.

De plus, les acteurs de l'assurance doivent faire face à des modes de gestion divers, en fonction des prestations offertes, qu'il s'agisse d'assurances-vie ou d'assurances non-vie. Le contrat d'assurance, en tant qu'accord juridique liant l'assuré et l'assureur, comprend une gamme variée de prestations. Sa mise en œuvre nécessite une souscription rigoureuse. Aujourd'hui, la digitalisation de cette procédure devient incontournable, contraignant les acteurs du secteur à se rendre disponibles en ligne, à travers des plateformes multiplateformes qui facilitent et accélèrent le traitement des dossiers de souscription.

C'est dans ce contexte dynamique que VERMEG a identifié la nécessité de moderniser les systèmes des compagnies d'assurance et de proposer une approche innovante en matière de transformation numérique. Cette initiative se concrétise par l'implémentation d'une solution web dédiée à la gestion des clauses bénéficiaires d'un contrat d'assurance-vie via un avenant administratif. Ce projet, au cœur de notre travail de fin d'études en ingénierie informatique, a pour objectif de répondre aux besoins croissants du secteur tout en contribuant à la compétitivité de nos clients dans un marché en perpétuelle évolution.

Notre rapport est structuré en six chapitres principaux : Nous débutons avec le chapitre "Contexte et cadre du projet", qui présente le cadre de notre projet, une analyse de l'existant, une critique de ce dernier, les objectifs poursuivis ainsi que la solution proposée, accompagnée de quelques notions théoriques liées à l'assurance.

Le deuxième chapitre, intitulé "Analyse des besoins et conception", comprend l'identification des besoins et des acteurs, un diagramme des cas d'utilisation global, ainsi que la planification des Sprints et un diagramme de déploiement.

Les chapitres suivants détaillent chaque Sprint, avec pour chaque étape : la gestion du backlog, les diagrammes des cas d'utilisation, des séquences, des captures d'écran des tests manuels et automatiques et des captures d'écran des interfaces digitales. Enfin, nous concluons par un résumé des travaux réalisés et des perspectives pour l'avenir du projet.

Chapitre 1 : Contexte et cadre du projet

Introduction

Dans ce chapitre introductif, nous commençons par présenter l'organisme d'accueil. Nous poursuivons en abordant la problématique identifiée, en mettant en lumière les besoins spécifiques et en proposant une solution adéquate. Pour conclure, nous détaillons la méthodologie de gestion de projet choisie pour assurer la réussite de cette étude.

1.1 Présentation de la société Vermeg

Cette section offre une brève introduction à l'organisme d'accueil dans lequel notre stage de Projet de Fin d'Études (PFE) a été réalisé. Elle débute par une présentation globale de la société Vermeg, puis met en lumière ses principaux domaines d'expertise ainsi que les produits qu'elle offre. Enfin, elle se conclut par une mise en avant des clients partenaires de l'entreprise.

1.1.1 Vue d'ensemble

VERMEG a vu le jour en 2002, suite à son autonomie acquise par rapport à la société mère BFI, réputée pour ses solutions numériques et logicielles dédiées aux secteurs bancaires, financier et assurantiel. En plus de ses logiciels standards répondant aux exigences croissantes de la transformation digitale, VERMEG propose des solutions sur mesure grâce à ses outils exclusifs et son expertise approfondie dans ces domaines.

Présente dans 40 pays et collaborant avec plus de 550 partenaires, parmi lesquels figurent des banques privées, des gestionnaires d'actifs, des compagnies d'assurance et des dépositaires, VERMEG s'appuie sur une équipe composée de 1 600 collaborateurs. Elle s'impose aujourd'hui comme un acteur incontournable dans le développement de logiciels innovants pour les secteurs de la finance et de l'assurance.

La croissance de VERMEG s'est accélérée grâce à une stratégie combinant développement interne et acquisitions ciblées :

- En 2011, elle intègre l'activité d'information réglementaire de Sofgen, y compris IDOM USA, un acteur majeur du marché américain en matière de réglementation.
- En 2014, elle acquiert BSB, une société belge spécialisée dans les logiciels financiers destinés aux secteurs de l'assurance et de la banque privée.
- En 2018, elle renforce sa position en intégrant Lombard Risk, un fournisseur international de solutions dédiées à la gestion des risques, aux reportings réglementaires et à la gestion des garanties.

1.1.2 Activités principales et produits

Les activités de VERMEG s'articulent autour de quatre domaines stratégiques :

- Banque
- Transformation numérique
- Marchés de capitaux
- Assurance.

VERMEG propose une gamme variée de solutions logicielles, adaptées aux besoins des acteurs du secteur financier et de l'assurance :

- **Solife** : Un des meilleurs systèmes de gestion de contrats d'assurance, conçu par et pour les assureurs. Cette solution complète couvre la gestion des contrats d'assurance-vie, d'épargne, de retraite et de prévoyance, aussi bien en individuel qu'en collectif.
- **Megara** : Une plateforme modulaire dédiée au traitement post-marché. Dotée d'une technologie avancée et d'une architecture innovante en trois dimensions (vue opérationnelle, vue client et vue marché), Megara facilite l'intégration de nouveaux marchés tout en répondant aux besoins des clients sans perturber les opérations en cours.
- **Soliam** : Une solution centralisée pour le suivi en temps réel des investissements. Elle permet une gestion complète des processus de front-office et de middle-office, et s'adresse aux gestionnaires de patrimoine, aux banquiers privés ainsi qu'aux family offices, grâce à sa capacité à gérer plusieurs comptes et dépositaires.
- **Palmyra** : Un outil innovant pour le développement d'applications et d'architectures numériques. Il permet de générer automatiquement une application complète à partir d'un simple modèle UML, accélérant ainsi la transformation numérique.

- **Agile Reporter** : Une solution stratégique conçue pour réduire les coûts et les risques tout en optimisant l'efficacité. Elle permet une gestion intégrée des changements réglementaires, un contrôle direct des processus et garantit la fiabilité des informations réglementaires à l'échelle mondiale.

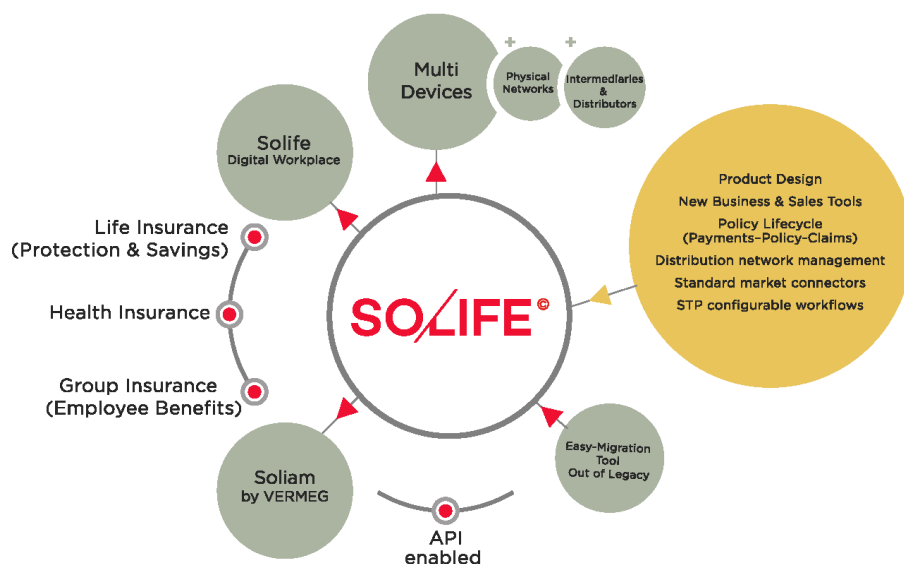


Figure 1 Figure 1. Composants du Produit Solife

1.1.3 Portefeuille clients

Notre clientèle comprend une vaste diversité d'acteurs, du secteur financier, allant des cabinets de gestion de patrimoine familial aux grandes institutions nationales et internationales, telles que la Banque de France, la Société Générale et la Bank of England. Parmi nos partenaires figurent également des compagnies d'assurance de renom, notamment Cardif Lux Vie, Milleis, AXA, MAIF, Lifetri et NNBE.

1.1.4 Service concerné par le projet

Pour réaliser notre projet, nous avons rejoint le département R&D, une équipe dédiée à la conception de solutions innovantes pour les compagnies d'assurance. Ce département se focalise principalement sur Solife, une plateforme avancée destinée à l'administration et à la gestion des contrats d'assurance vie.

1.2 Contexte et problématique

Dans le cadre de notre projet de fin d'études, nous avons entrepris de moderniser et d'optimiser les fonctionnalités de gestion des clauses bénéficiaires dans le système **Solife**, développé par VERMEG. En mobilisant les connaissances acquises tout au long de notre

formation, nous avons développé des WebServices REST pour numériser ces fonctionnalités, tout en adoptant une approche modulaire et structurée.

1.3.Analyse de l'existant

Cette section est dédiée à l'évaluation de la solution existante dans **Solife**, en mettant en lumière ses fonctionnalités actuelles ainsi que ses principales limitations.

1.3.1. Analyse technique

Pour assurer une application performante et répondant aux attentes des clients, nous avons réalisé une analyse détaillée des web services REST intégrés dans Solife. Actuellement, une digitalisation progressive est en place, permettant aux utilisateurs de bénéficier d'un accès simplifié à certaines fonctionnalités grâce aux API REST. Néanmoins, cette transformation ne couvre pas encore toutes les fonctionnalités du backend, en particulier celles liées à la gestion des clauses bénéficiaires.

1.3.2. Limites identifiées

L'approche actuelle, centrée sur un seul compartiment, présente des limitations majeures pour les assurés et les assureurs, particulièrement en matière de gestion des clauses bénéficiaires.

Pour les assurés, cette méthode impose des contraintes dès la souscription initiale du contrat, moment où ils doivent désigner les bénéficiaires et définir les modalités de distribution des prestations en cas de décès. Une fois le contrat souscrit, il devient difficile pour eux de modifier ou de mettre à jour les bénéficiaires, ce qui peut poser un problème en cas de changements dans leur situation familiale ou de nouvelles préférences.

Pour les assureurs, ces restrictions engendrent une gestion administrative complexe, avec un volume élevé de polices à traiter et un nombre important de modifications nécessaires lors de changements de bénéficiaires. De surcroît, cette approche limite la transparence et la flexibilité des produits proposés, restreignant ainsi la capacité des assureurs à répondre efficacement aux besoins évolutifs de leur clientèle.

1.4 Objectifs et solution envisagée

1.4.1.Approche adoptée

Face à ces enjeux, notre projet propose une approche innovante pour la gestion des clauses bénéficiaires dans Solife, en développant des APIs REST spécifiquement dédiées à cette fonction. Cette nouvelle approche se matérialisera par la création d'un processus d'avenants permettant de gérer les clauses bénéficiaires, avec la possibilité pour les assureurs de choisir parmi plusieurs sous-types tels que "ajouter", "modifier" ou "supprimer".

Notre solution inclura des fonctionnalités destinées aux assurés, leur permettant de soumettre des avenants pour ajouter de nouveaux bénéficiaires, modifier ceux existants ou encore supprimer des bénéficiaires désignés. Ces opérations seront effectuées via une interface simple, intuitive et adaptée. Ce processus vise à offrir une plus grande souplesse aux assurés, en leur permettant de mettre à jour leurs bénéficiaires en fonction de leurs besoins et de leur situation, tout en évitant les démarches complexes ou la nécessité de souscrire de nouveaux contrats. En intégrant cette méthodologie centrée sur les avenants de gestion des clauses bénéficiaires, notre objectif est de simplifier et d'optimiser les processus pour les assurés, tout en garantissant aux assureurs une flexibilité et une transparence accrues. Grâce à des APIs REST dédiées, notre solution vise également à améliorer l'efficacité opérationnelle et à accroître la satisfaction des clients dans le secteur de l'assurance vie.

1.4.2. Résultats attendus

Les objectifs majeurs de notre projet se déclinent comme suit :

- Concevoir et développer de nouveaux WebServices REST afin d'améliorer la gestion des clauses bénéficiaires dans Solife. Cela permettra aux assurés de soumettre des avenants pour ajouter, modifier ou supprimer des clauses et/ou des bénéficiaires désignés, offrant ainsi une plus grande flexibilité dans la gestion de leurs contrats d'assurance vie.
- Faciliter la consultation des contrats d'assurance, l'historique des avenants enregistrés et les types d'avenants éligibles pour chaque contrat, offrant ainsi aux assurés une meilleure visibilité sur le choix des bénéficiaires.
- Assurer la gestion des tiers.
- Simplifier la modification et l'ajustement des clauses bénéficiaires d'un contrat d'assurance vie. Une interface conviviale et performante permettra aux assurés de

mettre à jour leurs clauses et/ou bénéficiaires en fonction de leurs besoins et de leur situation personnelle.

En résumé, notre projet a pour objectif de moderniser et d'optimiser les fonctionnalités de gestion des clauses bénéficiaires dans Solife. Grâce à l'implémentation de WebServices REST efficaces et accessibles, nous offrons une solution plus flexible et transparente, renforçant ainsi la compétitivité de Vermeg sur le marché de l'assurance vie.

1.5 Méthode choisie

1.5.1 Adoption de la méthode SCRUM

Les choix exposés dans cette section reflètent un effort collaboratif entrepris dès les premières étapes du stage. Ces décisions, portant sur l'approche méthodologique, le langage de modélisation et le processus de développement, s'appuient à la fois sur la stratégie de l'entreprise et sur des recherches approfondies menées en amont.

L'objectif principal est d'assurer la réalisation du projet dans les délais prévus, tout en maintenant un niveau de qualité optimal et en minimisant les risques liés aux erreurs de spécification ou aux modifications imprévues des besoins. Pour y parvenir, nous avons opté pour une méthodologie agile, reconnue pour sa capacité à structurer efficacement le processus de développement. Parmi les diverses méthodes agiles disponibles, nous avons retenu Scrum, pour les raisons suivantes :

- **Conformité avec les pratiques internes** : Scrum est la méthodologie privilégiée par Vermeg pour la gestion de ses projets.
- **Adaptation aux cycles courts** : Cette méthode est particulièrement bien adaptée aux itérations rapides et contrôlées.
- **Simplicité et efficacité** : Scrum repose sur des processus clairs, ce qui facilite son implémentation.
- **Amélioration des performances** : Elle favorise une augmentation notable de la productivité.
- **Flexibilité accrue** : Scrum offre une grande capacité d'ajustement face aux évolutions du projet.

Cette approche nous permet d'assurer un développement harmonieux et cohérent, tout en respectant les exigences et les normes de qualité imposées par l'entreprise.

1.5.2 Organisation de l'équipe et des rôles

Actuellement, la méthode **Scrum** connaît une adoption croissante au sein des équipes de développement.

Cette approche agile s'impose comme une solution efficace face aux défis des projets en constante évolution, en mettant l'accent sur l'interaction entre les membres de l'équipe, les managers et en renforçant la collaboration avec le client. Elle favorise également une grande réactivité face aux changements et stimule un développement créatif et productif de produits à forte valeur ajoutée.

Les projets utilisant la méthode **Scrum** sont organisés en cycles de travail relativement courts appelés

"**Sprints**". Ces cycles permettent de planifier les étapes suivantes du projet et d'évaluer régulièrement les progrès réalisés. D'une durée allant d'une à quatre semaines, les sprints offrent également la possibilité de réorienter le projet si nécessaire. La figure [2] illustre le fonctionnement de Scrum, en mettant en avant les étapes clés et les différents rôles au sein de l'équipe.

Outre l'amélioration de la productivité et de la communication, la méthode **Scrum** présente de nombreux autres avantages. Elle repose sur une structure bien définie, composée de rôles, de responsabilités et de réunions fixes, tout en offrant une flexibilité et une capacité d'adaptation optimales dans la gestion des projets.

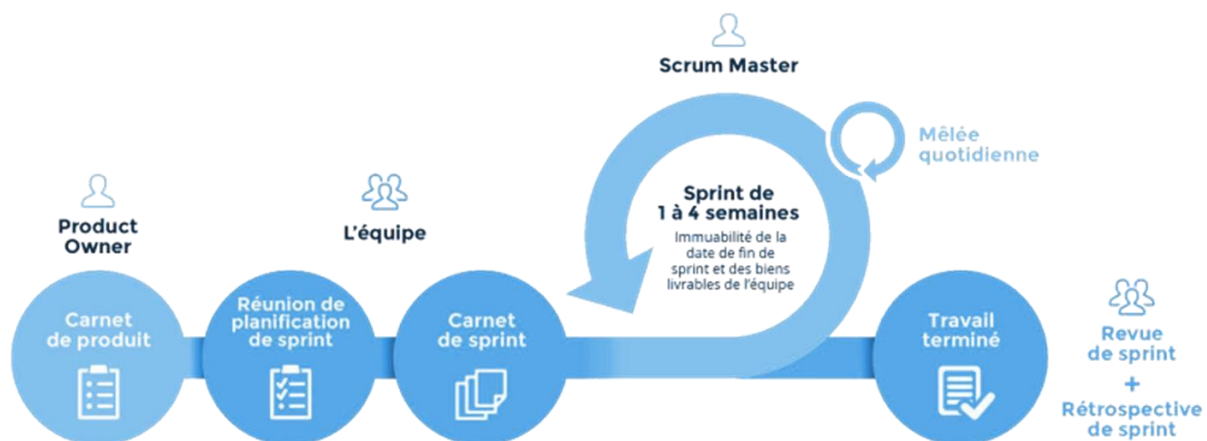


Figure 2. Méthode de Scrum

1.5.3 Rôles dans l'équipe SCRUM

Une équipe Scrum se compose de trois rôles essentiels :

- **Le Product Owner** : En tant que responsable du produit, il définit et conçoit les fonctionnalités.
Expert métier, il priorise les éléments du backlog pour assurer que l'équipe travaille sur les tâches les plus cruciales.
- **Le Scrum Master** : Il veille à l'application des pratiques Scrum, soutient l'équipe dans l'atteinte de ses objectifs et supprime les obstacles qui pourraient entraver l'avancement du projet.
- **L'équipe de développement** : Composée de divers profils (développeurs, analystes, architectes, testeurs, etc.), elle est responsable de la réalisation des objectifs définis et de la mise en œuvre des fonctionnalités du produit.

1.6 Exploration préalable : Secteur et processus métier

1.6.1 Domaine de l'assurance

Le secteur de l'assurance constitue un pilier essentiel de l'économie mondiale, offrant une couverture financière face à une multitude de risques et d'événements imprévus. Dans ce contexte, il est primordial de se familiariser avec certains termes techniques couramment employés dans le domaine de l'assurance.

1.6.1.1 Principaux concepts de l'assurance

- **Le contrat d'assurance**: Il s'agit d'un accord par lequel l'assureur s'engage à indemniser l'assuré en cas de sinistre, conformément aux termes établis. En contrepartie, l'assuré verse une prime, soit de manière ponctuelle, soit régulièrement, pour bénéficier d'une protection financière contre les dommages.
- **La police d'assurance**: Ce document contractuel formalise la relation entre l'assuré et l'assureur. Il précise les clauses du contrat, les garanties offertes, ainsi que les droits et obligations des deux parties.
- **L'assureur**: Il désigne l'entité (physique ou morale) qui fournit une couverture financière contre les aléas. Sa responsabilité se limite à la prise en charge des sinistres déclarés, en assumant les compensations prévues.

- **L'assuré:** C'est la personne directement concernée par le contrat, mais qui peut être différente du souscripteur. Elle doit déterminer, dès le départ, le montant du capital à assurer.
- **Le bénéficiaire:** Cette personne est désignée par le souscripteur pour percevoir le capital assuré dans le cadre d'un contrat d'assurance vie, en cas de décès de l'assuré. Pour préciser l'identité des bénéficiaires, une clause bénéficiaire est remplie par l'assuré.
- **Le souscripteur:** C'est l'individu qui souscrit un contrat d'assurance vie, soit pour lui-même, soit pour autrui (conjoint, parent, etc.). Il fournit les informations nécessaires à l'évaluation des risques et à la détermination de la prime.
- **L'avenant au contrat d'assurance:** Cet acte permet de modifier certains termes du contrat initial, comme l'ajout de garanties ou l'ajustement des montants couverts. Il doit être signé par les deux parties, de la même manière que le contrat de base.
- **L'avenant de gestion des clauses bénéficiaires:** Cet acte revêt une importance particulière en assurance vie, car il offre aux assurés la possibilité de mettre à jour les bénéficiaires désignés en fonction de leurs besoins ou changements de situation (familiale ou autre). Grâce à cet avenant, les bénéficiaires peuvent être ajoutés, modifiés ou supprimés, garantissant ainsi une gestion adaptée et efficace des contrats d'assurance vie tout en respectant les souhaits de l'assuré.

1.6.1.2 Différents type d'assurances

L'assurance se définit comme un contrat liant une personne physique ou morale, désignée comme l'assuré, à une compagnie d'assurance. Ce contrat stipule que l'assureur s'engage à compenser les pertes ou les dommages subis par l'assuré en contrepartie du paiement d'une prime. Parmi les produits proposés par l'assurance traditionnelle, on trouve:

- **Assurance non-vie : habitation/ automobile**
- **Assurance vie :** Elle peut prendre une forme classique, où l'objectif principal est de garantir le versement d'un capital ou d'une rente au bénéficiaire désigné en cas de décès de l'assuré.

Les contrats d'assurance-vie peuvent être structurés de manière à capital fixe, offrant ainsi une protection avec un taux d'intérêt garanti sur une période déterminée, ou à capital variable, permettant aux souscripteurs de choisir des fonds parmi une sélection proposée par la compagnie d'assurance, offrant ainsi une gestion plus souple de leurs investissements.

1.6.2 Processus lié aux avenants

Le processus global de création d'un avenant administratif dans Solife suit une série d'étapes précises, assurant la conformité et l'efficacité de chaque modification apportée au contrat d'assurance-vie. Lorsqu'un souscripteur soumet une demande de modification, celle-ci est d'abord enregistrée dans le système comme un avenant "en attente". Ensuite, une transaction commerciale est initiée, suivie d'un flux de travail "standard" pour traiter la demande, incluant une vérification minutieuse et la réception des documents nécessaires.

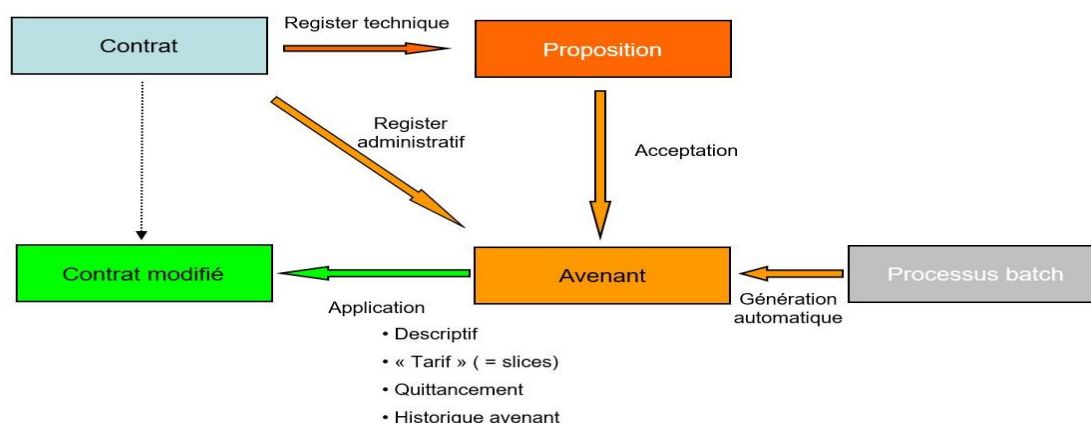


Figure 3. Processus global de création d'un avenant

L'étape suivante du processus consiste à intégrer l'avenant, une phase essentielle pour appliquer les modifications requises. Cette étape se décompose en plusieurs "vues" opérationnelles spécifiques, telles que "information générale", "suivi" et "clauses", ainsi que des vues adaptées en fonction du type d'avenant, comme "rôles", "bénéficiaires", "nantissement", "quittancement" et "indexation".

Une fois toutes les informations collectées et les changements validés, l'avenant est libéré (release) et sa confirmation finale est effectuée. Il est ensuite envoyé pour distribution (dispatch), et un suivi des impressions est assuré. Le processus se conclut par la mise en vigueur de l'avenant, avec l'application des modifications sur le contrat d'assurance-vie.

Chaque vue opérationnelle est conçue pour fournir des informations spécifiques et faciliter des actions ciblées, telles que la mise à jour des détails de paiement dans la vue "quittancement" ou la modification des bénéficiaires dans la vue "bénéficiaires". Ce processus bien structuré, soutenu par des vues détaillées, garantit une gestion fluide et conforme des modifications contractuelles, tout en assurant la satisfaction des clients et le respect des normes réglementaires.

1.6.3 Gestion des clauses bénéficiaires à travers un avenant administratif

Dans le cadre de la digitalisation de Solife, notre focus se porte sur l'avenant administratif de gestion des clauses bénéficiaires, un élément clé permettant aux souscripteurs de modifier les clauses bénéficiaires de leur contrat d'assurance-vie. Les sous-types associés à cet avenant comprennent :

- **L'ajout de clause bénéficiaire** : Ce sous-type, également appelé "Add clause beneficiary", permet aux souscripteurs d'ajouter de nouvelles clauses bénéficiaires ou de nouveaux bénéficiaires à leur contrat d'assurance-vie. Il est particulièrement utile en cas de changement de situation familiale ou de l'identification de nouveaux bénéficiaires à inclure.
- **La modification de clause bénéficiaire** : Désigné sous le nom "Edit clause beneficiary", ce sous-type permet de modifier les informations liées aux clauses ou des bénéficiaires déjà désignés dans le contrat. Cela peut concerner des changements de nom, de répartition des pourcentages, ou d'autres informations pertinentes.
- **La suppression de clause bénéficiaire** : Également connu sous "Remove clause beneficiary", ce sous-type permet aux souscripteurs de supprimer un ou plusieurs bénéficiaires ou clauses de leur contrat. Cette option est souvent nécessaire en cas de décès d'un bénéficiaire ou de changement de circonstances personnelles.
- En intégrant ces sous-types d'avenant administratif dans Solife, nous offrons aux souscripteurs une plus grande flexibilité pour gérer leurs clauses bénéficiaires. Cette approche leur permettra d'ajuster facilement les bénéficiaires désignés en fonction de leurs besoins et préférences, tout en garantissant que les prestations seront distribuées selon leurs souhaits en cas de décès.

Conclusion

Au sein de ce chapitre, nous avons explicité les concepts fondamentaux essentiels à la compréhension de notre projet. Le chapitre suivant sera consacré à l'examen détaillé des besoins fonctionnels et non fonctionnels, ainsi qu'à l'identification des acteurs impliqués dans le développement et la mise en œuvre du projet.

Chapitre 2 : Analyse des besoins et conception

Introduction

L'analyse et la compréhension des besoins représentent une étape essentielle pour la conception de toute application. Ce chapitre est dédié à l'identification des acteurs concernés par notre projet, tout en mettant en lumière les différentes fonctionnalités du système. Celui-ci doit répondre aux besoins fonctionnels de tous les utilisateurs tout en respectant les contraintes non fonctionnelles lors de son développement.

Par ailleurs, ce chapitre inclut une présentation des diagrammes spécifiques à la méthodologie Scrum. Enfin, il aborde l'architecture globale, l'environnement de travail, ainsi que le processus des tests automatisés.

2.1 Acteurs et besoins identifiés

La spécification des besoins se divise en deux catégories principales pour faciliter leur compréhension : les besoins fonctionnels, qui définissent les fonctionnalités et les actions que le système doit réaliser, et les besoins non fonctionnels, qui établissent les contraintes et les caractéristiques du système, telles que la performance, la sécurité ou encore l'ergonomie.

2.1.1 Identification des parties prenantes

Dans cette section, nous allons présenter une description des différents acteurs liés au système. Nous commencerons par une brève définition de la notion d'acteur, qui représente un rôle joué par des entités externes (utilisateur, matériel ou autre système) interagissant directement avec le système et utilisant ses fonctionnalités.

Les principaux acteurs identifiés pour notre système sont :

- **Testeur QA** : Cet acteur est chargé de tester les différentes fonctionnalités du système, de consulter les pipelines et de superviser la génération des versions (releases). En cas de dysfonctionnement des services web, le testeur QA crée un ticket JIRA décrivant le problème rencontré et l'assigne à l'équipe de développement pour résolution.

- **Agent** : Cet acteur représente le client de Vermeg, qui correspond à un assureur. Son rôle consiste à interagir avec le système pour gérer les différentes fonctionnalités mises à sa disposition.

2.1.2 Identification des besoins fonctionnels

Au cours de cette étape, nous détaillons les fonctionnalités du système, qui correspondent aux besoins définissant son comportement attendu. Ces fonctionnalités sont identifiées afin de répondre aux objectifs fixés. Les principales fonctionnalités prévues pour notre système sont les suivantes :

- **Gestion des polices** : offrir la possibilité de consulter les polices existantes, d'accéder à la vue des clauses bénéficiaires et de consulter l'historique des avenants associés à une police.
- **Gestion des clauses bénéficiaires** : permettre l'ajout de nouvelles clauses bénéficiaires, la modification des clauses existantes et leur suppression en cas de besoin.
- **Gestion des tiers** : inclure la possibilité d'ajouter de nouveaux tiers, de modifier ceux existants et de consulter les tiers disponibles selon les besoins.
- **Mise en place d'un outil de tests automatisés et intégration continue** : implémenter et tester les services web à l'aide de l'outil ReadyAPI.
- **Mise en place d'un outil de génération de rapports** : permettre la création de rapports quotidiens sur les tests automatisés afin de garantir la non-régression.

2.1.3 Identification des besoins non fonctionnels

Ces besoins précisent les attributs fondamentaux du système, comme la performance, les spécifications matérielles ou le type d'architecture adopté. Ils peuvent également intégrer des contraintes d'implémentation, telles que le choix du langage de programmation, du système de gestion de base de données (SGBD) ou encore du système d'exploitation.

Les principales exigences non fonctionnelles identifiées sont les suivantes:

- **Sécurité** : Utiliser **Keycloak** pour la gestion des accès et garantir un haut niveau de protection des données.
- **Performance** : Optimiser les traitements pour améliorer l'efficacité et réduire les temps de réponse du système.

- **Interopérabilité** : Veiller à ce que l'API puisse s'intégrer et interagir efficacement avec d'autres systèmes et services.
- **Confidentialité** : Assurer que les informations enregistrées dans les contrats restent strictement accessibles aux utilisateurs autorisés, interdisant tout accès non justifié.
- **Ergonomie** : Proposer une interface utilisateur intuitive et conviviale, facilitant son utilisation.
- **Maintenabilité** : Garantir un code lisible, structuré et compréhensible afin de simplifier sa maintenance et d'assurer son évolution.
- **Extensibilité** : Concevoir le système de manière flexible pour permettre l'ajout futur de nouvelles fonctionnalités sans compromettre son architecture existante.

2.2 Conception initiale

2.2.1 Utilisation UML

Pendant la phase de conception, nous avons fait usage du langage de modélisation unifié (UML) pour organiser et planifier notre solution de manière optimale. Les diagrammes UML sélectionnés ont facilité l'identification des acteurs du système et ont permis de définir les interactions entre ses différents éléments.

Parmi les diagrammes utilisés, nous avons particulièrement employé:

- Le diagramme de cas d'utilisation, qui illustre les interactions entre les utilisateurs (acteurs) et le système, en mettant l'accent sur les différentes fonctionnalités offertes.
- Le diagramme de séquence, qui décrit la dynamique des échanges entre les composants du système et les étapes de leurs interactions.

2.2.2 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation est le diagramme général du système qu'on modélise et qui décrit les interactions, les fonctionnalités et les relations globales entre les différents acteurs.

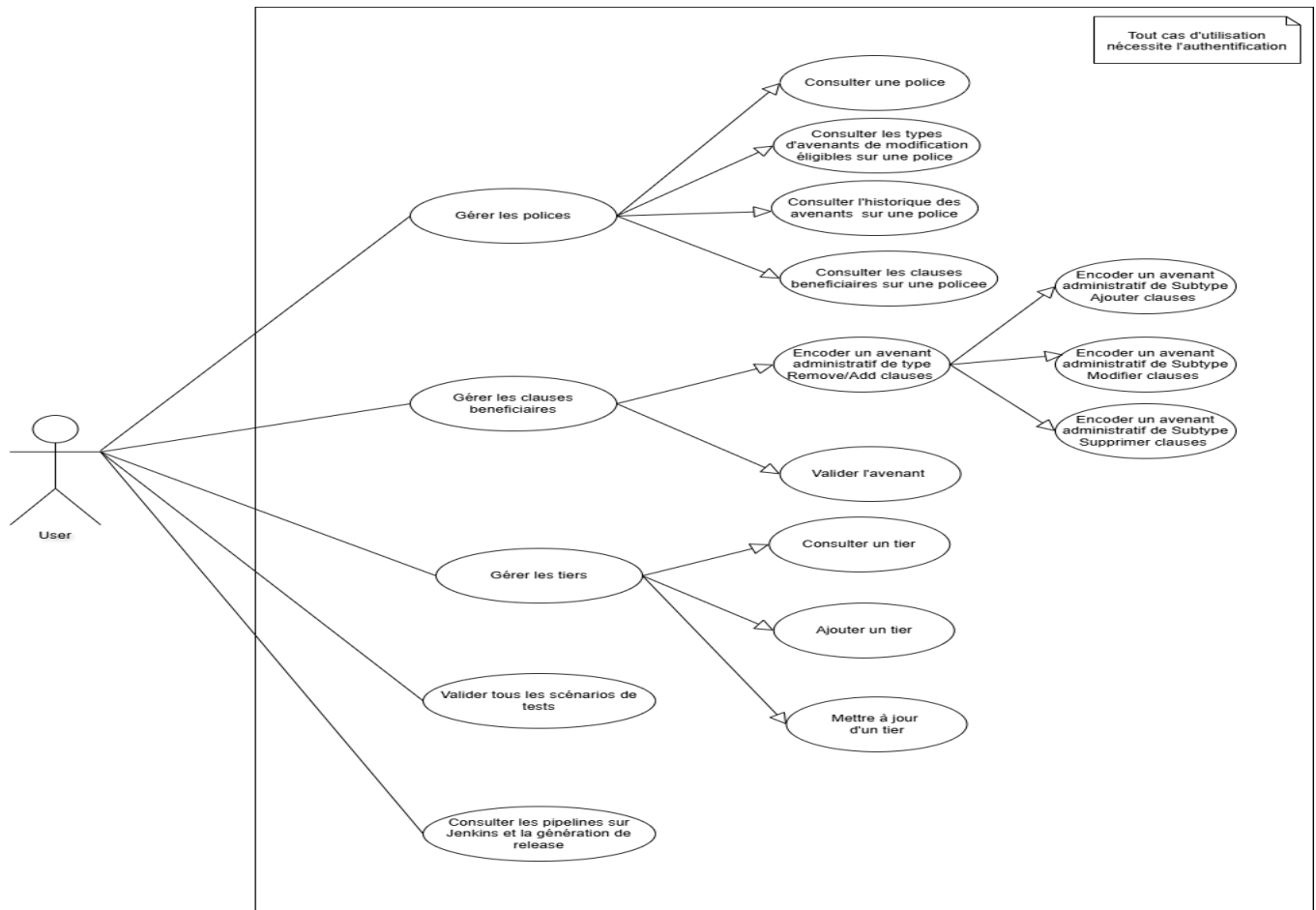


Figure 4. Diagramme de cas d'utilisation global du projet

2.3 Définition des fonctionnalités

Le Backlog est un artéfact très important dans SCRUM. C'est l'ensemble des caractéristiques fonctionnelles ou techniques qui constituent le produit souhaité. Nous allons les décrire en détails dans le tableau qui suit :

Tableau 1. Le backlog global du projet

Id	Titre	User Story	Priorité	Estimation
1	Gestion des polices	En tant qu'un testeur QA ou Agent , je peux consulter des polices.	Elevé	6 semaines
		En tant qu'un testeur QA ou Agent , je peux consulter les types d'avenants de modification éligibles sur une police.		
		En tant qu'un testeur QA ou Agent , je peux consulter l'historique des avenants sur une police.		
		En tant qu'un testeur QA ou Agent , je peux consulter les clauses bénéficiaires sur une police.		
2	Gestion des Clauses bénéficiaires	En tant qu'un testeur QA ou Agent , je peux créer des avenants administratifs de type add/modify/remove clauses	Elevé	6 semaines
		En tant qu'un testeur QA ou Agent , je peux valider des avenants administratifs de type add/ modify/remove clauses		
3	Gestion des tiers	En tant qu'un testeur QA ou Agent , je peux consulter des tiers.	Elevé	6 semaines
		En tant que QA ou Agent , je peux modifier des informations d'un tier.		
		En tant que QA ou Agent , je peux ajouter un tier		
4	Automatisation des tests et intégration continue	En tant qu'un testeur QA , je peux consulter les tests automatiques de gestion des polices sur Allure.	Moyenne	2 semaines
		En tant qu'un testeur QA , je peux consulter les tests automatiques de gestion des clauses bénéficiaires sur Allure.		
		En tant qu'un testeur QA , je peux consulter les tests automatiques de gestion des tiers sur Allure.		
		En tant qu'un testeur QA , je peux consulter les pipelines sur jenkins de la génération de release.		

2.4 Planification et organisation des Sprints

Après avoir finalisé et structuré le **Product Backlog**, nous avons déterminé, durant une session de planification des sprints, les durées approximatives des tâches à réaliser pendant chaque sprint.

Tableau 2. L'organisation des sprints

Les sprints	Nom du sprint
Sprint 1	Gestion des polices
Sprint 2	Gestion des clauses bénéficiaires
Sprint 3	Gestion des tiers
Sprint 4	Automatisation des tests et intégration continue

2.5 Architecture de la solution

Dans cette section, nous allons détailler l'architecture du projet en mettant en avant l'architecture REST.

L'illustration ci-dessous présente le fonctionnement fondamental de cette architecture. Dans un premier temps, le client lance une requête HTTP, spécifiant la ressource qu'il souhaite interroger. Ensuite, le serveur reçoit cette requête, traite la demande en accédant à sa base de données pour récupérer les informations nécessaires. Enfin, il renvoie au client une représentation de la ressource demandée, généralement sous un format standardisé tel que JSON ou XML.

L'architecture REST repose sur une communication sans état, ce qui signifie que chaque requête doit contenir toutes les informations nécessaires à son traitement, sans se référer à des informations de sessions précédentes. Cette approche garantit une scalabilité et une flexibilité accrue, tout en simplifiant l'intégration entre le client et le serveur.

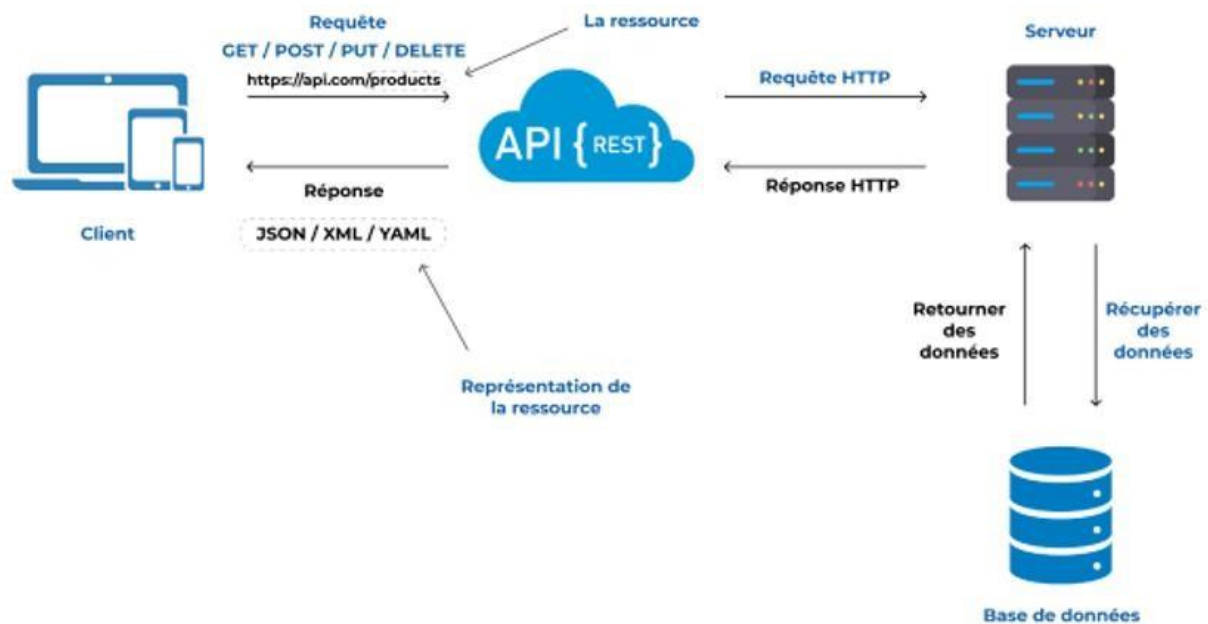


Figure 5.Vue Architecture REST

2.6 Automatisation des tests

Chaque fonctionnalité de Solife Digital nécessite la création de tests automatisés afin de prévenir toute régression. Le processus suit une méthodologie claire et structurée, divisée en plusieurs étapes.

Dans un premier temps, une phase de documentation est indispensable avant de commencer l'implémentation des tests. Cette documentation, rédigée de manière soignée, doit être partagée sous forme de fichiers et validée par le Product Owner. Cela permet de garantir que toutes les exigences et scénarios de test sont correctement compris et adressés.

Une fois la documentation validée, les tests sont développés sur ReadyAPI, une plateforme permettant de créer et d'exécuter les tests API de façon flexible. Après avoir intégré les ajustements nécessaires, un commit est effectué sur GitLab pour mettre à jour le référentiel de code.

Lorsque les demandes de fusion des modifications sont validées sur GitLab, deux jobs sont exécutés sur Jenkins : l'un pour l'installation du projet Solife et l'autre pour l'exécution des tests automatisés. Ces processus permettent de vérifier que les dernières modifications n'ont pas entraîné de régressions dans l'application.

Enfin, après l'exécution des tests, un rapport détaillé est généré et présenté sur Allure, permettant d'avoir une vue d'ensemble des résultats, d'identifier les erreurs éventuelles et de garantir la qualité du produit. Les jobs Jenkins sont exécutés chaque jour pour assurer que le système est constamment testé, réduisant ainsi les risques de régression à chaque nouvelle modification.

2.7 Configuration de l'environnement de travail

La phase de réalisation constitue une étape cruciale, car elle reflète l'aboutissement de tout le travail accompli tout au long de ce projet. Cette section met en lumière les différents outils matériels et logiciels utilisés pour le développement de notre application. Nous avons exploité divers logiciels et langages pour concevoir notre application.



JAVA: Java est un langage de programmation et une plateforme informatique lancée par Sun Microsystems en 1995. Il est orienté objet, basé sur des classes, et conçu pour être portable, ce qui signifie que les programmes Java peuvent s'exécuter sur une large gamme de matériels et de systèmes d'exploitation. Java est couramment utilisé dans le développement d'applications d'entreprise, d'applications mobiles, de jeux vidéo, ainsi que pour d'autres types de logiciels [2].



IntelliJ IDEA : est un IDE principalement utilisé pour le développement en Java, tout en prenant en charge d'autres langages tels que Kotlin, Groovy, Scala, JavaScript, TypeScript, et bien d'autres. Développé par JetBrains, IntelliJ IDEA est réputé pour sa richesse en fonctionnalités, sa convivialité et son intégration étroite avec diverses technologies et frameworks de développement [3].



SQL : SQL, ou Structured Query Language, est un langage de programmation utilisé pour interagir avec les bases de données relationnelles. Il permet principalement de manipuler et de gérer les données stockées dans ces bases. Grâce à SQL, les utilisateurs

peuvent exécuter des requêtes pour insérer, mettre à jour, supprimer et récupérer des données, ainsi que gérer la structure de la base de données elle-même.



POSTMAN *Postman* : est un outil permettant aux développeurs de tester et de documenter les API

(Interfaces de Programmation d'Applications). Il offre la possibilité de créer des requêtes HTTP (GET, POST, PUT, etc.), de gérer les paramètres et les en-têtes, de vérifier les réponses et les statuts de retour, de sauvegarder

les requêtes pour une utilisation future. Il permet également de créer des collections de requêtes pour organiser et partager les tests avec d'autres développeurs. Postman est disponible en tant qu'application de bureau ainsi qu'en extension de navigateur [4].



SMARTBEAR
ReadyAPI

ReadyAPI: est une suite d'outils de test d'API offrant diverses fonctionnalités, telles que les tests fonctionnels, les tests de performance, les tests de sécurité, la simulation d'API, et bien plus encore. Elle permet aux développeurs et aux testeurs de valider et tester les interfaces de programmation d'applications (API) de manière automatisée [5].



Allure
Report

Allure : est un outil de génération de rapports destiné aux tests automatisés. Il fournit des rapports détaillés, interactifs et visuellement attrayants qui résument les résultats des tests. Il présente les tendances, statistiques, graphiques et résultats de manière claire et facile à comprendre. Allure aide les équipes à améliorer la qualité des logiciels en facilitant l'analyse et l'interprétation des résultats des tests [6].



Angular : est une plateforme open source côté client, développée en TypeScript. Elle est maintenue conjointement par l'équipe Angular chez Google et une communauté internationale composée d'individus et d'entreprises. Il s'agit d'une reconstruction complète du framework AngularJS, réalisée par la même équipe. Angular

simplifie la création d'applications web modernes, avec un accent particulier sur les applications monopages, offrant une navigation fluide et dynamique au sein d'une seule page.



Jenkins

Jenkins : Jenkins est un outil logiciel d'intégration continu. Il s'agit d'un logiciel open source, développé à l'aide du langage de programmation Java. Il permet de tester et de rapporter les changements effectués sur une large base de code en temps réel. En utilisant ce logiciel, les développeurs peuvent détecter et résoudre les problèmes dans une base de code et rapidement. Ainsi les tests de nouveaux builds peuvent être automatisés, ce qui permet d'intégrer plus facilement des changements à un projet, de façon continue. L'objectif de Jenkins est en effet d'accélérer le développement de logiciels par le biais de l'automatisation. Jenkins permet l'intégration de toutes les étapes du cycle de développement.



Keycloak: Keycloak est une solution open-source permettant la mise en place d'un SSO (Single Sign-On) interne, facilitant ainsi la gestion des authentifications pour vos systèmes et applications. Lors de la conception de vos projets, cette solution simplifie l'intégration de l'authentification au sein de votre application. Vous n'avez pas à vous préoccuper de la maintenance ni des éventuelles vulnérabilités de sécurité. Une fois l'utilisateur connecté, il n'a plus besoin de ressaisir ses identifiants pour accéder aux différentes fonctionnalités de l'application [7].

Conclusion

À la fin de ce chapitre, nous avons présenté les différentes fonctionnalités de notre système en identifiant les acteurs et en analysant les besoins à travers des diagrammes de cas d'utilisation, afin de mieux comprendre le fonctionnement de l'application. Dans le chapitre suivant, nous débuterons le développement du premier sprint en suivant les directives que nous avons établies.

Chapitre 3 : Sprint 1 - Étude et réalisation

Introduction

Ce chapitre est dédié à la description détaillée des différentes étapes liées à la conception, la mise en œuvre et l'exécution du premier sprint du projet intitulé 'Gestion des polices d'assurance'.

3.1 Définition des tâches

Le tableau suivant rassemble les différentes fonctionnalités du premier sprint.

Tableau 3. Lebacklog du sprint 1

ID	Tâche	User story	Priorité
1	Consulter une police d'assurance	En tant qu'un testeur QA ou Agent , je peux consulter les polices d'assurance.	Élevée
2	Consulter des types des avenants de modification éligibles sur une police d'assurance.	En tant qu'un testeur QA ou Agent , je peux consulter les types d'avenants de modification éligibles sur une police.	Élevée
3	Consulter l'historique des avenants encodés sur une police d'assurance	En tant qu'un testeur QA ou Agent , je peux consulter l'historique des avenants encodés sur une police d'assurance.	Élevée
4	Consulter les clauses bénéficiaires sur une police d'assurance	En tant qu'un testeur QA ou Agent , je peux consulter les clauses bénéficiaires sur une police.	Élevée

3.2 Cas d'utilisation détaillés

Dans cette section, nous détaillerons les cas d'utilisation les plus prioritaires. Pour chaque cas, nous présenterons les préconditions, les post conditions, le scénario principal ainsi que, le cas échéant, les exceptions associées.

3.2.1 Diagrammes de cas d'utilisation du sprint 1

La figure suivante représente le diagramme de cas d'utilisation qui montre la spécification du cas d'utilisation Gestion des polices dans Solife.

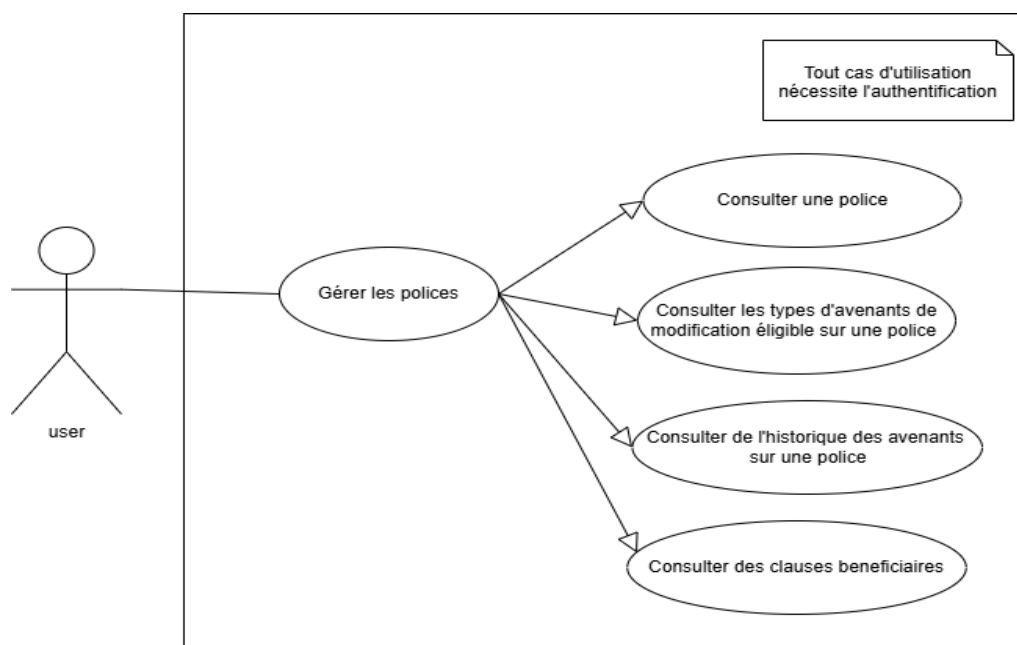


Figure 6. Le diagramme de cas d'utilisation de sprint 1 de projet

3.2.2 Détails des cas d'utilisation

3.2.2.1 Consulter une police d'assurance

Ce tableau offre un résumé du scénario relatif au cas d'utilisation intitulé 'Consulter une police', en incluant également les différents scénarios d'exception associés.

Tableau 4.Scénario de cas d'utilisation Consulter une police

Titre	Consulter une police d'assurance
Acteurs	• Testeur QA • Agent
Résumé	• L'utilisateur saisit le numéro d'une police.
Précondition	• L'utilisateur est authentifié. • Opération de consultation est sélectionnée.
Post condition	• Les informations du contrat sont affichées.
Scénario Principal	1. L'utilisateur saisit l'identifiant de police. 2. L'utilisateur confirme l'opération. 3. Le système retourne le statut 200 OK accompagné des informations du contrat.

Scénario d'exception	1. Date de création de l'avenant antérieure à la date de création de la police : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est affiché, indiquant que la date de création est invalide. 2. Numéro de police introuvable dans la base de données Solife : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est généré, précisant que le numéro de police est incorrect. 3. Date de création postérieure à la date actuelle : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur signale que la date saisie est invalide.
-----------------------------	--

3.2.2.2 Consulter l'historique des avenants d'une police d'assurance

Ce tableau offre un résumé relatif au scénario du cas d'utilisation intitulé "Consulter les avenants d'une police". Le testeur QA ou l'agent, après authentification, doit être en mesure de consulter les avenants associés au contrat concerné.

Tableau 5.Scénario de cas d'utilisation Consulter l'historique des avenants sur une police

Titre	Consulter historique des avenants sur une police
Acteurs	•Testeur QA •Agent
Résumé	•L'utilisateur introduit le numéro de la police avec la date de la consultation.
Pré-condition	• L'utilisateur est authentifié. •Opération de consultation est sélectionnée.
Post condition	• L'historique des avenants relatifs au contrat est affiché.
Scénario Principal	1. L'utilisateur saisit l'identifiant de la police. 2. L'utilisateur confirme l'opération. 3. Le système retourne le statut 200 OK avec les avenants encodés sur le contrat.
Scénario d'exception	1. Date de création de l'avenant antérieure à la date de création de la police : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est affiché, indiquant que la date de création est invalide. 2. Numéro de police introuvable dans la base de données Solife : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est généré, précisant que le numéro de police est incorrect. 3. Date de création postérieure à la date actuelle : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur signale que la date saisie est invalide.

3.2.2.3 Consulter les types d'avenants de modification éligibles sur une police

Ce tableau offre un résumé relatif au scénario du cas d'utilisation intitulé "Consulter les types d'avenants de modification éligibles sur une police". Le testeur QA ou l'agent, après authentification, doit être en mesure de consulter les types d'avenants de modification éligibles associés au contrat concerné.

Tableau 6.Scénario de cas d'utilisation Consulter les types d'avenants de modification éligibles sur une police

Titre	Consulter les types d'avenants de modification éligibles sur une police
Acteurs	• Testeur QA • Agent
Résumé	•L'utilisateur saisit le numéro de la police.
Précondition	• L'utilisateur est authentifié. • Opération de consultation est sélectionnée.
Postcondition	• Les avenants de modification relative au contrat sont affichés.
Scénario Principal	1. L'utilisateur saisit l'identifiant relatif au contrat. 2. L'utilisateur confirme l'opération. 3. Le système retourne le statut 200 OK avec les types d'avenants de modification éligibles sur police.
Scénario d'exception	1. Date de création de l'avenant antérieure à la date de création de la police : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est affiché, indiquant que la date de création est invalide. 2. Numéro de police introuvable dans la base de données Solife : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est généré, précisant que le numéro de police est incorrect. 3. Date de création postérieure à la date actuelle : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur signale que la date saisie est invalide.

Ce tableau offre la description relative au scénario du cas d'utilisation intitulé "Consulter les clauses bénéficiaires sur une police".

Le Testeur QA ou Agent, après avoir été authentifié, doit pouvoir faire consulter les clauses bénéficiaires sur le contrat en question.

Tableau 7.Scénario de cas d'utilisation Consulter les clauses bénéficiaires sur une police

Titre	Consulter les types les clauses bénéficiaires sur un contrat
Acteurs	• Testeur QA •Agent
Résumé	•L'utilisateur saisit le numéro de la police.
Précondition	• L'utilisateur est authentifié. •Opération de consultation est sélectionnée.
Post condition	•Les clauses bénéficiaires relatives au contrat sont affichées.
Scénario Principal	1. L'utilisateur saisit l'identifiant relatif au contrat. 2. L'utilisateur confirme l'opération. 3. Le système retourne le statut 200 OK accompagné des clauses bénéficiaires relatives à la police.
Scénario d'exception	1. Numéro de police introuvable dans la base de données Solife : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est généré, précisant que le numéro de police est incorrect.

3.3 Conception technique

3.3.1 Séquences principales

Le diagramme de séquences, tel que défini dans les sprints 1 et 2, sert à représenter les scénarios de chaque cas d'utilisation en illustrant l'ordre des opérations et les interactions entre les objets. Dans ce chapitre, nous présenterons plusieurs diagrammes de séquences afin d'illustrer ces processus. Ces diagrammes montrent comment les différentes composantes du système interagissent, offrant ainsi une meilleure compréhension du déroulement des actions au sein du système.

3.3.2 Diagramme de séquences

La figure [7] montre le diagramme de séquences de cas d'utilisation « Consulter une police d'assurance ».

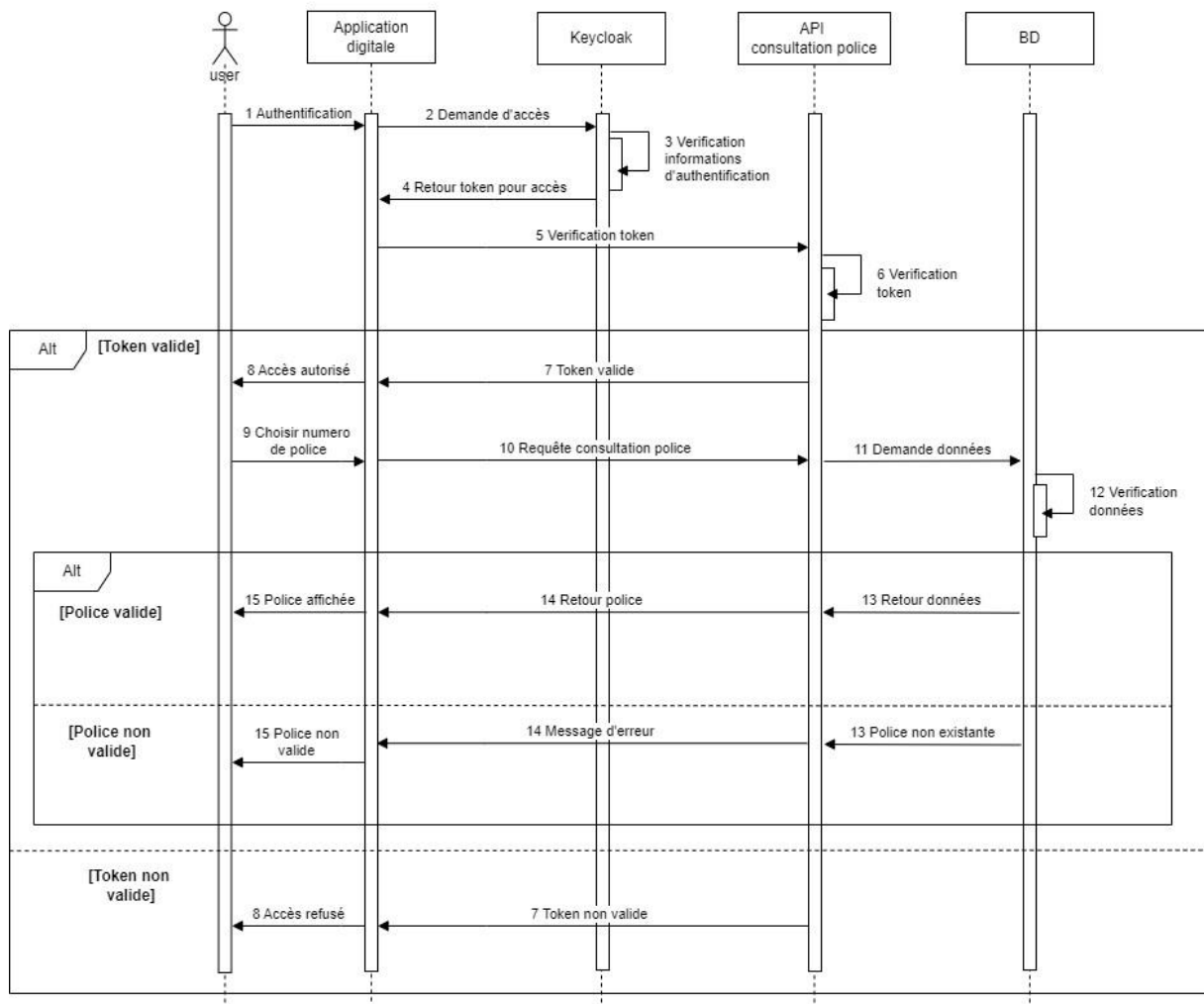


Figure 7. Le diagramme de séquence de consulter une police

La figure [8] montre le diagramme de séquences de cas d'utilisation « Consulter l'historique des avenants sur une police d'assurance ».

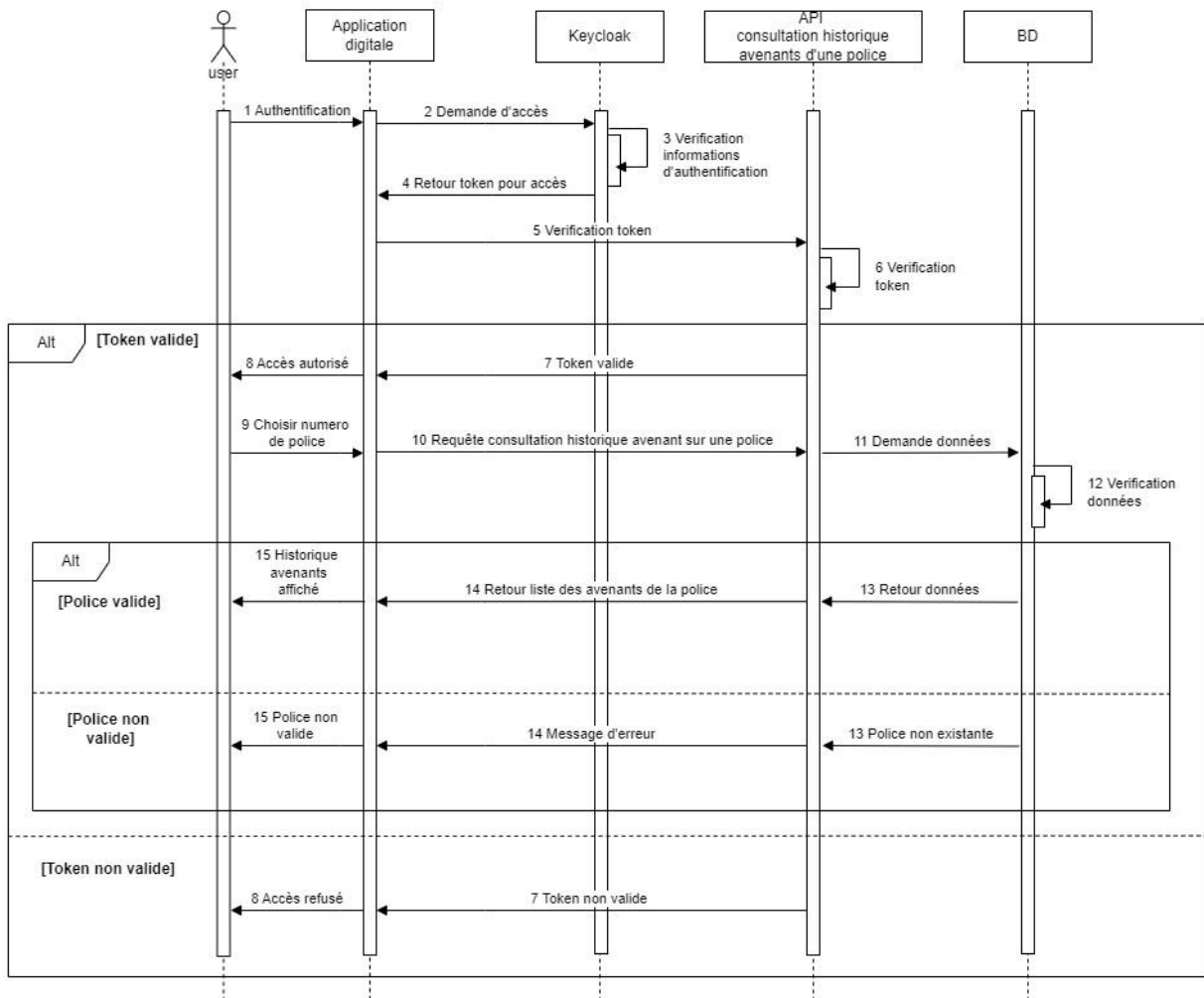


Figure 8. Le diagramme de séquence de consulter l'historique des avenants sur une police

La figure [9] montre le diagramme de séquences de cas d'utilisation « Consulter les types d'avenants de modification éligibles sur une police ».

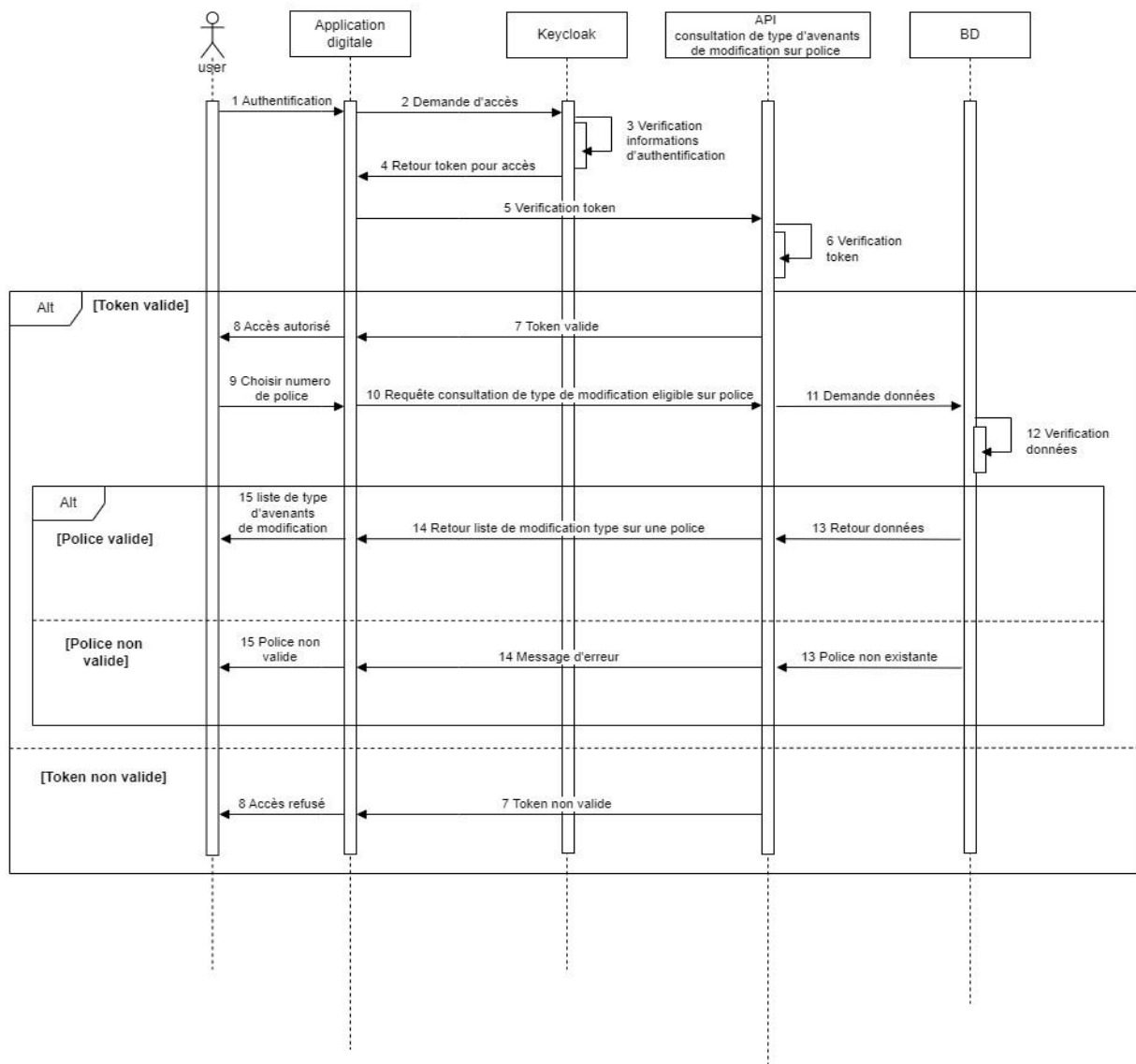


Figure 9. Le diagramme de séquence de consulter les types d'avenants de modification éligibles sur une police

3.4 Réalisation et tests

Une fois la conception du sprint 1 achevée, cette section aborde le dernier aspect de ce sprint : la phase de réalisation. Cette étape correspond à la mise en œuvre des travaux accomplis, où les idées et concepts développés lors de la phase de conception se concrétisent et se traduisent par des résultats tangibles. Elle marque la conclusion du sprint et permet de valider les progrès du projet.

3.4.1 Configuration des outils

3.4.1.1 Configuration de l'outil Keycloak

L'outil utilisé pour gérer l'accès aux services web de Solife est Keycloak, qui simplifie le processus d'authentification en prenant en charge à la fois les identités et les accès. La gestion des utilisateurs se décompose en deux étapes principales :

1. Création des utilisateurs : Cette première étape consiste à définir les utilisateurs ayant accès à l'API Solife. Différents types d'utilisateurs peuvent être configurés, tels que `user_dig`, le broker, le customer etc. Chaque rôle possède des privilèges et des droits spécifiques en fonction des actions qu'il est autorisé à accomplir.
2. Attribution des privilèges : La deuxième étape consiste à attribuer les privilèges nécessaires à chaque utilisateur en fonction de son rôle et de ses responsabilités. Ces privilèges déterminent les actions que l'utilisateur peut effectuer sur l'API, telles que la consultation ou la création de contrats.

Exemple d'utilisateur : Prenons l'exemple de l'utilisateur **user_dig**, qui est autorisé à créer et consulter des contrats.

La figure [10] illustre la création de l'utilisateur **user_dig** dans Keycloak.

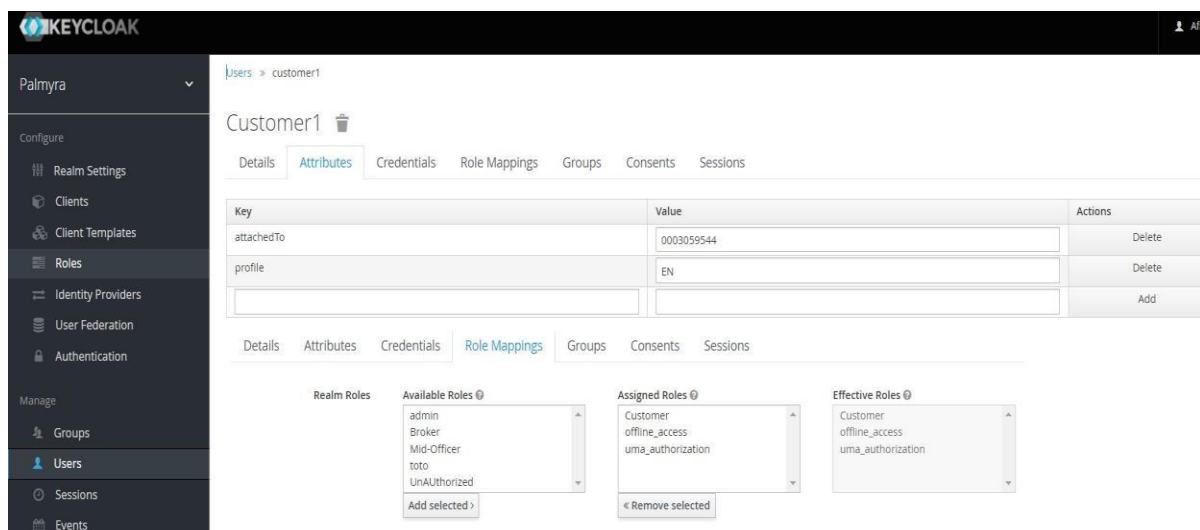


Figure 10. Configuration de l'outil Keycloak

3.4.1.2 Configuration de l'outil Postman

Pour tester nos APIs, nous avons utilisé l'outil Postman. Afin de l'intégrer correctement dans notre projet, une configuration a été nécessaire pour générer un token d'authentification, assurant ainsi une connexion sécurisée.

Cette configuration a impliqué la saisie des paramètres requis ainsi que du nom d'utilisateur créé au préalable dans Keycloak. Ces informations permettent à Postman de générer un token valide, lequel est ensuite utilisé pour accéder aux APIs de manière sécurisée.

La figure [11] montre le processus de génération du token via Postman.

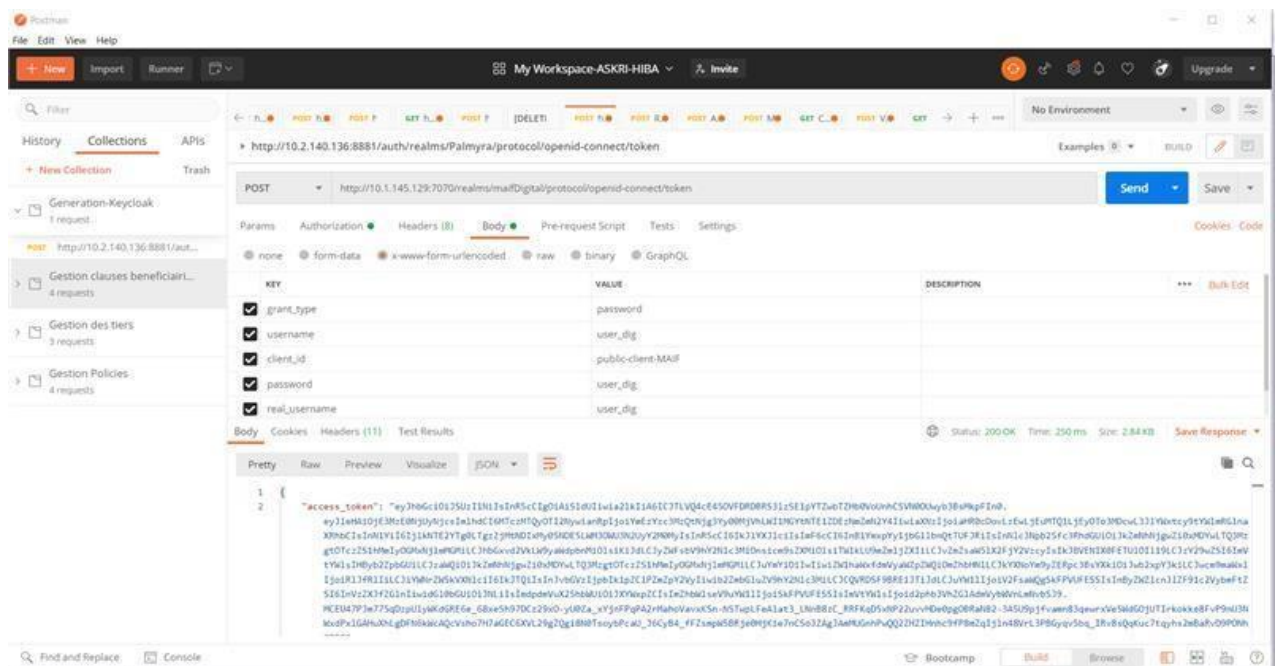


Figure 11. Génération du token via Postman

3.4.1.3 Autenticazione via interfaccia digitale

La figure suivante présente l'interface de **login** et d'**authentification** via l'application digitale. Cette interface permet aux utilisateurs de s'identifier et d'accéder aux fonctionnalités sécurisées de l'application en fournissant leurs informations d'authentification.

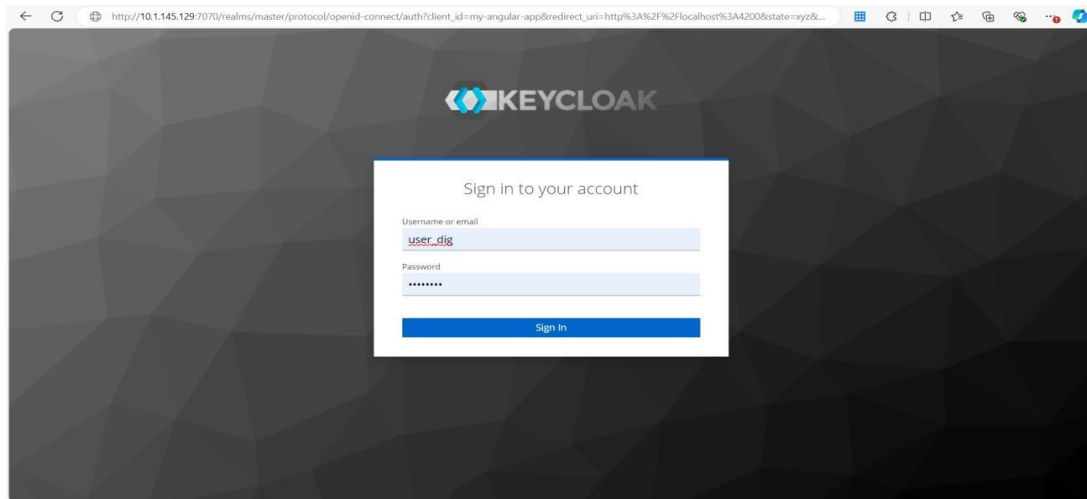


Figure 12. Interface digitale authentication

3.4.2 Tests manuels de sprint 1

3.4.2.1 Consulter une police d'assurance

Pour tester l'API "Consulter une police d'assurance", l'acteur effectue une requête de type "GET" en utilisant l'URL

"http://localhost:31080/solife/services/api/policies/identi_policy_number", en spécifiant l'identifiant de la police et en insérant le token généré.

a) Consulter une police pour user_dig via postman

La figure suivante illustre le test manuel de la consultation d'une police via Postman.

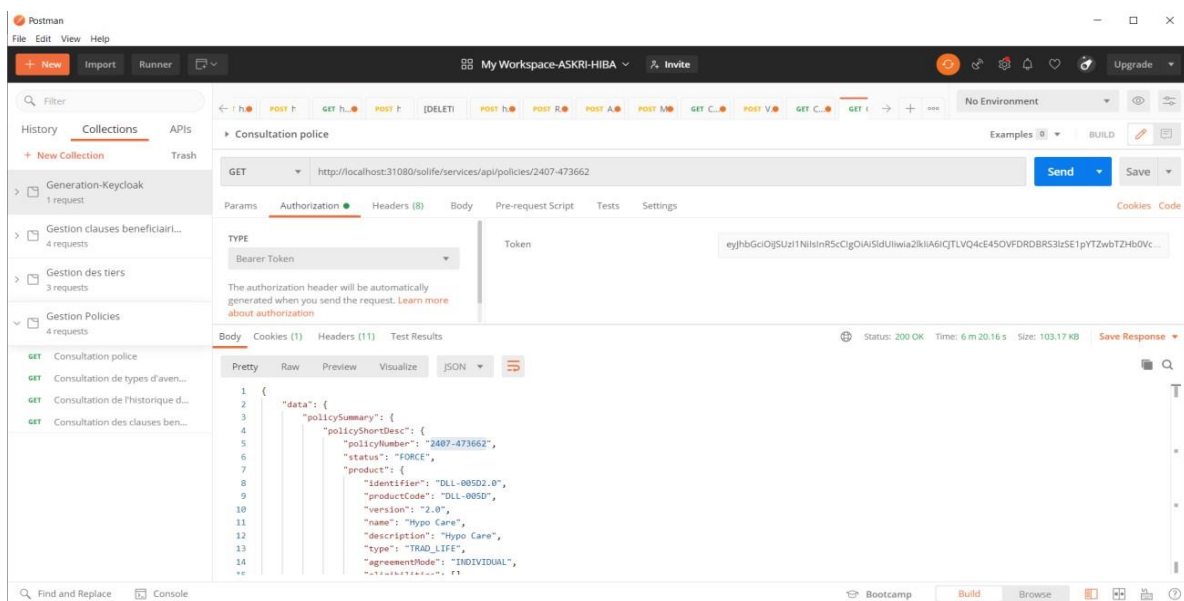


Figure 13. Test API consultation police avec user_dig via Postman

b) Consulter une police pour user_dig via interface digitale

La figure [14] montre l'interface digitale de consultation d'une police

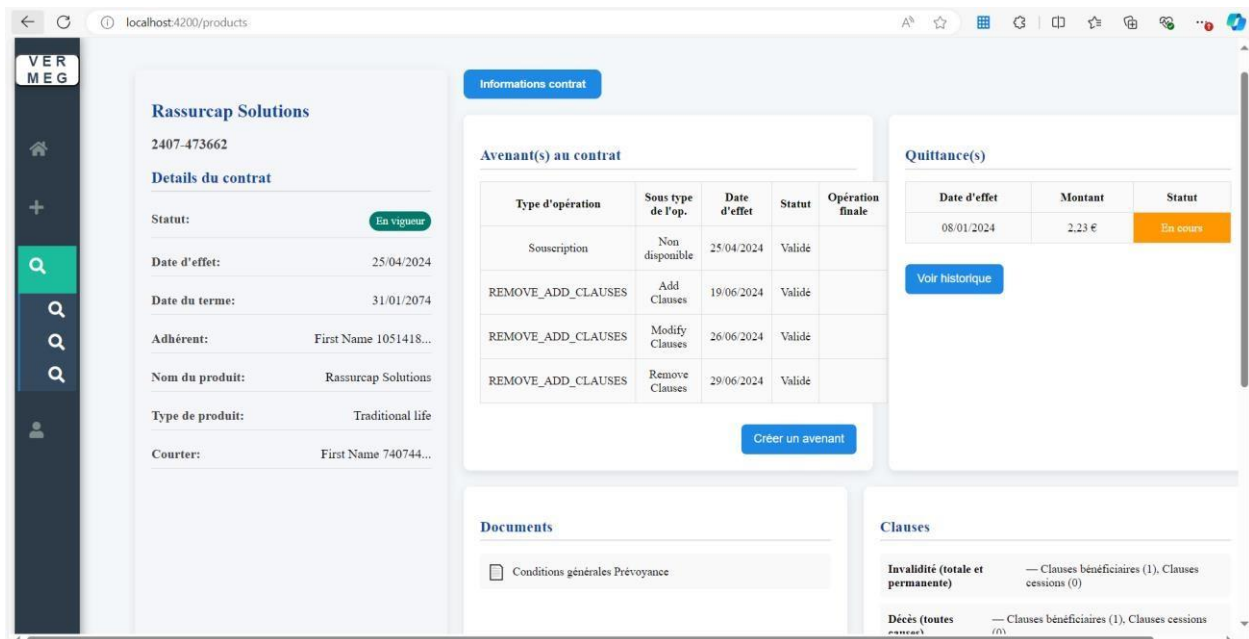


Figure 14. Interface digitale consulter police

3.4.2.2 Consulter l'historique des avenants sur une police

Pour tester l'API "Consulter historique d'avenants sur une police d'assurance", l'acteur envoie une requête de type "Get" avec le lien

"http://localhost:31080/solife/services/api/policies/ident_policy_number/endorsements" tout en spécifiant l'identifiant de la police et en insérant le token préalablement généré.

a) Consulter historique des avenants sur une police pour user_dig via postman

La figure suivante représente le test manuel de la consultation de l'historique des avenants sur une police via Postman

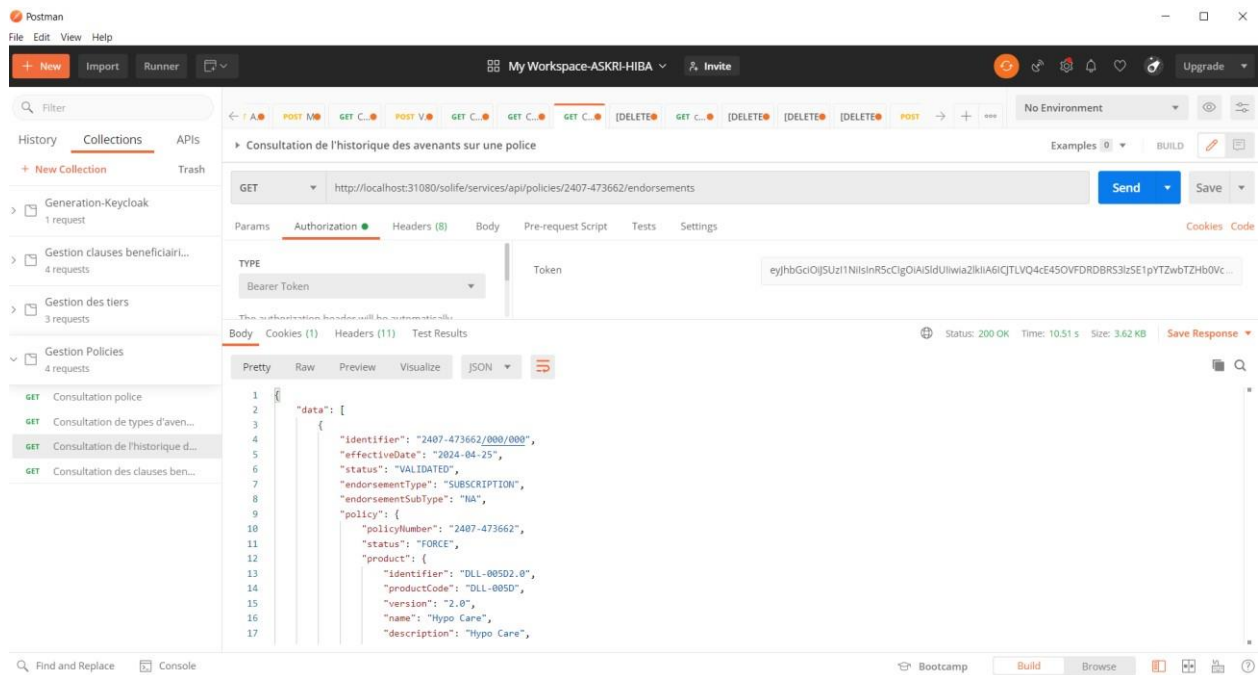


Figure 15. Test API consultation historique avenants sur police avec user_dig via Postman

b) Consulter historique des avenants sur une police pour user_dig via interface digitale

La figure [16] montre l'interface digitale de consultation de l'historique des avenants d'une police

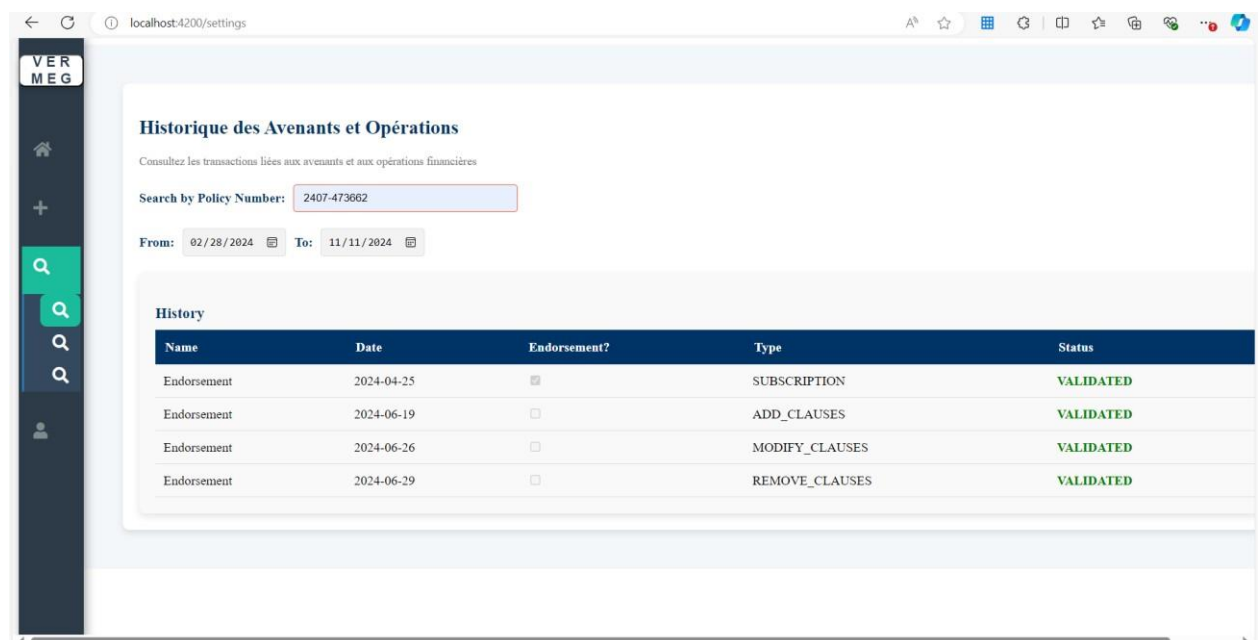


Figure 16. Test consultation historique avenants sur police via Interface digitale

3.4.2.3 Consulter les types d'avenants de modification éligibles sur une police

Pour tester l'API "Consulter les types d'avenants de modification éligibles sur une police d'assurance", l'acteur envoie une requête de type "Get" avec le lien "http://localhost:31080/solife/services/api/endorsements/available-modification-types/identi_policy_number" tout en spécifiant l'identifiant de la police et en insérant le token préalablement généré.

a) Consulter les types d'avenants de modification sur une police pour user_dig via postman

La figure suivante représente le test manuel de la consultation des types d'avenants de modification éligibles sur une police via Postman

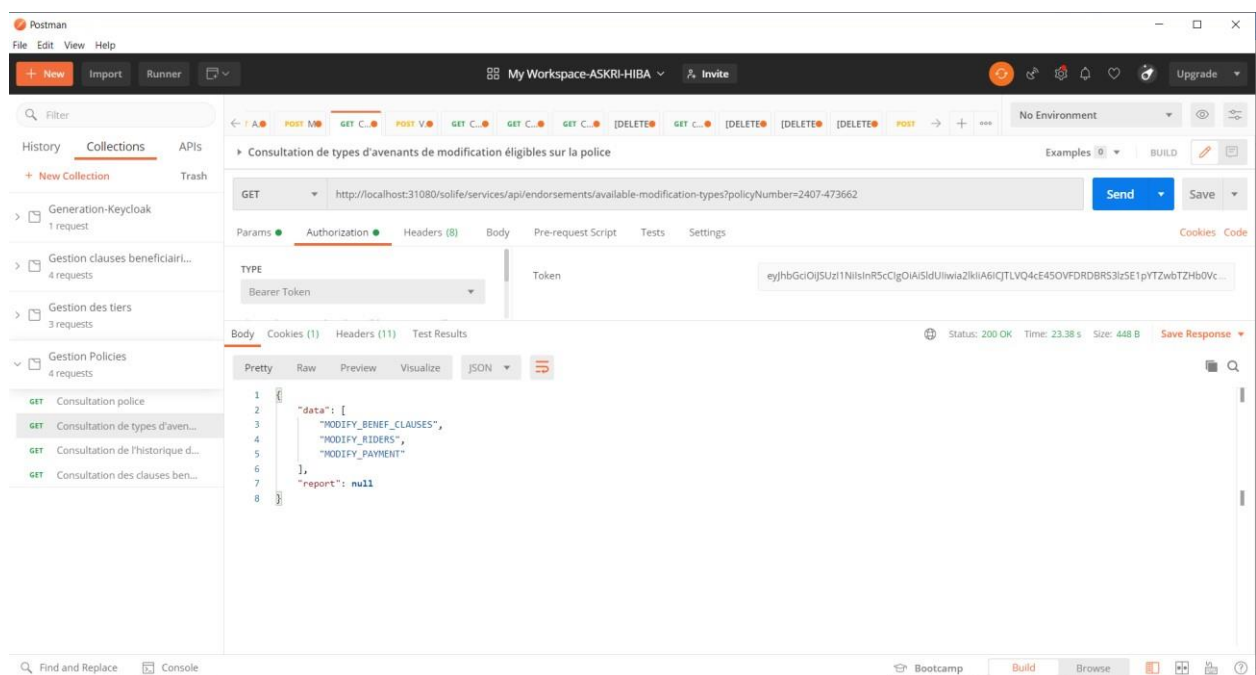


Figure 17. Test API consultation types d'avenants de modification éligibles sur police via Postman

b) Consulter les types d'avenants de modification sur une police pour user_dig via interface digitale

La figure suivante représente l'interface digitale de la consultation des types d'avenants de modification éligibles sur une police :



Figure 18. Test consultation types de modification éligibles sur police via Postman

3.4.2.4 Consulter les clauses bénéficiaires sur une police

Pour tester l'API "Consulter les clauses bénéficiaires sur une police d'assurance", l'acteur lance une requête de type "Get" avec le lien "http://localhost:31080/solife/ services/api/ policies/policy_number/beneficiaryclauses" tout en introduisant l'identifiant de la police et en insérant le token précédemment généré.

a) Consulter les clauses bénéficiaires sur une police pour user_dig via postman

La figure [19] montre le test manuel de la consultation des clauses bénéficiaires sur une police via Postman

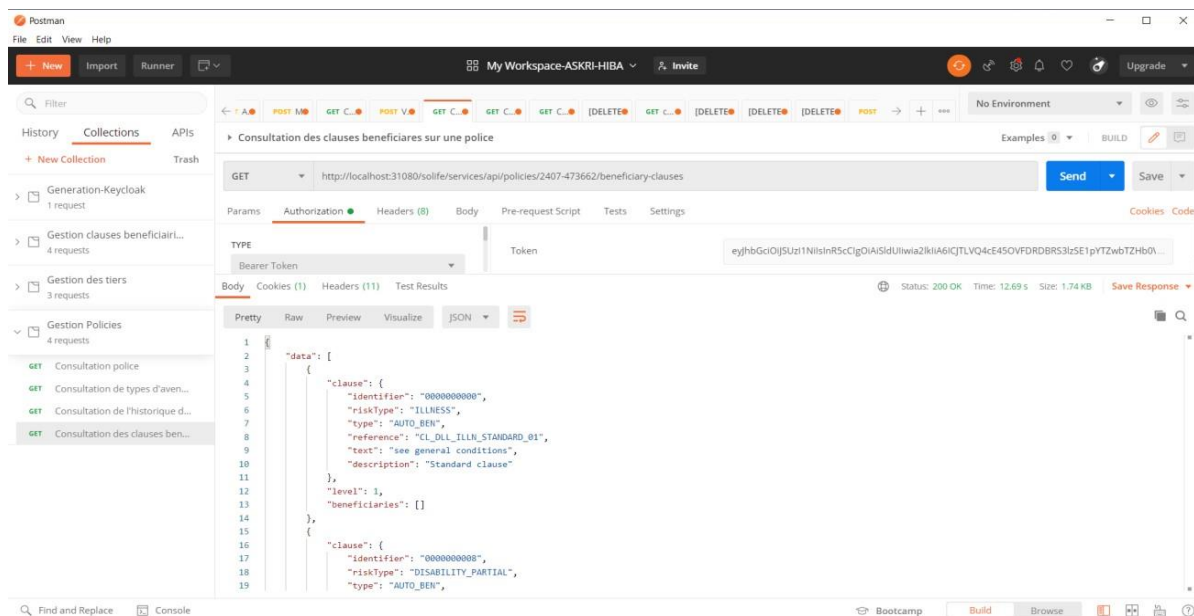


Figure 19. Test API consultation des clauses bénéficiaires sur police via Postman

b) Consulter les clauses bénéficiaires sur une police pour user_dig via interface digitale

La figure suivante représente l'interface digitale de la consultation des clauses bénéficiaires sur une police

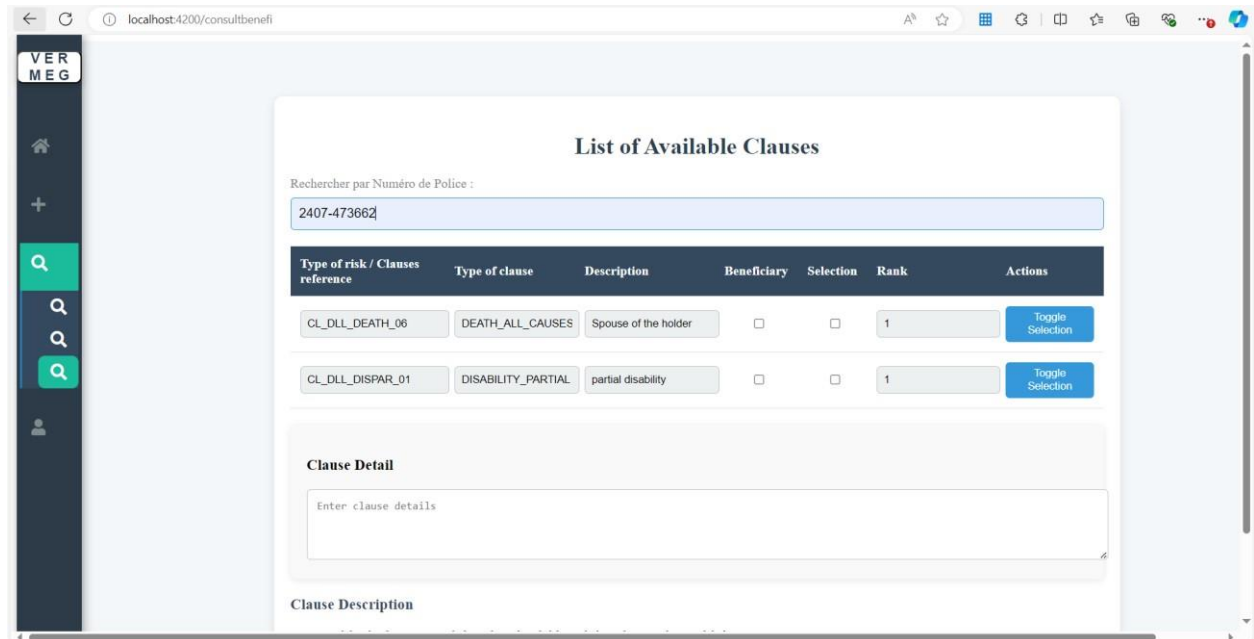


Figure 20. Test consultation des clauses bénéficiaires sur police via Interface digitale

Conclusion

Tout au long de ce chapitre, nous avons présenté une vue détaillée du premier sprint en commençant par la gestion du backlog, la phase de conception, la configuration de Keycloak, ainsi que l'implémentation des tests manuels et le développement des interfaces digitales.

Chapitre 4 : Sprint 2 - Étude et réalisation

Introduction

Ce chapitre consiste à détailler les différentes étapes de la réalisation et de l'exécution du deuxième sprint du projet intitulé "Gestion des clauses bénéficiaires".

4.1 Définition des tâches

Le tableau suivant rassemble les différentes fonctionnalités du deuxième sprint.

Tableau 8.Backlog du sprint 2

ID	Tâche	User story	Priorité
1	Encoder un administratif de Add Clauses avenant subtype	En tant qu'un testeur QA ou Agent , je peux encoder un avenant administratif de subtypeAdd Clauses.	Élevée
2	Encoder un administratif de Modify Clauses avenant subtype	En tant qu'un testeur QA ou Agent , je peux encoder un avenant administratif de subtypeModify Clauses.	Élevée
3	Encoder un administratif de Remove Clauses avenant subtype	En tant qu'un testeur QA ou Agent , je peux encoder un avenant administratif de subtypeRemove Clauses.	Élevée
4	Valider un avenant de gestion des clauses bénéficiaires.	En tant qu'un testeur QA ou Agent , je peux valider un avenant de gestion des clauses bénéficiaires.	Élevée

4.2 Cas d'utilisation détaillés

Au cours de cette activité nous allons détailler les cas d'utilisation les plus prioritaires en décrivant, dans chaque cas, les préconditions et les post conditions ainsi que le scénario principal, et éventuellement les exceptions liées à ce cas.

4.2.1 Diagramme de cas d'utilisation du sprint 2

Le diagramme de cas d'utilisation ci-dessous montre les interactions entre l'acteur et le système pour les fonctionnalités du deuxième sprint, portant sur la gestion des clauses bénéficiaires.

La figure suivante illustre ces interactions :

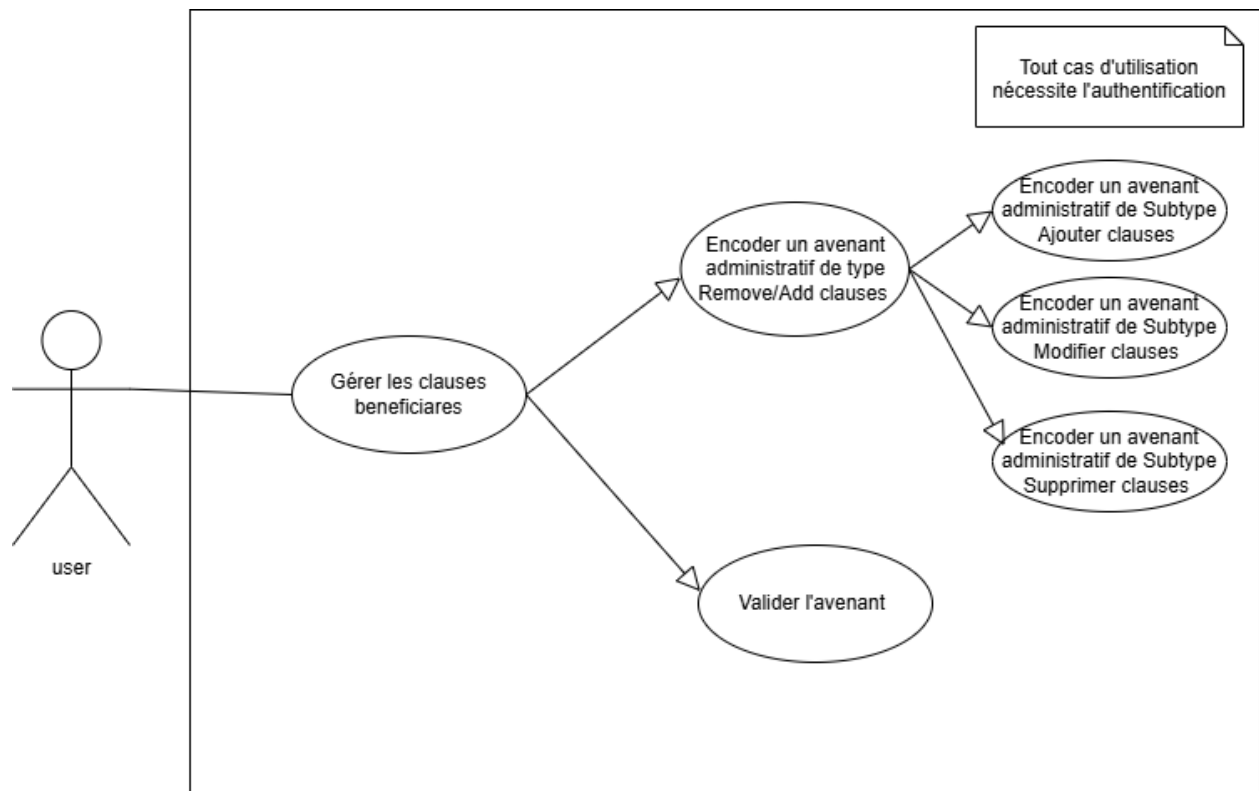


Figure 21. Le diagramme de cas d'utilisation du sprint 2 du projet

4.2.2 Détails des cas d'utilisation

4.2.2.1 Encoder un avenant administratif de subtype add clauses

Ce tableau résume le scénario du cas d'utilisation intitulé "**Encoder un avenant administratif de subtype Add clauses**", incluant quelques scénarios d'exception. Une fois authentifié, l'acteur peut procéder à l'encodage de l'avenant.

Tableau 9. Le scénario de cas d'utilisation « Encoder un avenant administratif Add clauses »

Titre	Encoder un avenant de subtypeAdd clauses
Acteurs	• Testeur QA • Agent
Résumé	• L'utilisateur saisit le numéro de la police. • L'utilisateur sélectionne le type d'avenant pour la gestion des clauses. • L'utilisateur renseigne le sous-type d'avenant pour l'ajout de clauses. • L'acteur entre la date de création de l'avenant.
Précondition	• L'utilisateur est authentifié. • Opération de création est sélectionnée.
Postcondition	• L'avenant est créé.
Scénario Principal	1. L'utilisateur saisit les informations relatives à l'avenant. 2. L'utilisateur confirme l'opération. 3. Le système valide les données saisies. 4. Le système retourne le statut 200 OK accompagné d'identifiant d'avenant créé.
Scénario d'exception	1. Date de création de l'avenant antérieure à la date de création de la police : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est affiché, indiquant que la date de création est invalide. 2. Numéro de police introuvable dans la base de données Solife : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est généré, précisant que le numéro de police est incorrect. 3. Date de création postérieure à la date actuelle : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur signale que la date saisie est invalide.

4.2.2.2 Encoder un avenant administratif de subtypeModify clauses

Ce tableau présente un résumé du scénario lié au cas d'utilisation intitulé "**Encoder un avenant administratif de subtypeModify Clauses**", tout en incluant certains scénarios d'exception. Une fois authentifié, l'acteur est habilité à procéder à l'encodage de l'avenant.

Tableau 10. Le scénario de cas d'utilisation « Encoder un avenant administratif Modify clauses»

Titre	Encoder un avenant de subtypeModify clauses
Acteurs	•Testeur QA •Agent
Résumé	• L'utilisateur saisit le numéro de la police. • L'utilisateur sélectionne le type d'avenant pour la gestion des clauses. • L'utilisateur renseigne le sous-type d'avenant pour la modification de clauses. • L'acteur entre la date de création de l'avenant.
Précondition	• L'utilisateur est authentifié. •Opération de création est sélectionnée.
Postcondition	•L'avenantestcréé.
Scénario Principal	1. L'utilisateur saisit les informations relatives à l'avenant. 2. L'utilisateurconfirme l'opération. 3. Le système valide les données saisies. 4. Le système retourne le statut 200 OK accompagné d'identifiant d'avenant crée.
Scénario d'exception	1. Date de création de l'avenant antérieure à la date de création de la police : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est affiché, indiquant que la date de création est invalide. 2. Numéro de police introuvable dans la base de données Solife : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est généré, précisant que le numéro de police est incorrect.
	3.Date de création postérieure à la date actuelle : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur signale que la date saisie est invalide.

4.2.2.3 Encoder un avenant administratif de subtype remove clauses

Ce tableau présente un résumé du scénario lié au cas d'utilisation intitulé "**Encoder un avenant administratif de subtype Remove Clauses**", en incluant quelques scénarios d'exception. Une fois authentifié, l'acteur est habilité à effectuer l'encodage de l'avenant.

Tableau 11. Le scénario de cas d'utilisation « Encoder un avenant administratif Remove clauses »

Titre	Encoder un avenant de subtype remove clauses
Acteurs	• Testeur QA • Agent
Résumé	• L'utilisateur saisit le numéro de la police. • L'utilisateur sélectionne le type d'avenant pour la gestion des clauses. • L'utilisateur renseigne le sous-type d'avenant pour remove de clauses. • L'utilisateur entre la date de création de l'avenant.
Précondition	• L'utilisateur est authentifié. • Opération de création est sélectionnée.
Postcondition	• L'avenant est créé.
Scénario Principal	1. L'utilisateur saisit les informations relatives à l'avenant. 2. L'utilisateur confirme l'opération. 3. Le système valide les données saisies. 4. Le système retourne le statut 200 OK accompagné d'identifiant d'avenant crée.
Scénario d'exception	1. Date de création de l'avenant antérieure à la date de création de la police : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est affiché, indiquant que la date de création est invalide. 2. Numéro de police introuvable dans la base de données Solife : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est généré, précisant que le numéro de police est incorrect. 3. Date de création postérieure à la date actuelle : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur signale que la date saisie est invalide.

4.2.2.4 Valider un avenant administratif de gestion des clauses bénéficiaires

Ce tableau présente un résumé du scénario associé au cas d'utilisation intitulé "**Valider l'avenant de gestion de clauses bénéficiaires**", tout en incluant quelques scénarios d'exception. Une fois authentifié, l'acteur est en mesure de procéder à la validation de l'avenant.

Tableau 12. Le scénario de cas d'utilisation « Valider un avenant administratif de gestion clauses bénéficiaires »

Titre	Valider un avenant de gestion des clauses bénéficiaires
Acteurs	• Testeur QA • Agent
Résumé	• L'utilisateur saisit le numéro de l'avenant.
Précondition	• L'utilisateur est authentifié. • Opération de validation est sélectionnée.
Postcondition	• L'avenant est validé.
Scénario Principal	1- L'utilisateur introduit le numéro relatif à l'avenant. 2- L'utilisateur valide l'opération. 3- Le système vérifie la saisie. 4- Le système affiche la statut 200 ok.
Scénario d'exception	1- Numéro de police introuvable dans la base de données Solife : a. Le webservice retourne une réponse avec le statut 400. b. Un message d'erreur est généré, précisant que le numéro de police est incorrect.

4.3 Conception technique

4.3.1 Diagrammes de séquences spécifiques

Le diagramme de séquences, tel que décrit précédemment, permet de modéliser les scénarios de chaque cas d'utilisation tout en mettant en évidence la chronologie des interactions entre les différents objets. Dans ce chapitre, nous présenterons plusieurs diagrammes de séquences pour illustrer ces interactions.

4.3.1.1 Diagramme de séquences « Encoder un avenant administratif de add clauses »

La figure suivante illustre le diagramme de séquences associé au cas d'utilisation intitulé "Encoder un avenant administratif de subtypeAdd Clauses". Ce diagramme met en évidence les interactions et la chronologie des opérations nécessaires pour réaliser cet encodage.

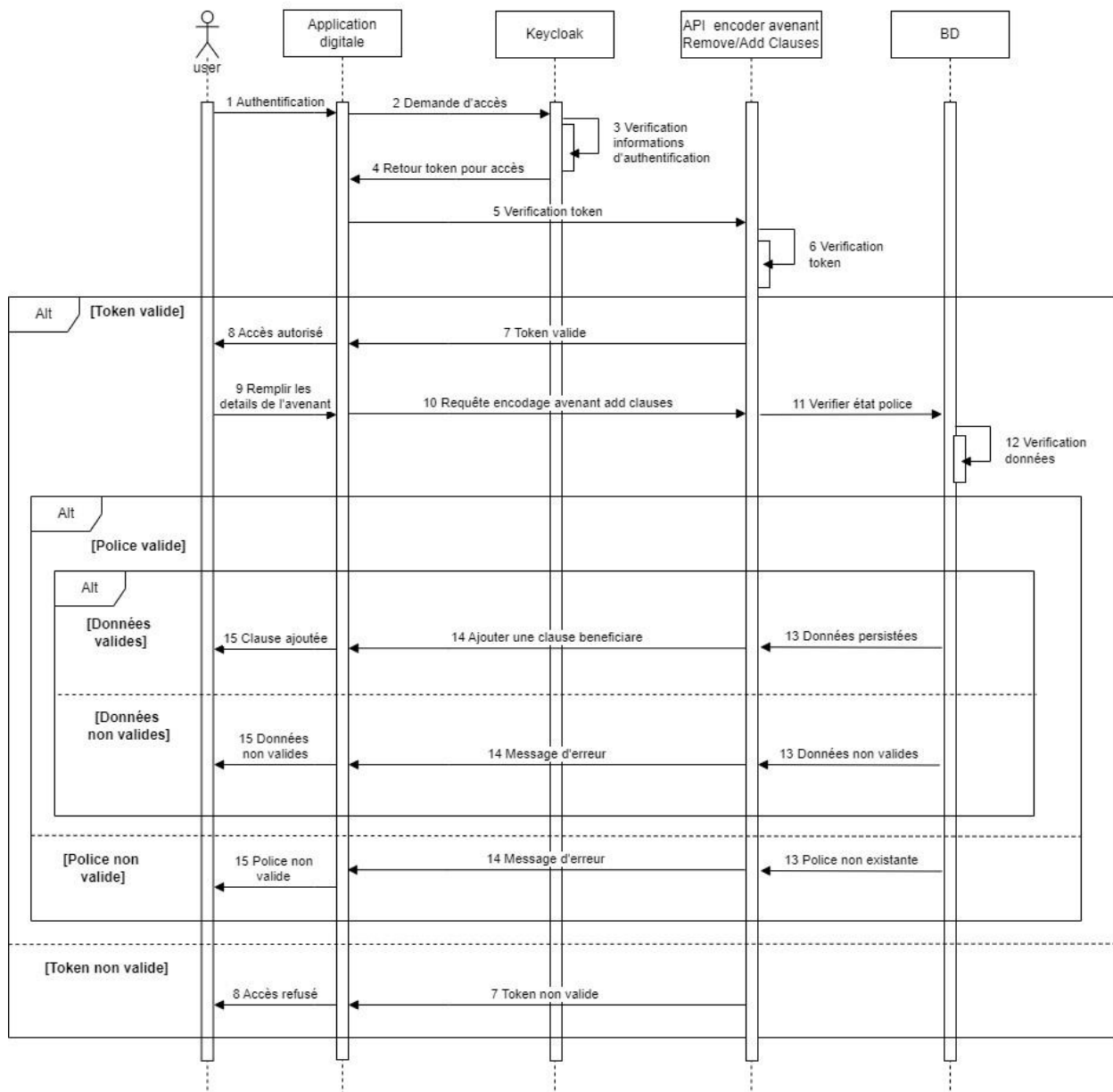


Figure 22. Le diagramme de séquence de « add clauses »

4.3.1.2 Diagramme de séquences « Valider un avenant administratif de gestion des clauses »

La figure suivante présente le diagramme de séquences relatif au cas d'utilisation intitulé **"Valider un avenant administratif de gestion de clauses bénéficiaires"**. Ce diagramme illustre la chronologie des interactions et des opérations nécessaires pour effectuer la validation de l'avenant.

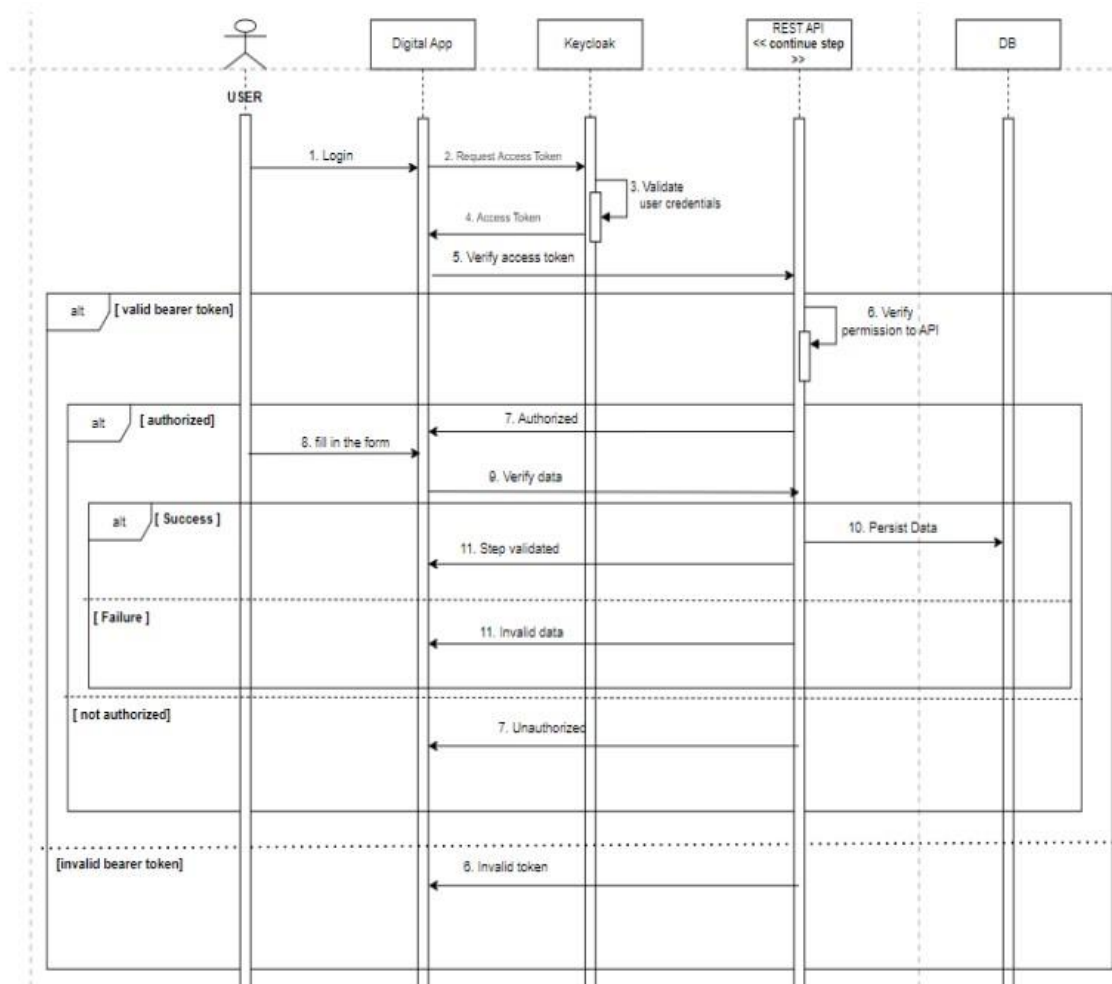


Figure 23. Le diagramme de séquence de « valider un avenant »

4.4 Réalisation et tests

Une fois la conception de notre Sprint 2 élaborée, nous entamons cette section en abordant le dernier aspect de ce sprint. L'objectif est de présenter la phase de réalisation, qui représente la concrétisation finale du résultat obtenu au cours de ce sprint.

4.4.1 Tests manuels de sprint 2

4.4.1.1 Encoder un avenant administratif de type add clauses

a) Encoder un avenant administratif de type add clauses pour user_dig via postman

Pour tester l'API "Encoder un avenant administratif de subtype add clauses", l'acteur envoie une requête de type "POST" à l'URL suivante :

<http://localhost:31080/solife/services/api/endorsements/addclauses>, avec le body suivant :

```
{
  "policyNumber": "2407-473662",
  "endorsementCreationDate": "2024-06-19",
  "modificationTypes": [
    "MODIFY_BENEF_CLAUSES"
  ],
  "beneficiaryClauses": [
    {
      "clause": {
        "identifiant": "0000830001",
        "riskType":
"DEATH_ALL_CAUSES",
        "type": "AUTO_BEN",
        "reference":
"CL_DLL_DEATH_04",
        "text": "Children of the holder",
        "description": "Children of the holder"
      },
      "level": 2,
      "beneficiaries": [
        {
          "beneficiary": "5876791",
          "split": 1,
          "irrevocable": false,
          "accepting": false
        }
      ]
    }
  ]
}
```

Le token généré préalablement doit également être ajouté dans les en-têtes de la requête. La figure suivante illustre le test manuel de la création de l'avenant add clauses via Postman.

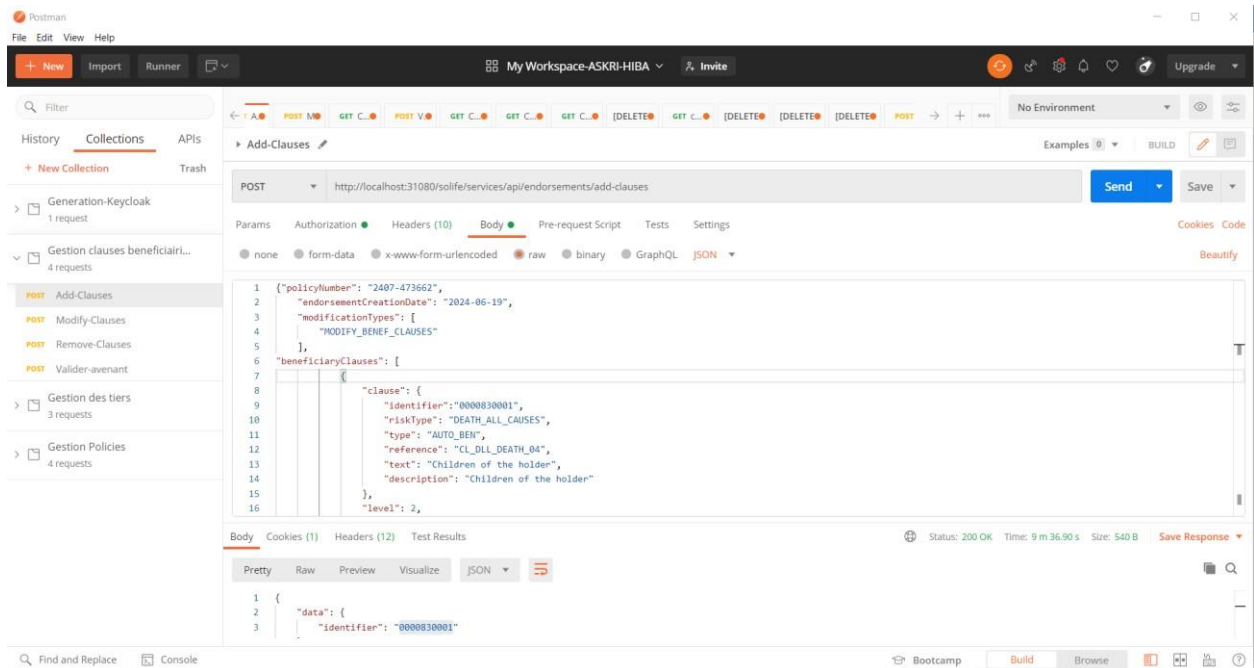


Figure 24. Test API « Encoder un avenant administratif add clauses » via Postman

b) Encoder un avenant administratif de type add clauses pour user_dig via interface digitale

Les deux figures suivantes illustrent les interfaces digitales de création de l'avenant add clauses bénéficiaires.

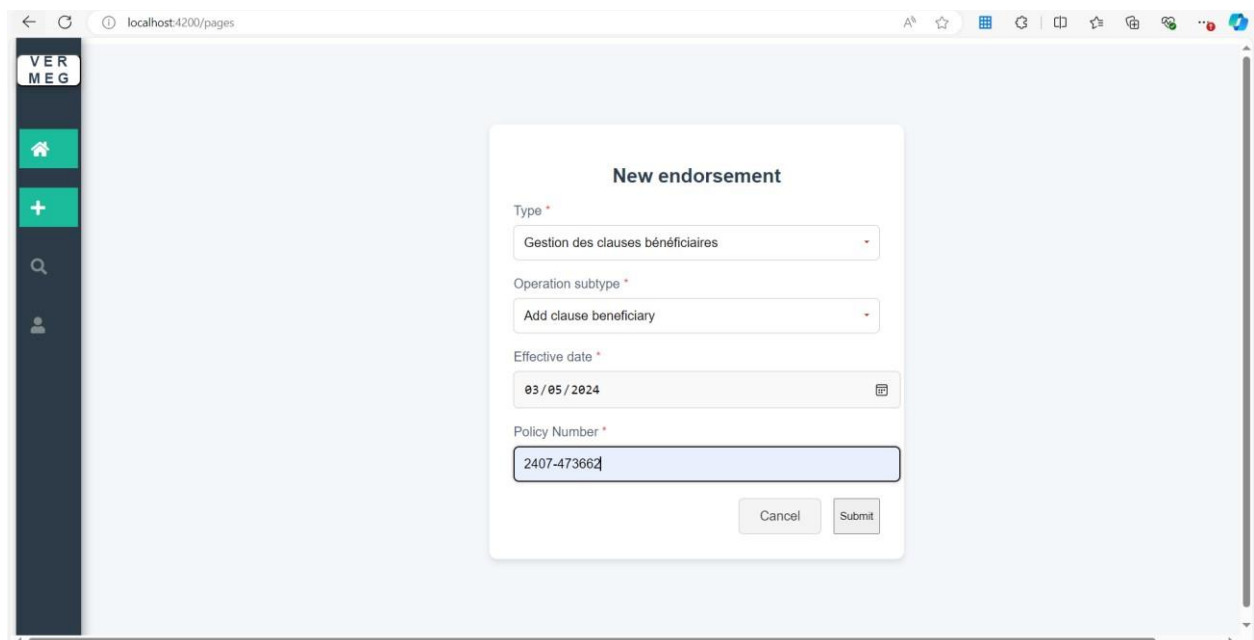


Figure 25. Interface digitale « Encoder un avenant administratif add clauses »

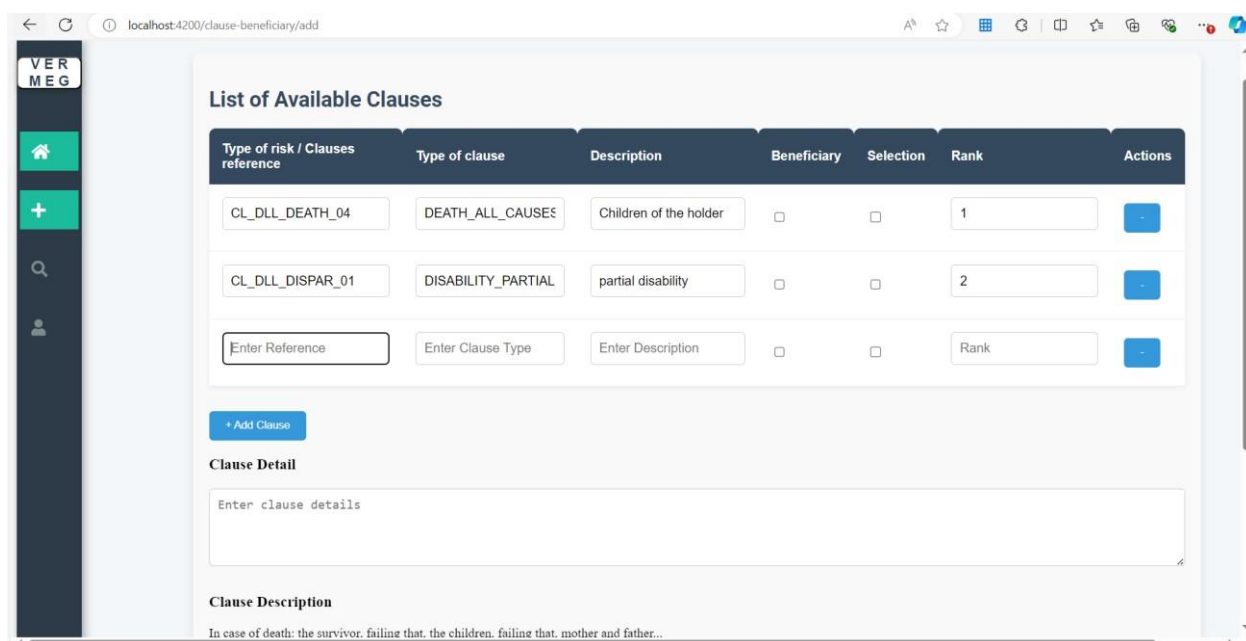


Figure 26. Interface digitale « Encoder un avenant administratif add clauses »

4.4.1.2 Encoder un avenant administratif de type modify clauses

a) Encoder un avenant administratif de type modify clauses pour user_dig via postman

Pour tester l'API "Encoder un avenant administratif de subtypemodify clauses", l'acteur envoie une requête de type "POST" à l'URL suivante :

<http://localhost:31080/solife/services/api/endorsements/modify-clauses>, avec le body suivant:

```
{
  "policyNumber": "2407-473662",
  "endorsementCreationDate": "2024-06-26",
  "modificationTypes": [
    "MODIFY_BENEF_CLAUSES"
  ],
  "beneficiaryClauses": [
    {
      "clause": {
        "identifiant": "0000830001",
        "riskType":
"DEATH_ALL_CAUSES",
        "type": "AUTO_BEN",
```

```

    "reference":
"CL_DLL_DEATH_04",
    "text": "Children of the holder",
    "description": "Children of the holder"
  },
  "level": 2,
  "beneficiaries": [
    {
      "beneficiary": "5876791",
      "split": 1,
      "irrevocable": false,
      "accepting": false
    }
  ]
}
]
}
]
}

```

Le token généré préalablement doit également être ajouté dans les en-têtes de la requête. La figure suivante présente le test manuel de la création de l'avenant modify clauses via Postman.

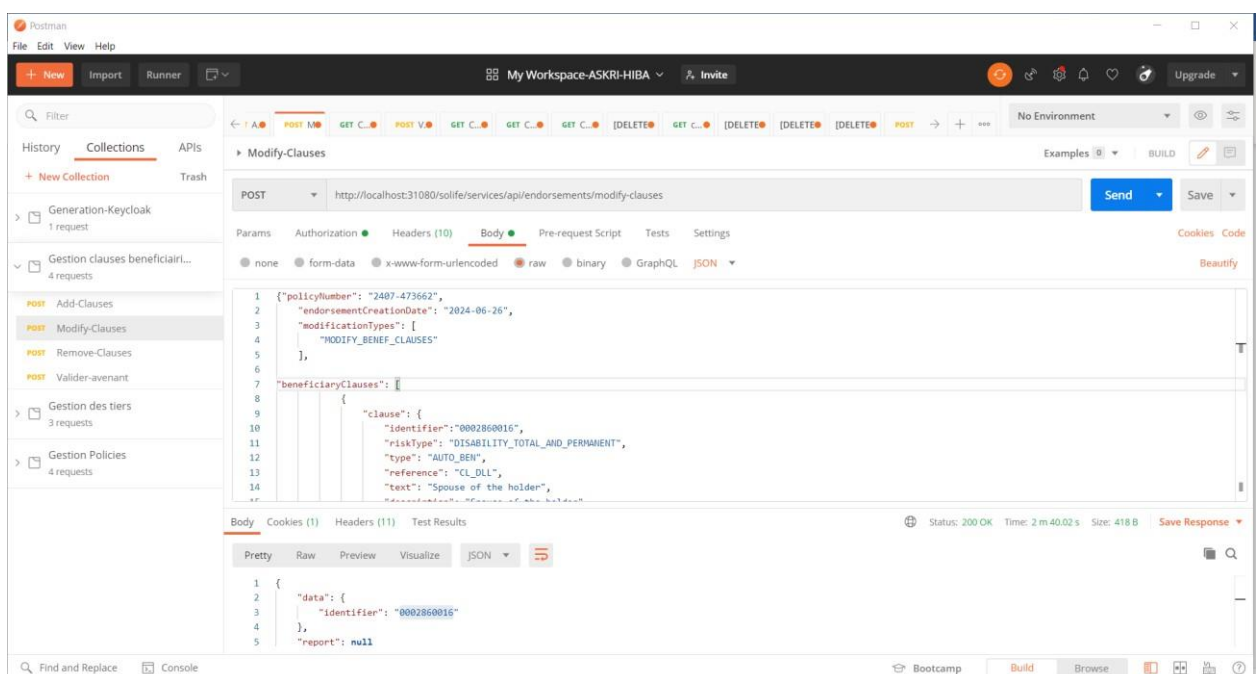


Figure 27. Test API « Encoder un avenant administratif modify clauses via Postman »

b) Encoder un avenant administratif de type modify clauses pour user_dig via interface digitale

Les deux figures suivantes illustrent les interfaces digitales de création de l'avenant modify clauses bénéficiaires.

The screenshot shows a web browser window with the address bar displaying 'localhost:4200/pages'. On the left, there is a dark sidebar with a 'VER MEG' logo and several icons. The main content area features a light blue background with a central white form titled 'New endorsement'. The form contains the following fields: 'Type *' with a dropdown menu showing 'Gestion des clauses bénéficiaires'; 'Operation subtype *' with a dropdown menu showing 'Update clause beneficiary'; 'Effective date *' with a date input field showing '06/26/2024' and a calendar icon; and 'Policy Number *' with a text input field showing '2407-473662'. At the bottom right of the form are 'Cancel' and 'Submit' buttons.

Figure 28.Interface digitale « Encoder un avenant administratif modify clauses »

The screenshot shows a web browser window with the address bar displaying 'localhost:4200/clause-beneficiary/update'. On the left, there is a dark sidebar with a 'VER MEG' logo and several icons. The main content area features a light blue background with a central white form titled 'List of Available Clauses'. The form contains a table with the following columns: 'Type of Risk / Clauses Reference', 'Type of Clause', 'Description', 'Beneficiary', 'Selection', 'Rank', and 'Actions'. The table has two rows of data. The first row shows 'CL_DLL_DEAT', 'DEATH_AL', 'Children of the', a checkbox, a checked checkbox, and '1'. The second row shows 'CL_DLL_DISP', 'DISABILIT', 'partial disability', a checkbox, an unchecked checkbox, and '2'. To the right of the table, there are two 'Edit Clause' sections. The first section has input fields for 'CL_DLL_DEATH_04', 'DEATH_ALL_CAUSES', and 'Children of the holder', a checkbox for 'Beneficiary', and a text input field with '1'. The second section has input fields for 'CL_DLL_DEATH_04' and 'DEATH_ALL_CAUSES'. Below each 'Edit Clause' section is a 'Save Clause' button.

Figure 29.Interface digitale « Encoder un avenant administratif modify clauses »

4.4.1.3 Encoder un avenant administratif de type remove clauses

a) Encoder un avenant administratif de type remove clauses pour user_dig via postman

Pour tester l'API "Encoder un avenant administratif de subtyperremove clauses", l'acteur

envoie une requête de type "POST" à l'URL suivante :

<http://localhost:31080/solife/services/api/endorsements/remove-clauses>, avec le body suivant :

```
{
  "policyNumber": "2407-473662",
  "endorsementCreationDate": "2024-06-29",
  "modificationTypes": [
    "MODIFY_BENEF_CLAUSES"
  ],
  "beneficiaryClauses": [
    {
      "clause": {
        "identifiant": "0000830001",
        "riskType":
"DEATH_ALL_CAUSES",
        "type": "AUTO_BEN",
        "reference":
"CL_DLL_DEATH_04",
        "text": "Children of the holder",
        "description": "Children of the holder"
      },
      "level": 2
    }
  ]
}
```

Le token généré préalablement doit également être ajouté dans les en-têtes de la requête. La figure suivante présente le test manuel de la création de l'avenant remove clauses via Postman.

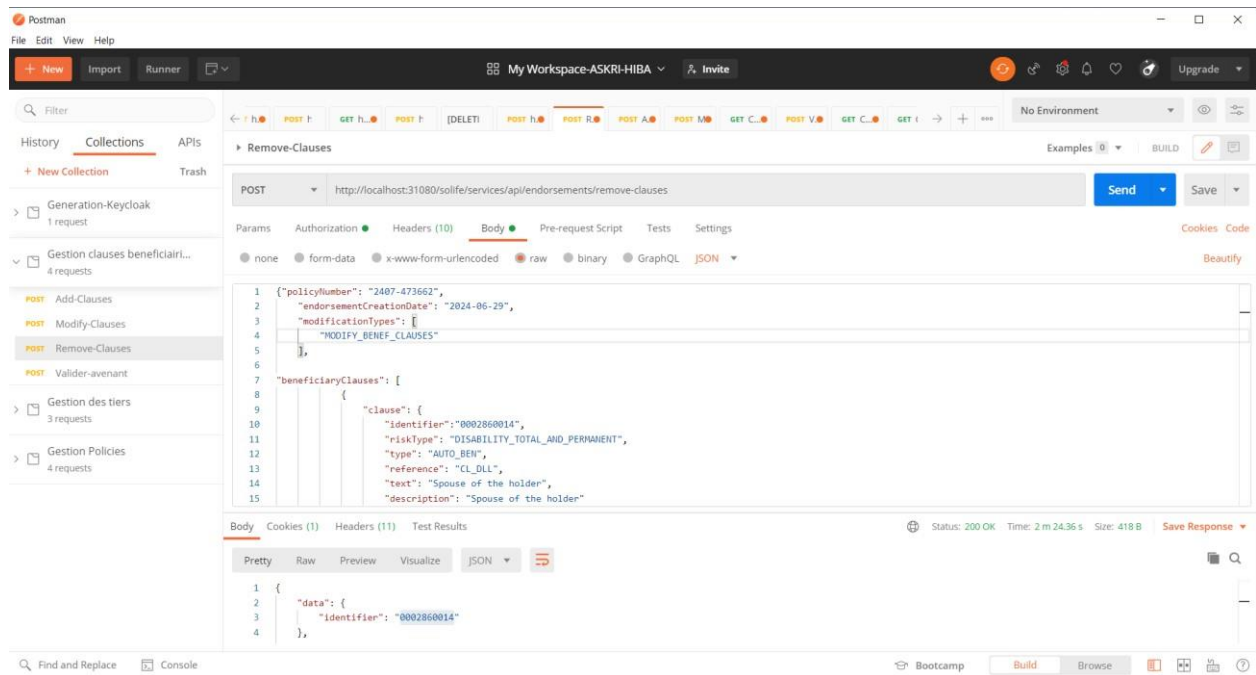


Figure 30. Test API « Encoder un avenant administratif remove clauses » via postman

b) Encoder un avenant administratif de type remove clauses pour user_dig via interface digitale

Les deux figures suivantes illustrent les interfaces digitales de création de l'avenant remove clauses bénéficiaires.

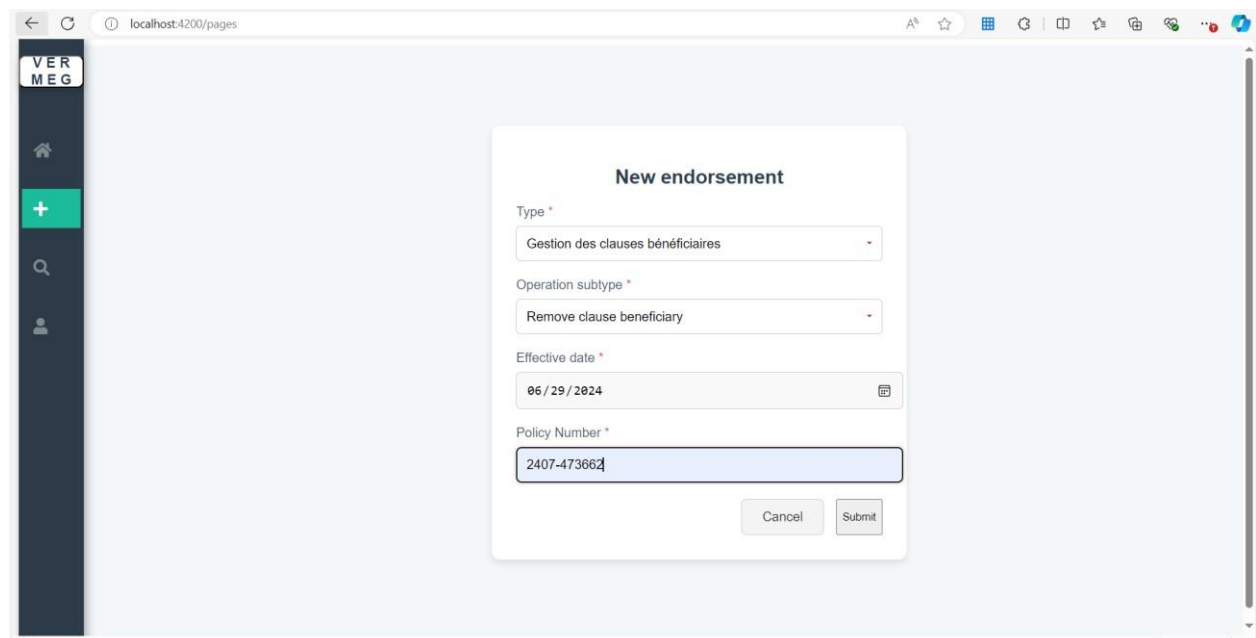


Figure 31. Interface digitale « Encoder un avenant administratif remove clauses »

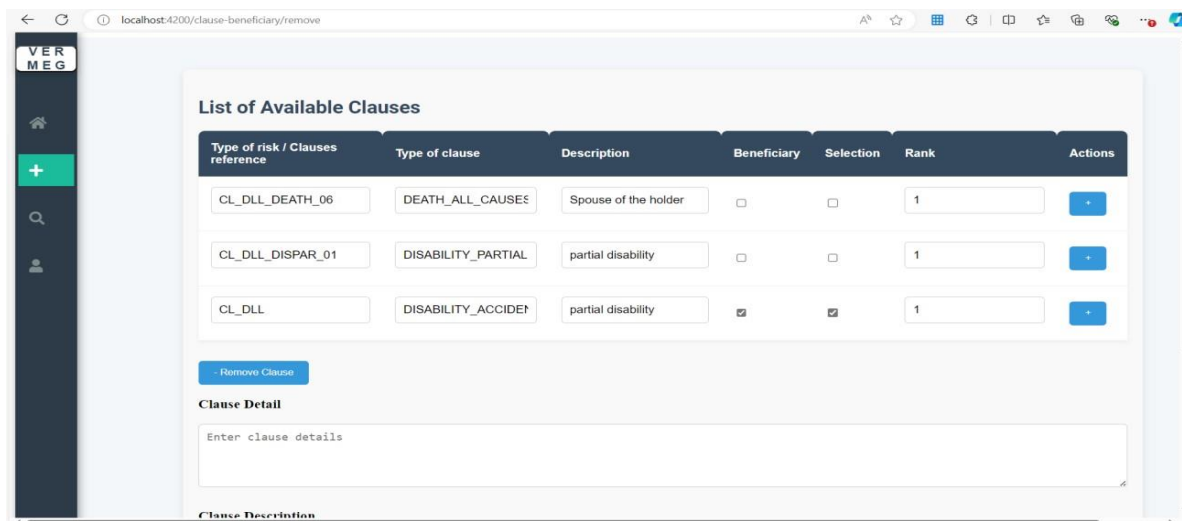


Figure 32. Interface digitale « Encoder un avenant administratif remove clauses »

4.4.1.4 Valider un avenant administratif de type gestion des clauses

a) Valider un avenant administratif de type gestion des clauses pour user_dig via postman

Pour tester l'API "Valider un avenant de gestion de clauses", l'acteur envoie une requête de type "POST" à l'URL suivante: <http://localhost:31080/solife/services/api/endorsements/0002860014/submit>, en insérant le token généré précédemment dans les en-têtes de la requête pour garantir l'authentification. La figure suivante représente le test manuel de la validation de l'avenant via Postman.

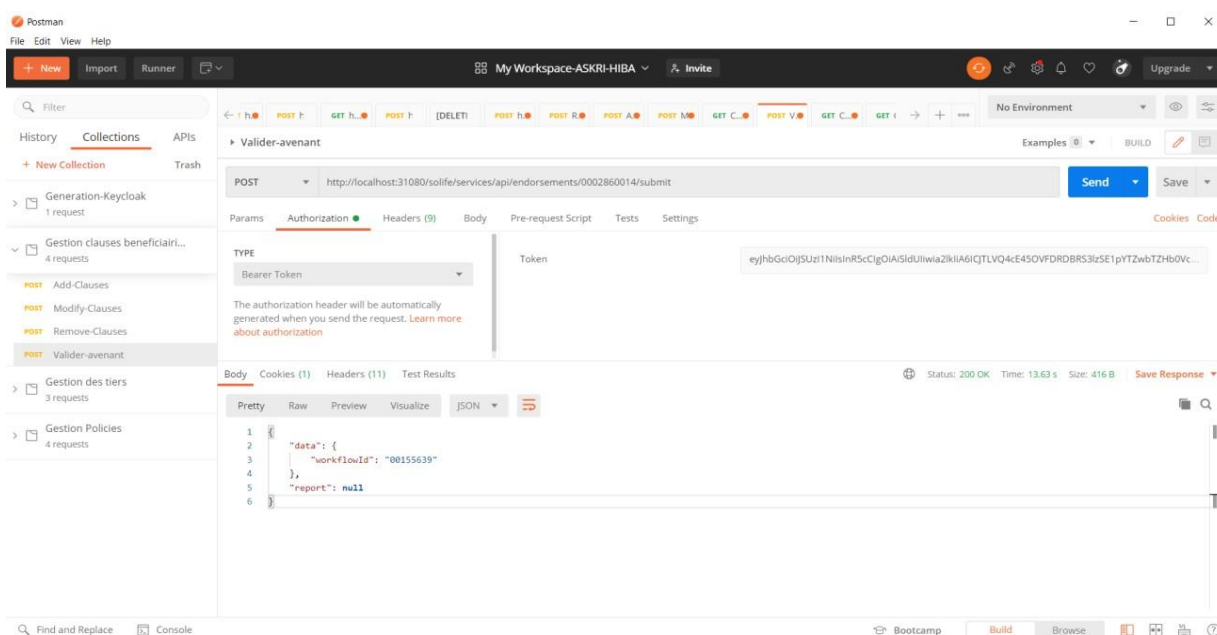


Figure 33. Test API « valider un avenant administratif de gestion des clauses » via postman

b) Valider un avenant administratif de type gestion des clauses pour user_dig via interface digitale

La figure suivante illustre l'interface digitale de validation d'un avenant de gestion de clauses bénéficiaires.

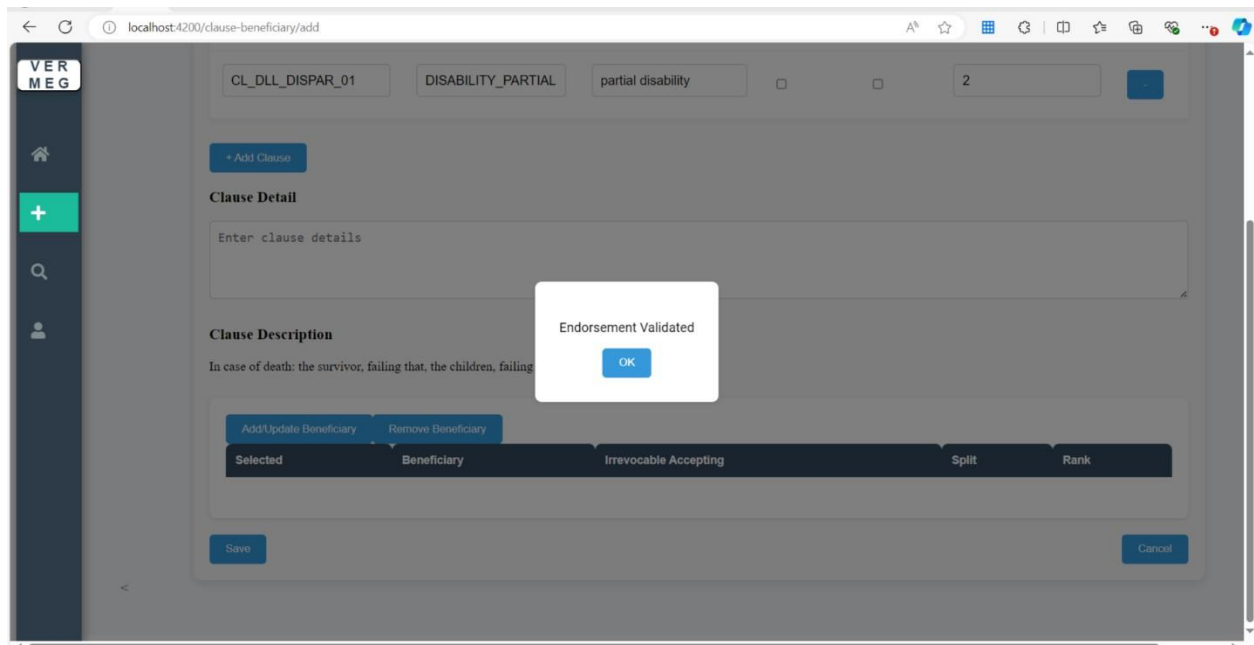


Figure 34. Interface digitale « valider un avenant administratif de gestion des clauses »

Conclusion

Tout au long de ce chapitre, nous avons présenté une vue détaillée des différentes étapes de deuxième sprint en commençant par la gestion du backlog et la phase de conception, jusqu'à l'implémentation des tests manuels et des interfaces digitales.

Chapitre 5 : Sprint 3 - Étude et Réalisation

Introduction

Ce chapitre vise à détailler les diverses étapes de la réalisation et de l'exécution du troisième sprint du projet intitulé 'Gestion des tiers'.

5.1 Définition des tâches

Le tableau suivant rassemble les différentes fonctionnalités du deuxième sprint.

Tableau 13. Le backlog du sprint 3

ID	Tâche	User story	Priorité
1	Ajouter un tier	En tant qu'un testeur QA ou Agent , je peux ajouter un tier dans Solife.	Élevée
2	Mettre à jour à un tier	En tant qu'un testeur QA ou Agent , je peux mettre à jour à un tier.	Élevée
3	Consulter un tier	En tant qu'un testeur QA ou Agent , je peux consulter un tier.	Élevée

5.2 Cas d'utilisation détaillés

5.2.1 Diagramme de cas d'utilisation du sprint 3

Dans le cadre de cette activité, nous détaillerons les cas d'utilisation prioritaires en spécifiant, pour chaque cas, les préconditions, les postconditions, ainsi que le scénario principal. Nous traiterons également des exceptions potentielles associées à chaque cas.

5.2.2 Détails des cas d'utilisation

La figure [35] présente la spécification du cas d'utilisation Gestion des tiers

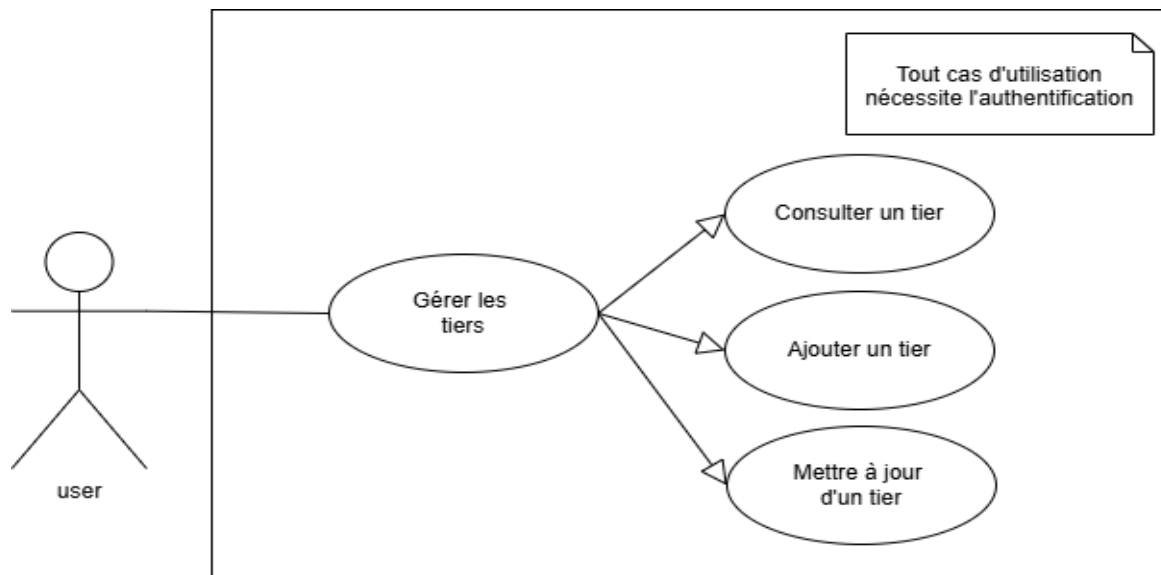


Figure 35. Le Diagramme raffiné du cas d'utilisation Sprint3

5.2.2.1 Ajouter un tier

Ce tableau présente un résumé du scénario du cas d'utilisation "Ajouter un tiers", ainsi que quelques scénarios d'exception. Une fois authentifié, l'utilisateur peut procéder à l'ajout d'un tier.

Tableau 14. Le scénario de cas d'utilisation « Ajouter un tier »

Titre	Ajouter un tier
Acteurs	• Testeur QA • Agent
Résumé	• L'utilisateur saisit les informations relatives au tier
Précondition	• L'utilisateur est authentifié. • Opération de création est sélectionnée.
Postcondition	• Le tier est créé.
Scénario Principal	1- L'utilisateur introduit les données relatives au tier.
	2- L'utilisateur confirme l'opération.
Scénario d'exception	3- Le système contrôle la saisie.
	4- Le système retourne la statut 200 ok : tier ajouté
Scénario d'exception	1- Saisie de body de l'api n'est pas correcte :
	a- Le webservice retourne une réponse avec le statut 400.
	b- Un message d'erreur est généré par le système.

5.2.2.2 Modifier un tier

Ce tableau résume la description du scénario du cas d'utilisation intitulé "Mettre à jour un tier " avec quelques scénarios d'exception. Une fois authentifié, l'acteur peut faire une mise à jour sur un tier.

Tableau 15. Le scénario de cas d'utilisation « Mettre à jour un tier »

Titre	Mettre à jour d'un tier
Acteurs	• Testeur QA • Agent
Résumé	• L'utilisateur saisit les informations relatives au tier
Précondition	• L'utilisateur est authentifié. • Opération de mise à jour est sélectionnée.
Postcondition	• Les données de tier sont mis à jour.
Scénario Principal	1- L'utilisateur saisit les données relatives au tier. 2- L'utilisateur confirme l'opération. 3- Le système contrôle la saisie. 4- Le système affiche la statut 200 ok : tier modifié
Scénario d'exception	1- Saisie de body de l'api n'est pas correcte : - a- Le webservice retourne une réponse avec le statut 400. - b- Un message d'erreur est généré par le système. 2- L'identifiant du tier non disponible dans la base de données Solife - a- Le webservice retourne une réponse avec le statut 400. - b- Un message d'erreur est généré par le système

5.2.2.3 Consulter un tier

Ce tableau résume la description du scénario du cas d'utilisation intitulé "Consulter un tier " avec quelques scénarios d'exception. Une fois authentifié, l'acteur peut consulter un tier.

Tableau 16. Le scénario de cas d'utilisation de « consulter un tier »

Titre	Consulter un tier
Acteurs	•Testeur QA •Agent
Résumé	•L'utilisateur saisit l'identifiant du tier
Précondition	• L'utilisateur est authentifié. •Opération de consultation est sélectionnée.
Postcondition	•Données de tier sont affichées.
Scénario Principal	1- L'utilisateur introduit l'identifiant du tier. 2-l'utilisateur confirme l'opération. 3-Le système contrôle la saisie. 4-Le système renvoie la statut 200 ok : informations tier affichés.
Scénario d'exception	1-L'identifiant du tier non trouvé dans la base de données Solife : - a- Le webservice retourne une réponse avec le statut 400. - b- Un message d'erreur est généré par le système

5.3 Conception technique

5.3.3 Diagramme de séquences spécifiques

Le diagramme de séquences permet de décrire les scénarios de chaque cas d'utilisation en mettant l'accent sur la chronologie des opérations en interaction avec les objets. Dans ce chapitre, nous allons présenter quelques diagrammes de séquences.

5.3.3.1 Diagramme de séquences de cas utilisation « Ajouter un tier »

La figure [36] illustre le diagramme de séquences du Cas d'utilisation «Ajouter un tier »

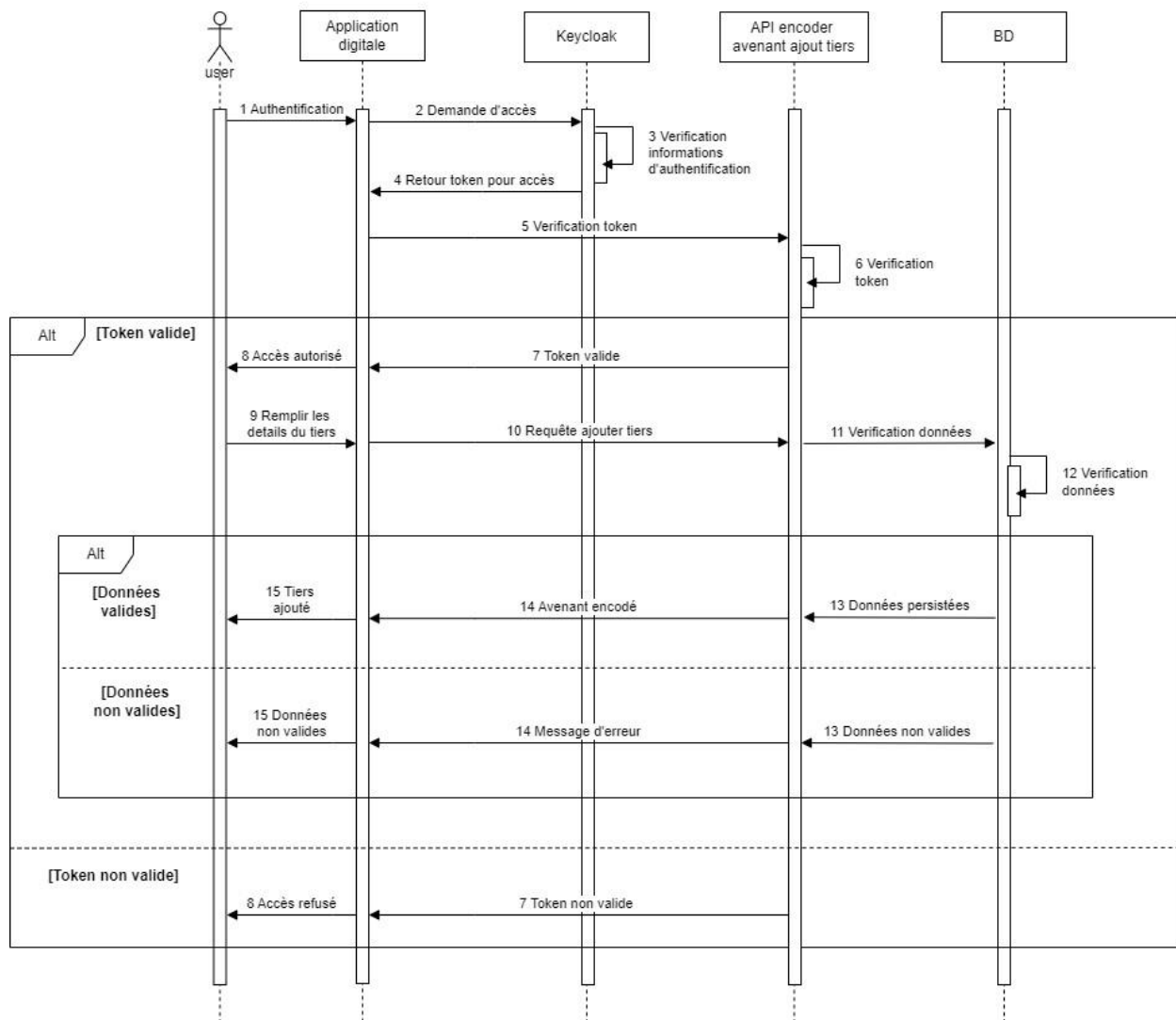


Figure 36. Le diagramme de séquence de cas d'utilisation « Ajouter un tier »

5.4 Réalisation et tests

Une fois que la conception de notre **sprint3** a été élaborée, nous entamons cette section en abordant le dernier aspect de ce sprint. L'objectif est de présenter la phase de **réalisation**, qui constitue la concrétisation ultime du résultat obtenu au cours de ce sprint.

5.4.1 Tests manuels de sprint 3

5.4.1.1 Ajouter un tier

a) Ajouter un tier pour user_dig via postman

Pour tester l'API "Ajouter un tier", l'acteur lance une requête de type "POST" avec le lien " <http://localhost:31080/solife/services/api/thirdparties>" et le body de l'api en introduisant le token précédemment généré.

La figure suivante montre le test manuel d'ajout d'un tier via Postman.

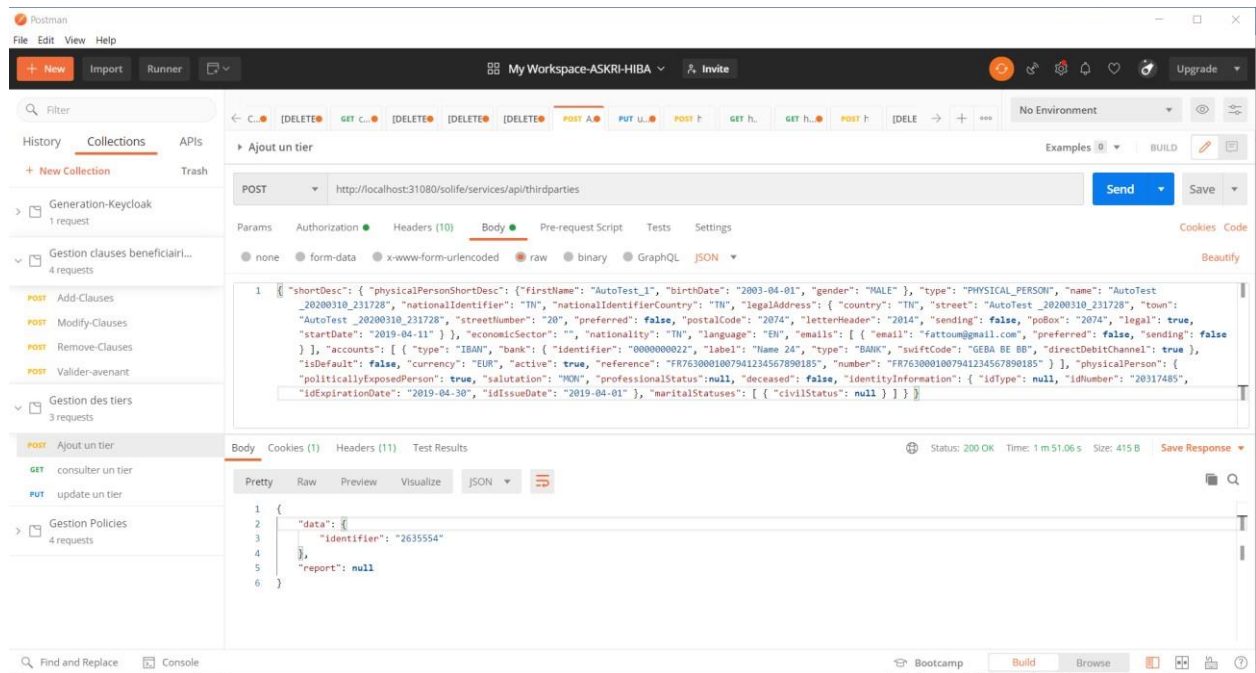


Figure 37. Test Api Ajouter un tier avec user_dig via Postman

b) Ajouter un tier pour user_dig via interface digitale

La figure suivante montre l'interface digitale de l'ajout d'un tier

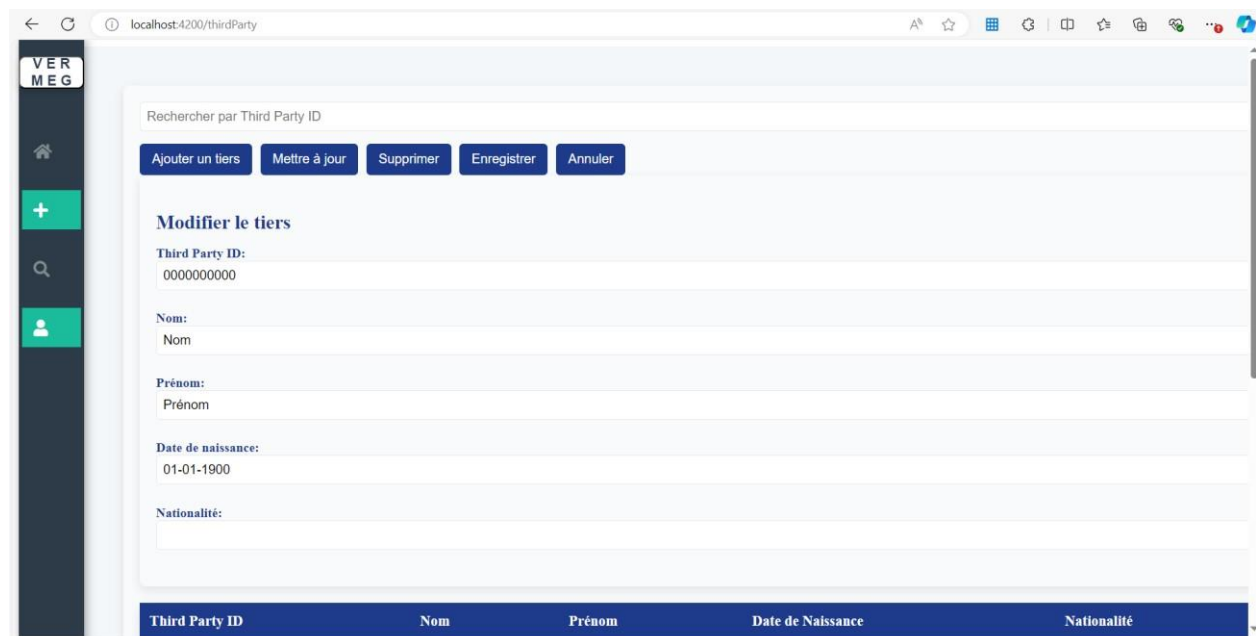


Figure 38. Interface digitale d'ajout un tier

5.4.2.1 Mettre à jour un tier

a) Mettre à jour un tier pour user_dig via postman

Pour tester l'api "Mise à jour d'un tier", l'acteur lance une requête de type "PUT" avec le lien "http://localhost:31080/solife/services/api/thirdparties" et le body de l'api en insérant le token précédemment généré.

La figure suivante montre le test manuel de la mise à jour d'un tier via Postman.

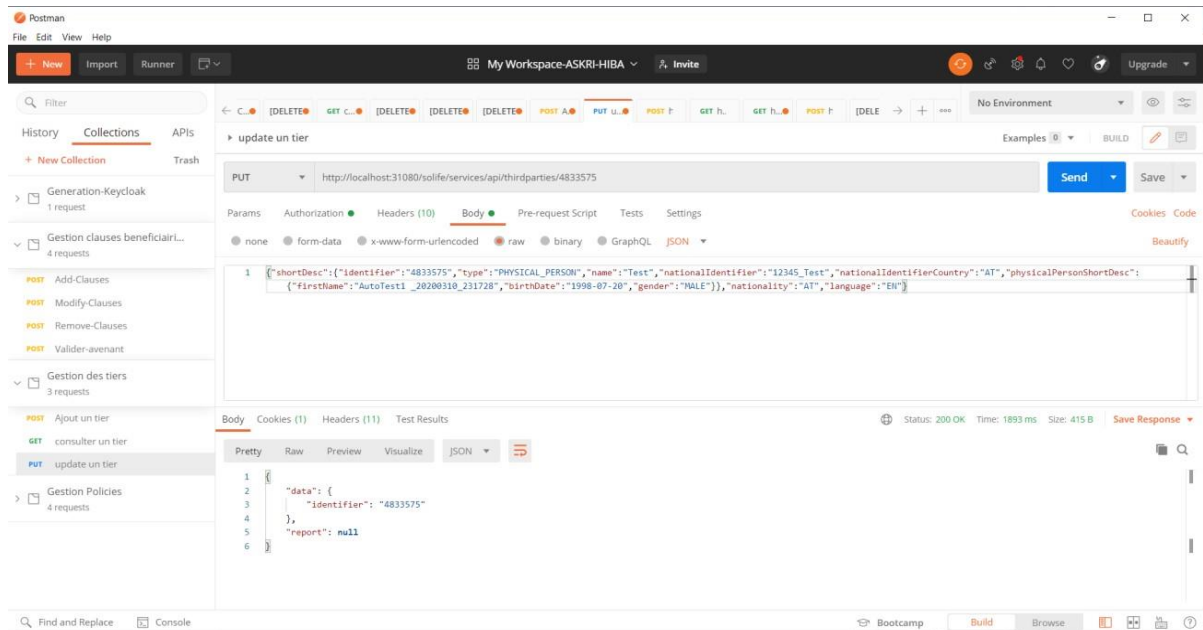


Figure 39. Test Api Mettre à jour un tier avec user_dig via Postman

b) Mettre à jour un tier pour user_dig via interface digitale

La figure [40] montre l'interface digitale de la mise à jour d'un tier

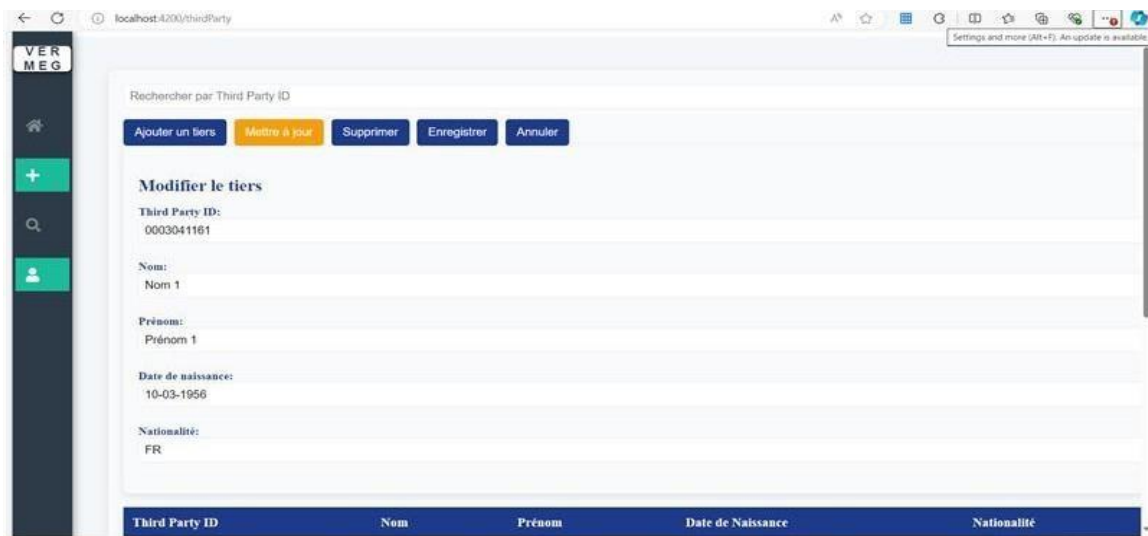


Figure 40. Interface digitale Mettre à jour un tier

5.4.3.1 Consulter un tier

a) Consulter un tier pour user_dig via postman

Pour tester l'api "Consulter un tier", l'acteur lance une requête de type "GET" avec le lien "http://localhost:31080/solife/services/api/thirdparties" et en insérant le token précédemment généré.

La figure suivante représente le test manuel de la consultation d'un tier via Postman.

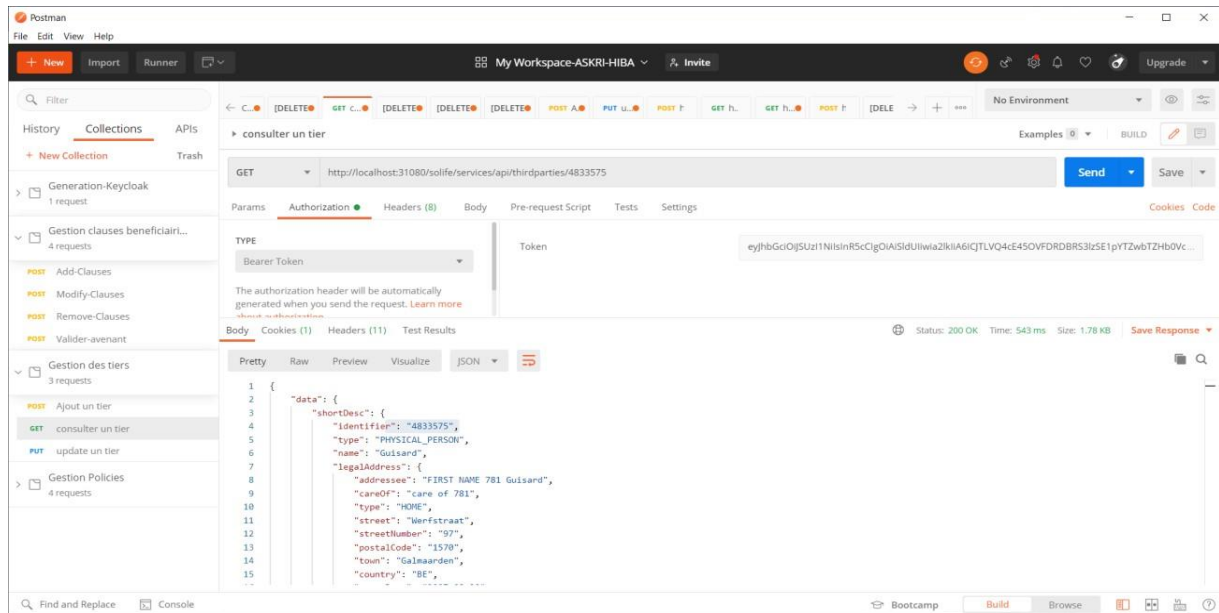


Figure 41. Test Api Consulter un tier avec user_dig via Postman

b) Consulter un tier pour user_dig via interface digitale

La figure [42] montre l'interface digitale de la consultation d'un tier

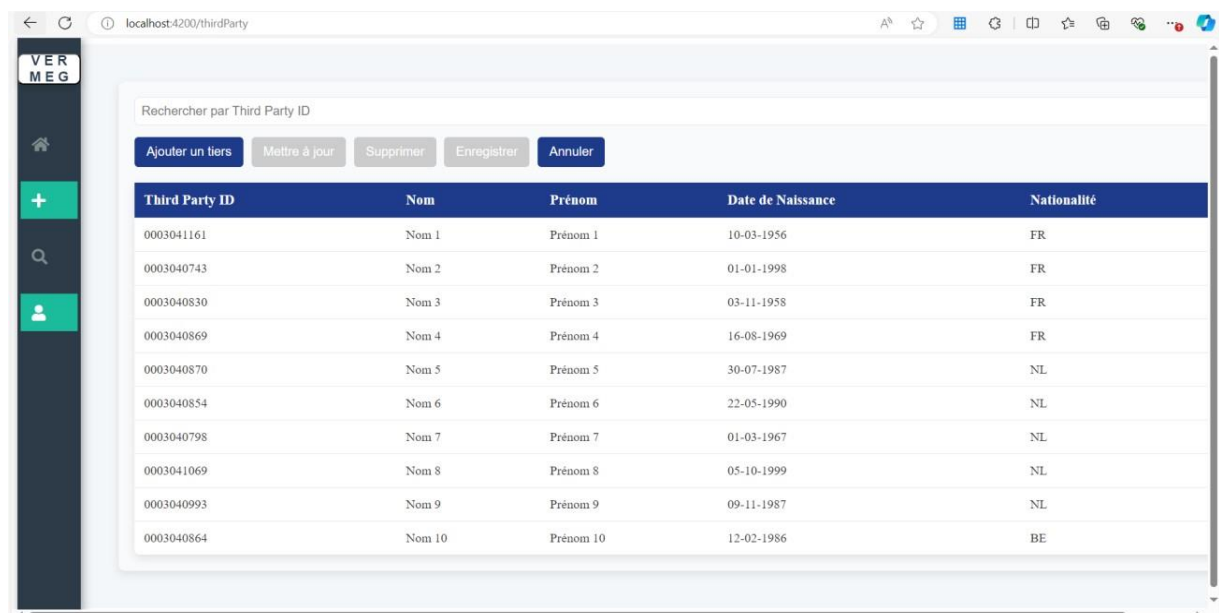


Figure 42. Interface digitale consulter un tier

Conclusion

Tout au long de ce chapitre, nous avons présenté une vue détaillée des différentes étapes de troisième sprint en commençant par la gestion du backlog et la phase de conception, jusqu'à l'implémentation des tests manuels et des interfaces digitales.

Chapitre 6 : Sprint 4 - Automatisation et déploiement

Introduction

Ce chapitre consiste à détailler les différentes étapes de la réalisation et de l'exécution du dernier sprint du projet intitulé “ **Automatisation des tests et intégration continue**”.

6.1 Définition des tâches

Le tableau suivant rassemble les différentes fonctionnalités du quatrième sprint.

Tableau 17. Lebacklog du sprint 4

ID	Tâche	User story	Priorité
1	Automatisation des tests et intégration continue	En tant qu'un testeur QA, je peux consulter les tests automatiques de gestion des polices sur Allure. En tant qu'un testeur QA, je peux consulter les tests automatiques de gestion des clauses bénéficiaires sur Allure. En tant qu'un testeur QA, je peux consulter les tests automatiques de gestion des tiers sur Allure. En tant qu'un testeur QA, je peux consulter les pipelines sur jenkins de la génération de release.	Élevée

6.2 Détails des cas d'utilisations

Au cours de cette activité nous allons détailler les cas d'utilisation les plus prioritaires en décrivant, dans chaque cas, les préconditions et les post conditions ainsi que le scénario principal, et éventuellement les exceptions liées à ce cas.

6.2.1 Spécification de cas d'utilisation « Automatisation des tests et intégration continue »

La figure suivante montre la spécification du cas d'utilisation « Automatisation des tests et intégration continue ».

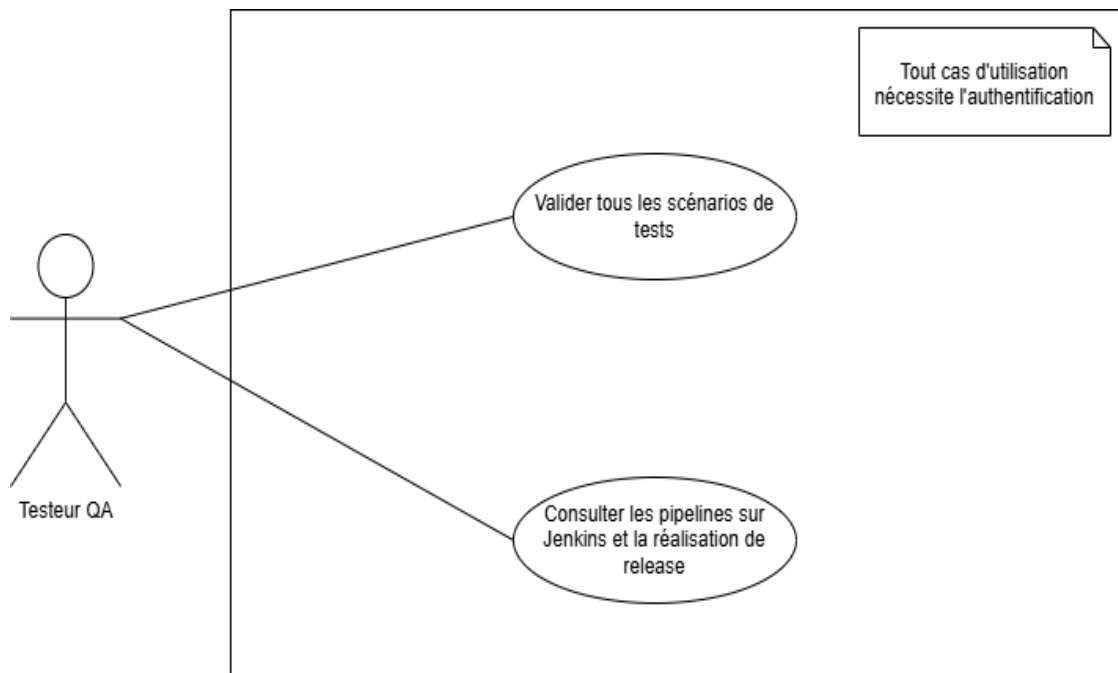


Figure 43.Le diagramme raffiné de cas d'utilisation Automatisation des tests et intégration continue

6.2.1.1 Valider tous les scenarios de tests

Ce tableau résume la description du scénario du cas d'utilisation intitulé "Valider tous les scenarios de test " avec quelques scénarios d'exception. Une fois authentifié, l'acteur testeur QA peut commencer à tester tous les scenarios possibles.

Tableau 18. Le scénario de cas d'utilisation valider les scénarios de tests

Titre	Valider les scenarios de test
Acteurs	•Testeur QA
Résumé	•L'acteur teste chaque api selon son propre plan de test
Précondition	•Acteur est authentifié. •Opération sélectionnée.
Postcondition	•Test réalisé
Scénario Principal	1-L'acteur teste le scenario. 2-l'acteur confirme l'opération. 3-Le système contrôle la saisie. 4-Le système renvoie la statut 200 ok.
Scénario d'exception	1-En cas d'erreur : a-La réponse statut 400 webservice est retournée. b- Le système renvoie un message d'erreur en spécifiant le type d'erreur.

6.2.1.2 Consulter les pipelines Jenkins et la réalisation de la release

Ce tableau résume la description du scénario du cas d'utilisation intitulé " Consulter les pipelines Jenkins et la réalisation de la release " avec quelques scénarios d'exception.

Une fois authentifié sur Jenkins, l'acteur testeur QA peut commencer à suivre les pipelines et la réalisation de la release.

Tableau 19. Le scénario de cas d'utilisation consulter les pipelines et la génération de release

Titre	Consulter les pipelines Jenkins et la réalisation de la release
Acteurs	•Testeur QA
Résumé	•L'acteur visualise les pipelines et la génération de release
Précondition	•Acteur est authentifié sur Jenkins.
Postcondition	•Release est générée
Scénario Principal	1-L'acteur visualise le process d'avancement des pipelines sans souci. 2-L'acteur suit la génération de release
Scénario d'exception	1-En cas d'erreur : a-La pipeline faille b-La release non générée.

6.3 Réalisation des tests automatisés

6.3.1 Tests automatisés sprint 1

Une fois la conception du sprint 4 achevée, cette section se concentre sur le dernier volet de ce sprint. L'objectif principal est de détailler la phase d'exécution des tests automatisés.

Afin d'automatiser les tests relatifs à la gestion des polices dans le sprint 1, l'outil ReadyAPI a été utilisé. Le processus débute par la création d'une requête de type "GET", qui comprend différents cas de test. La configuration de cette requête nécessite de définir l'Endpoint suivant : "<http://localhost:31080/solife/services/api>", ainsi que la ressource "/policies/policy_number". Le numéro de la police est ensuite ajouté en tant que paramètre. Une fois cette configuration effectuée, la requête est lancée pour valider les résultats obtenus.

La figure ci-dessous illustre le test effectué pour la consultation d'une police via ReadyAPI. *

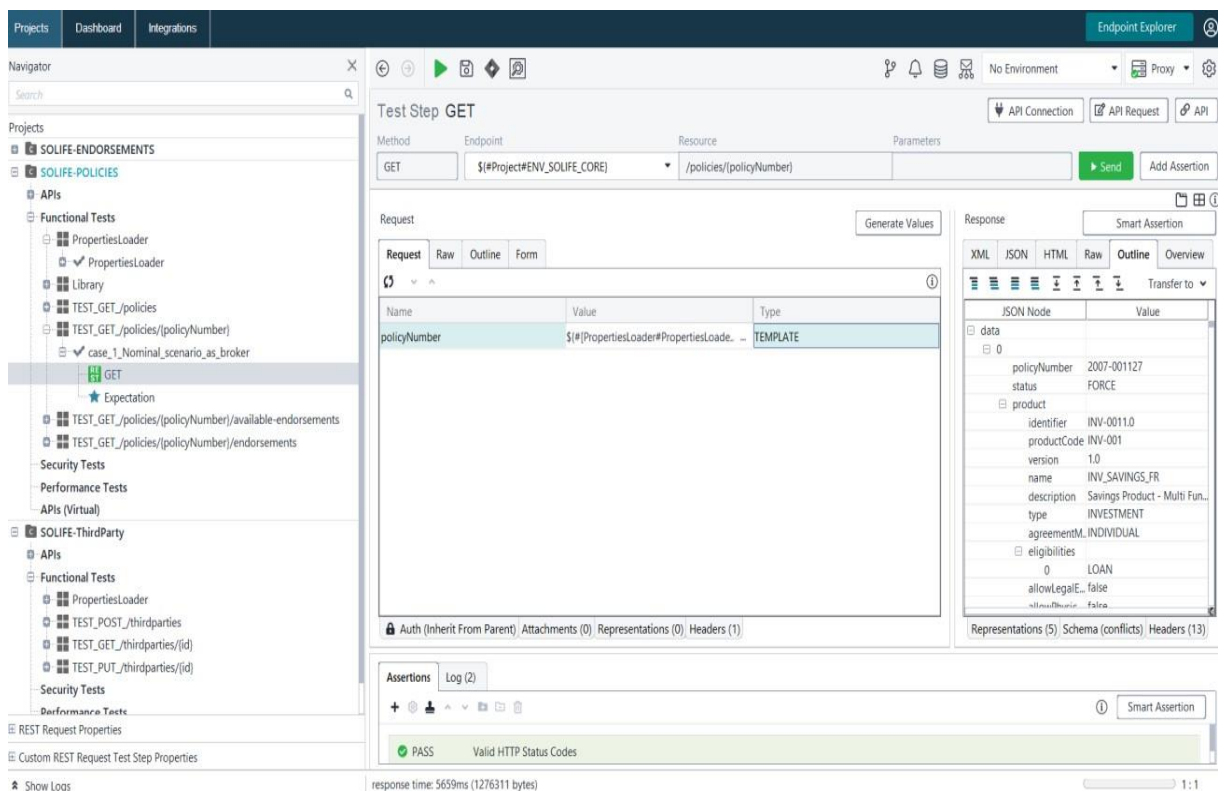


Figure 44. Test automatisé API consulter une police via ReadyApi

Afin d'automatiser les tests de consultation de l'historique des avenants d'une police, nous avons utilisé l'outil ReadyApi.

Le processus débute par la création d'une requête de type "GET", comprenant les différents cas de test associés. Cette requête est configurée avec l'Endpoint suivant : "<http://localhost:31080/solife/services/api>", ainsi que la ressource "/policies/policy_number/endorsements". Le numéro de la police est alors défini comme paramètre, permettant ainsi de consulter l'historique des avenants liés. Une fois la configuration terminée, la requête est exécutée pour valider les résultats obtenus.

La figure ci-dessous montre le test effectué pour consulter l'historique des avenants d'une police à l'aide de ReadyAPI.

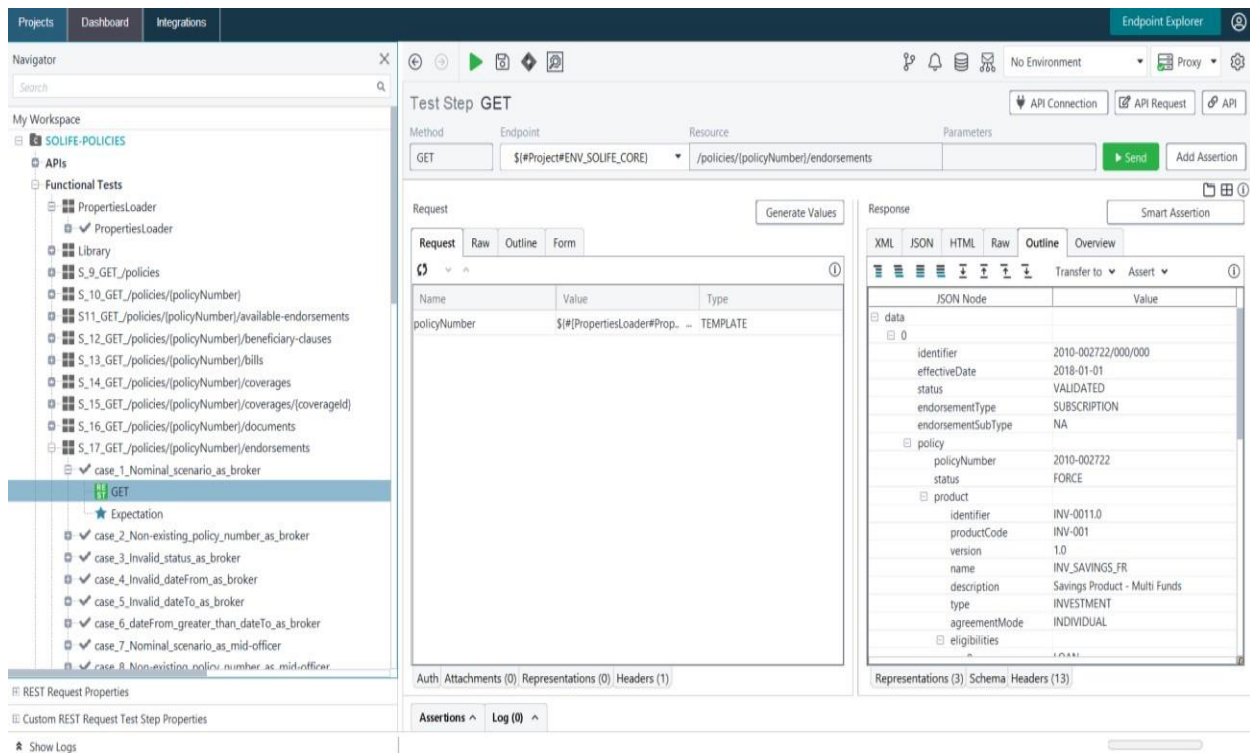


Figure 45. Test automatisé API consulter historique avenant une police via ReadyApi

Afin d'automatiser les tests de consultation de types d'avenants de modification éligibles sur une police, nous avons utilisé l'outil ReadyApi.

La première étape consiste à créer une requête de type "GET", accompagnée de ses différents cas de test. Cette requête est ensuite configurée avec l'Endpoint "<http://localhost:31080/solife/services/api>" et la ressource `"/policies/available-modificationtypes/ identi_policy_number "`. Le numéro de police est ajouté en tant que paramètre pour permettre la consultation. Enfin, la requête est exécutée afin de vérifier le résultat obtenu.

La figure ci-dessous montre le test réalisé pour consulter les types d'avenants de modification éligibles sur une police en utilisant ReadyAPI.

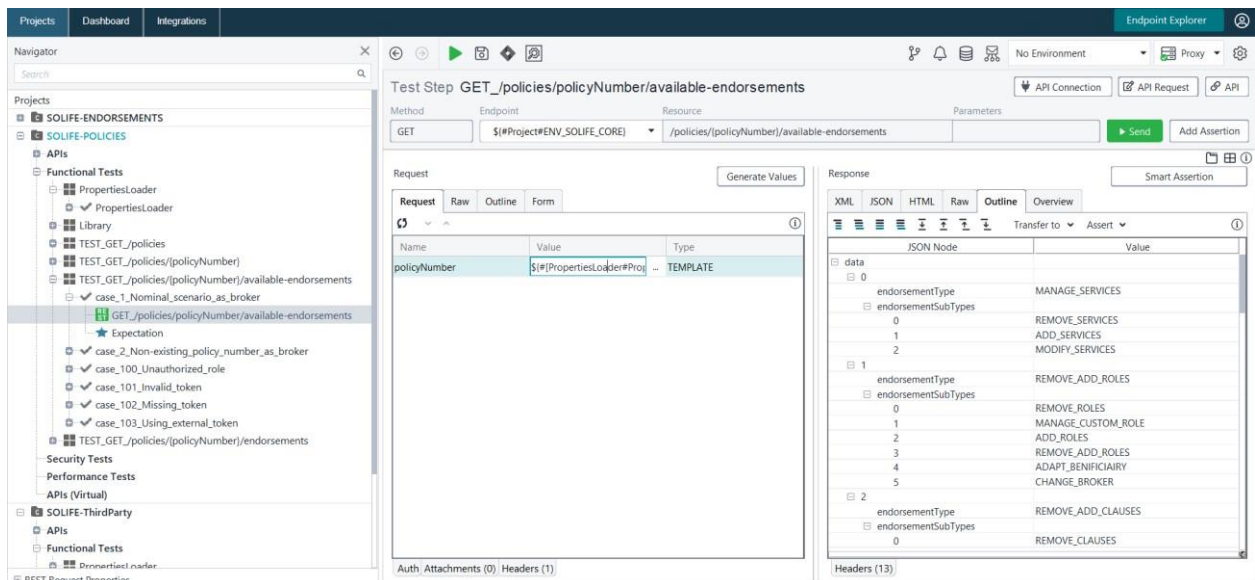


Figure 46. Test automatisé API consulter des avenants de modification éligibles sur une police via ReadyApi

6.3.2 Génération des rapports des tests automatisés du sprint 1

Après l'exécution des tests automatisés déclenchés sur ReadyAPI, les rapports détaillés sont générés et visualisables sur Allure, comme illustré dans la figure suivante.

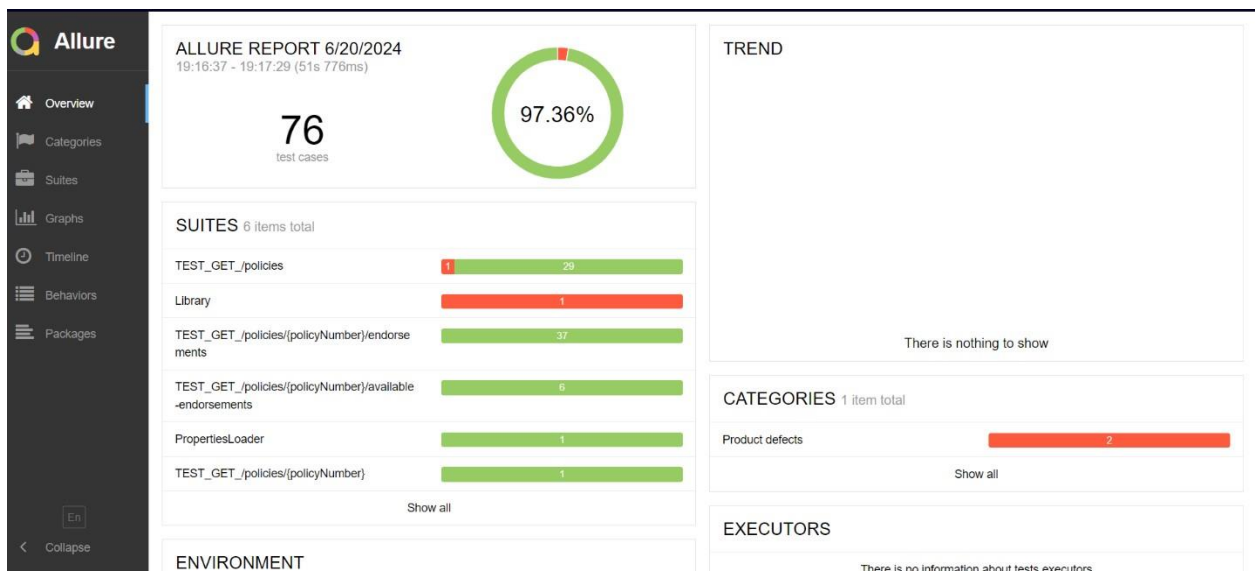


Figure 47. Génération rapport consulter une police

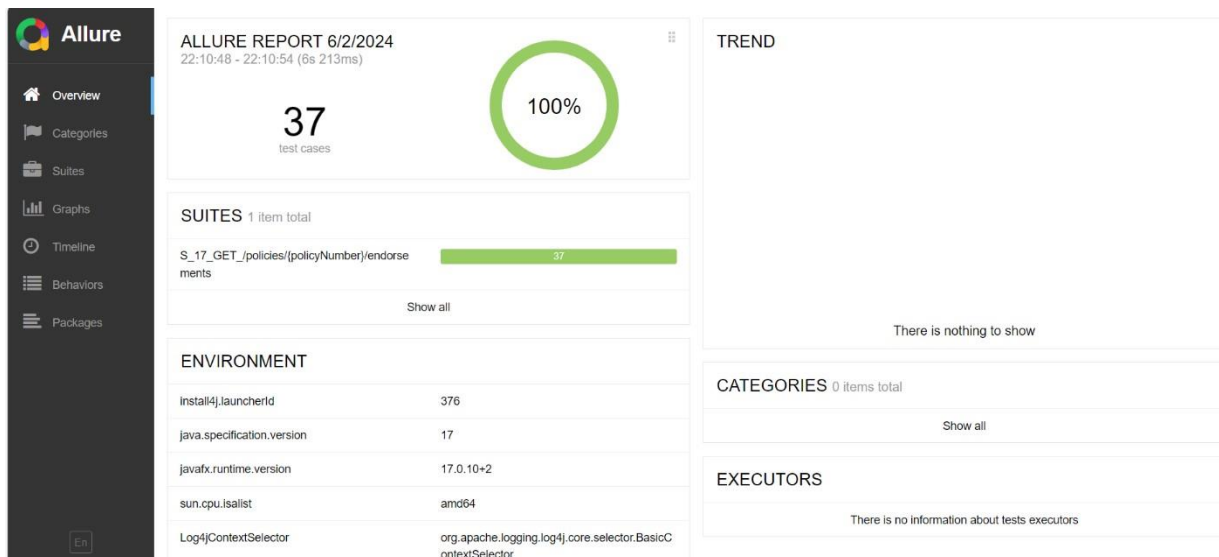


Figure 48. Génération rapport consulter historique des avenants d'une police

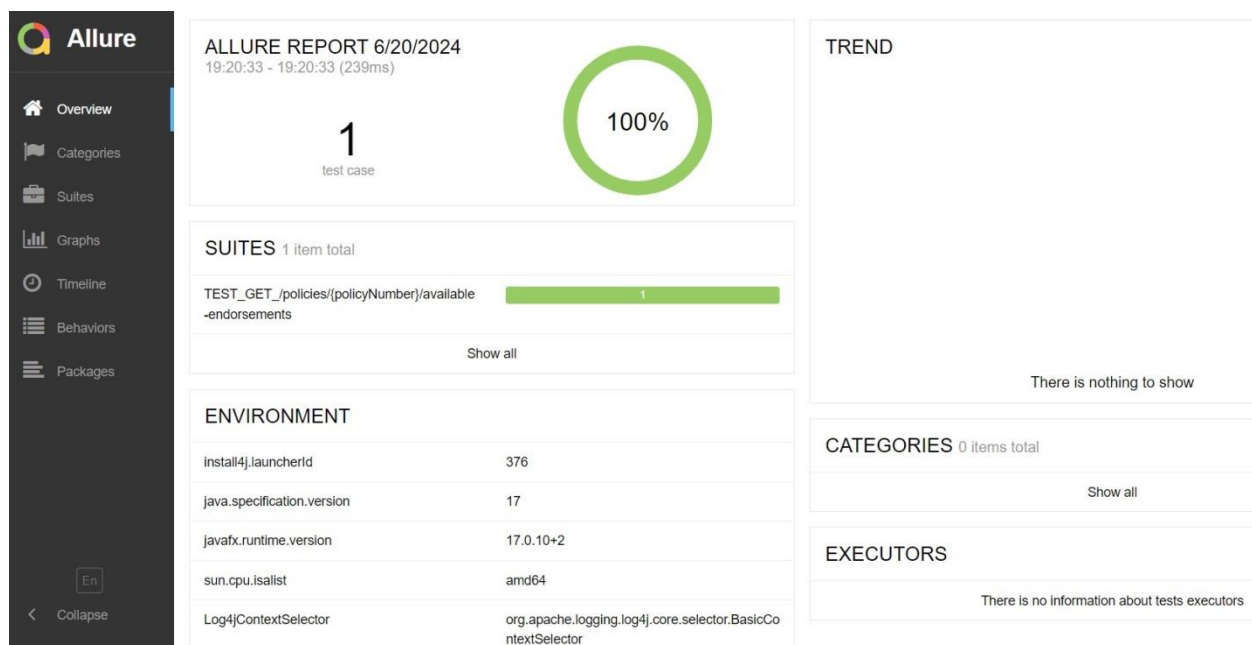


Figure 49. Génération rapport consulter des avenants de modification éligibles sur une police

6.3.3 Tests automatisés Sprint 2

Afin d'automatiser les tests de gestion des clauses bénéficiaires, nous avons utilisé l'outil ReadyApi.

Pour créer un avenant add clauses : Nous débutons par la création d'une requête de type "POST", accompagnée de ses différents cas de test. Ensuite, la configuration est réalisée en

spécifiant l'Endpoint "<http://localhost:31080/solife/services/api>" ainsi que la ressource "/endorsement/add-clauses".

Ensuite, les détails de l'avenant à créer sont spécifiés avec précision. Enfin, la requête est exécutée et le résultat est vérifié pour s'assurer de sa conformité.

La figure ci-dessous représente le test de la création d'avenant d'ajout des clauses bénéficiaires via ReadyApi

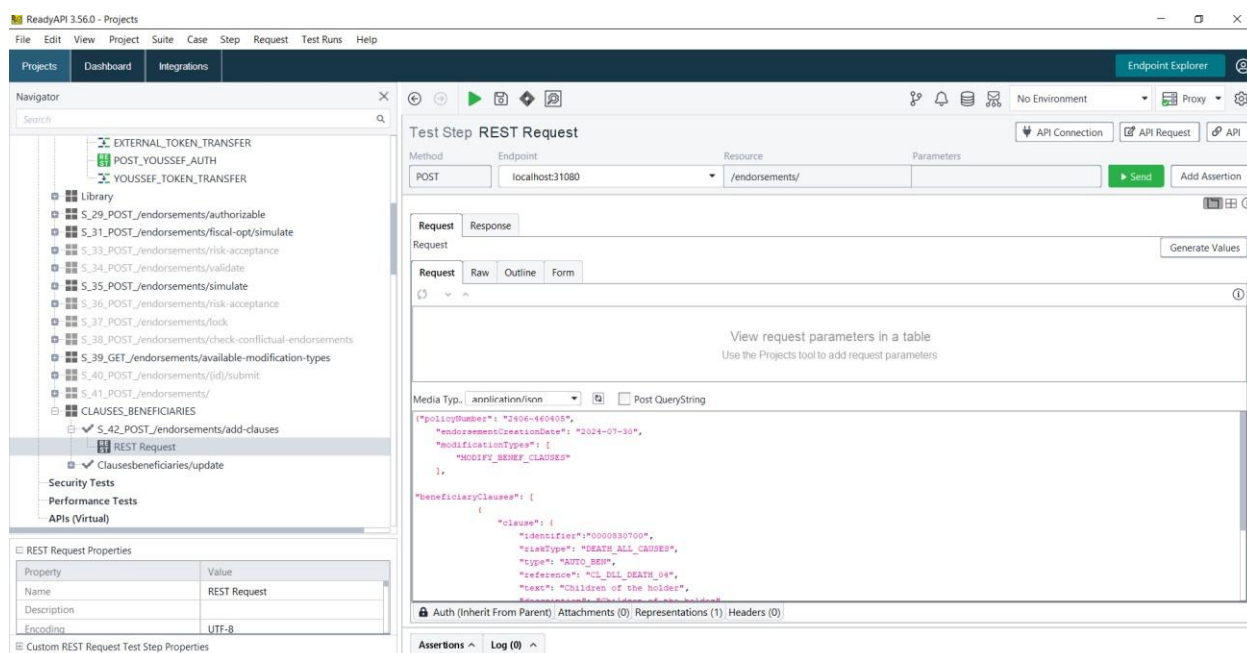


Figure 50. Test automatisé API add clauses via ReadyApi

Pour créer un avenant Modify clauses : Nous débutons par la création d'une requête de type "POST", accompagnée de ses différents cas de test. Ensuite, nous configurons la requête en définissant l'Endpoint "<http://localhost:31080/solife/services/api>" et la ressource "/endorsement/modify-clauses".

Les détails de l'avenant à créer sont ensuite précisés. Enfin, la requête est exécutée et les résultats obtenus sont vérifiés.

La figure ci-dessous illustre le test de création d'un avenant pour modifier les clauses bénéficiaires en utilisant ReadyAPI.

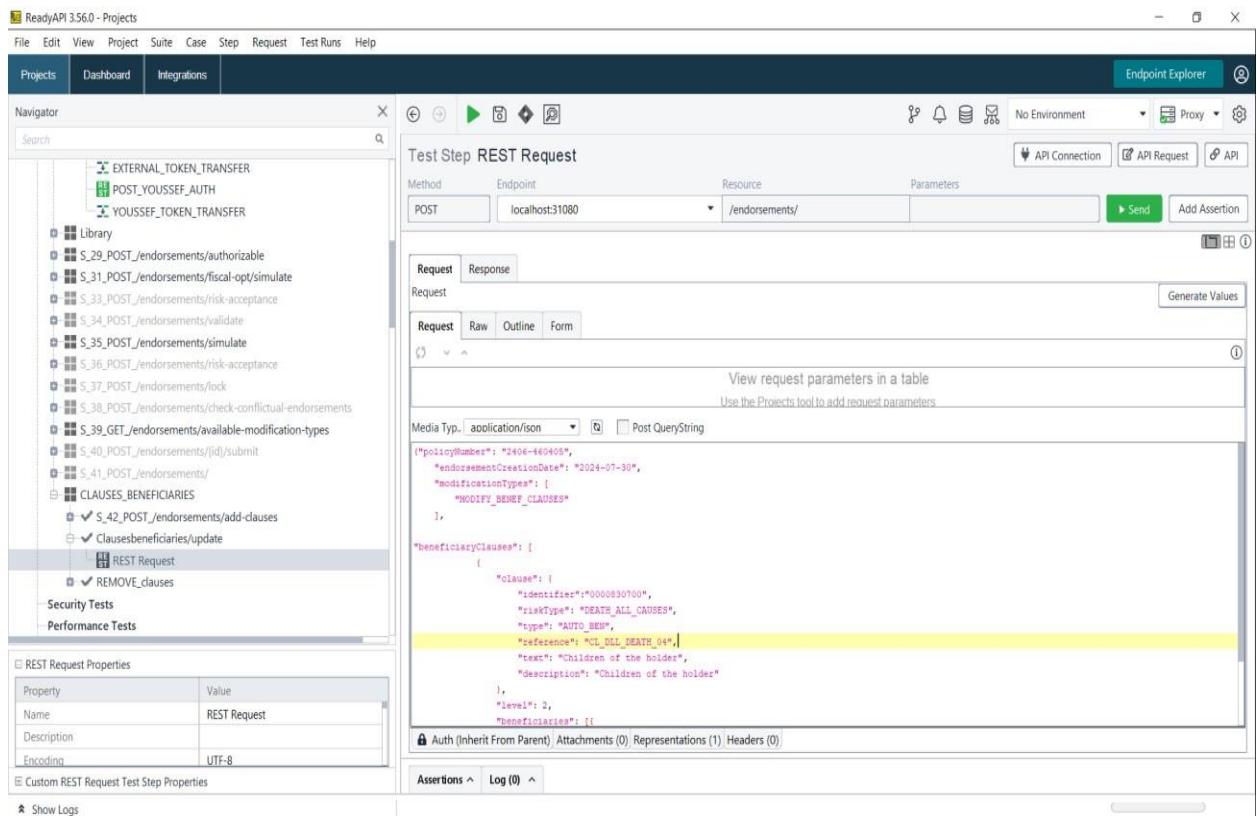


Figure 51. Test automatisé API Modify clauses via ReadyAPI

Pour créer un avenant remove clauses : Nous débutons par la création d'une requête de type "POST", accompagnée de ses différents cas de test. Ensuite, nous configurons la requête en définissant l'Endpoint "http://localhost:31080/solife/services/api" et la ressource "/endorsement/ remove-clauses".

Les détails de l'avenant à créer sont ensuite précisés. Enfin, la requête est exécutée et les résultats obtenus sont vérifiés.

La figure ci-dessous illustre le test de création d'un avenant pour supprimer les clauses bénéficiaires en utilisant ReadyAPI.

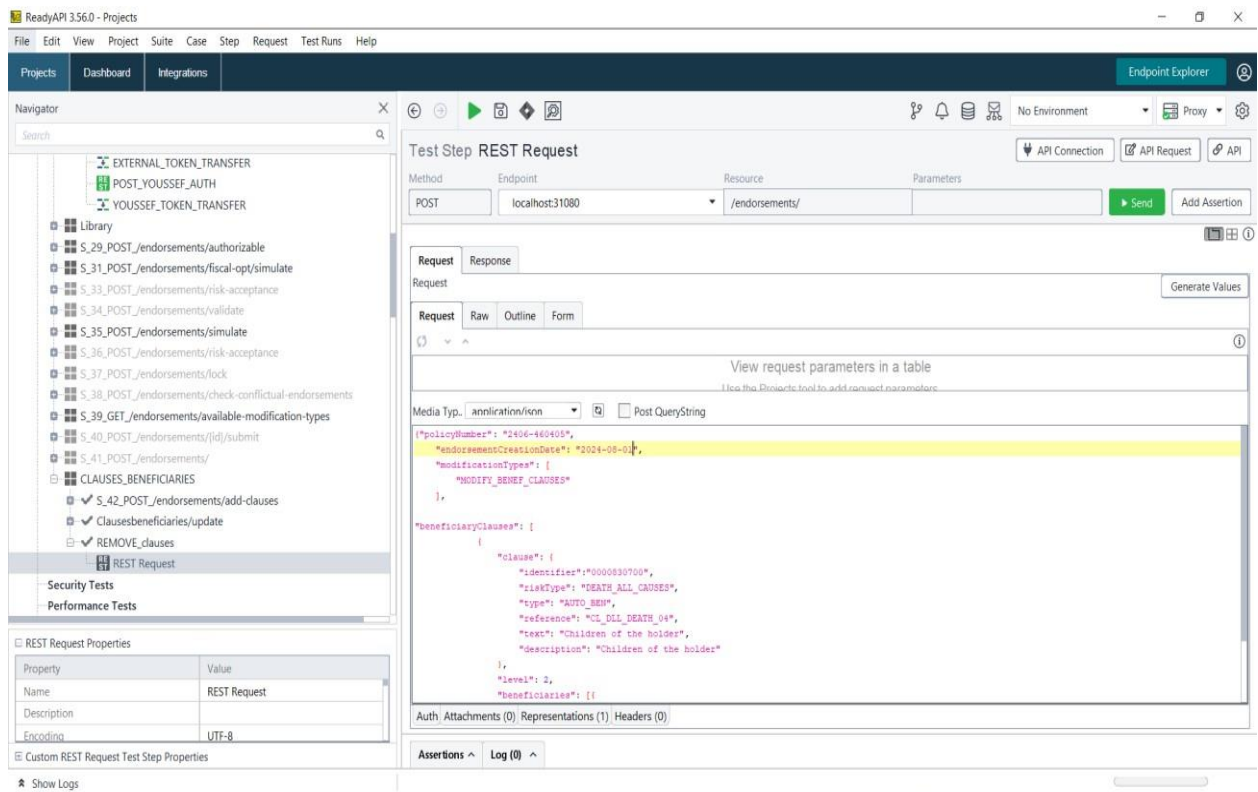


Figure 52. Test automatisé API remove clauses via ReadyApi

6.3.4 Génération des rapports des tests automatiques sprint 2

Après l'exécution des tests automatisés déclenchés sur ReadyAPI, les rapports détaillés sont générés et visualisables sur Allure, comme illustré dans la figure suivante.

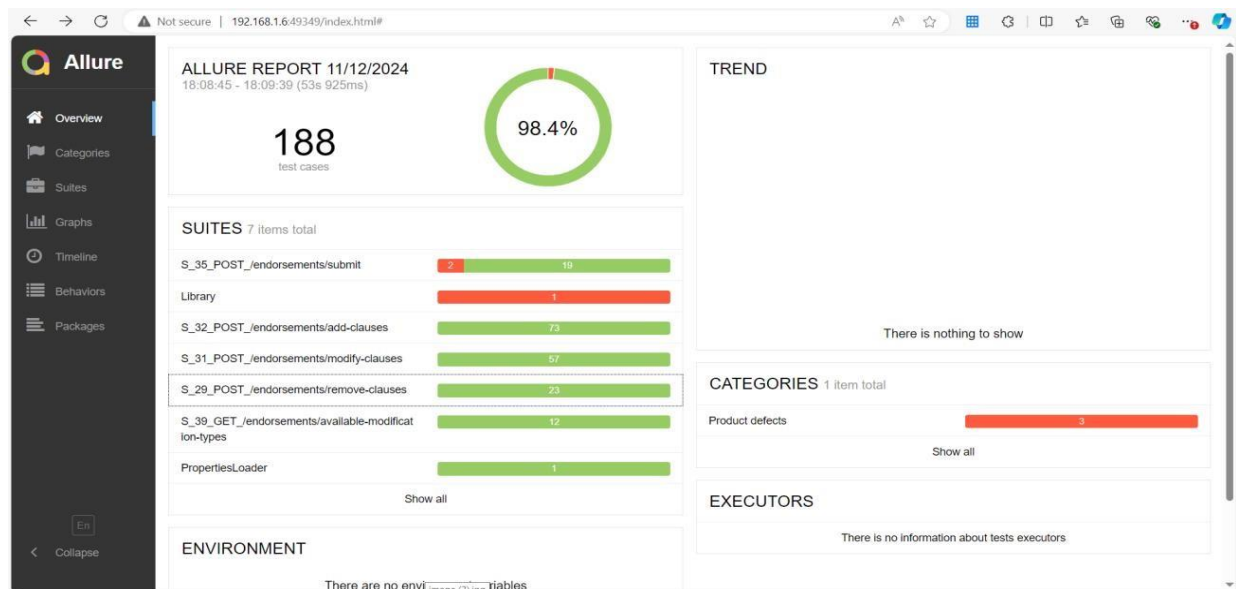


Figure 53. Génération rapport des apis de gestion clauses bénéficiaires sur une police

6.3.5 Tests automatisés Sprint 3

Afin d'automatiser les tests de gestion des tiers, nous avons utilisé l'outil ReadyApi.

Consulter un tier : Nous débutons par la création d'une requête de type "GET", incluant ses différents cas de test. Ensuite, la configuration est effectuée en renseignant l'Endpoint "<http://localhost:31080/solife/services/api>" et la ressource "/thirdparties/id". L'identifiant du tiers est ajouté en tant que paramètre pour la consultation. Enfin, la requête est exécutée et les résultats obtenus sont analysés.

La figure ci-dessous montre le test réalisé pour consulter les tiers à l'aide de ReadyAPI.

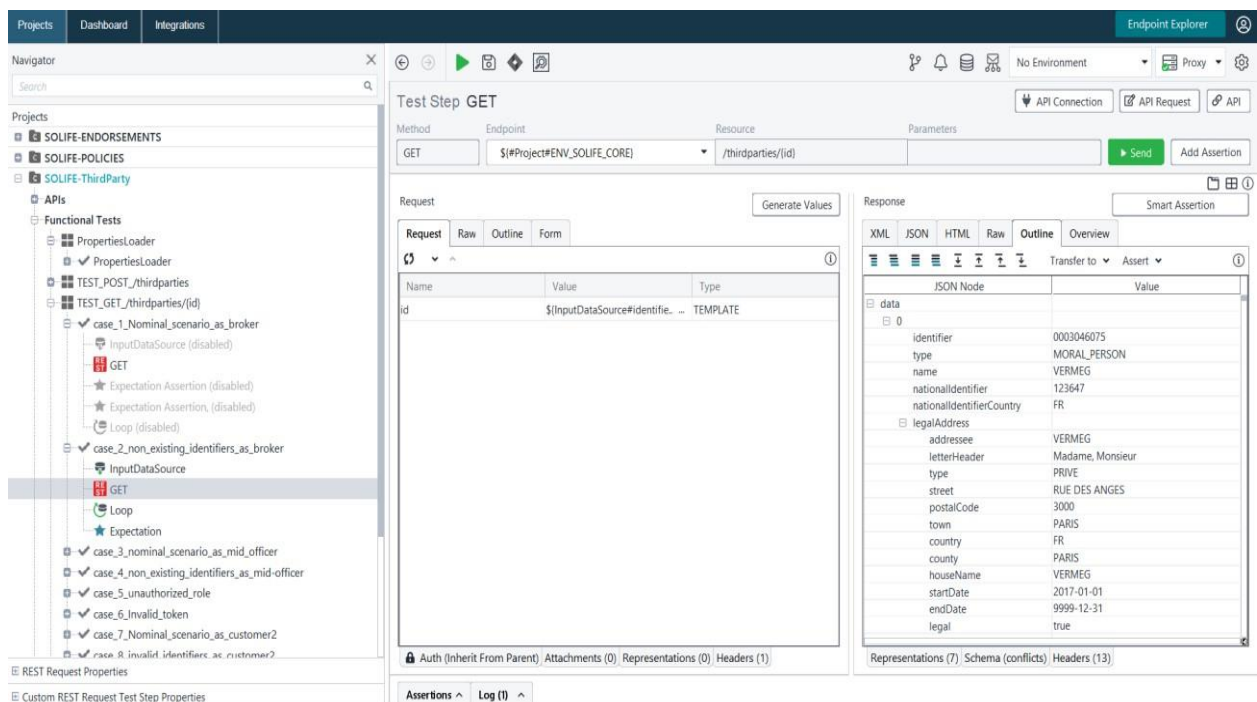


Figure 54. Test automatisé API consulter un tier via ReadyApi

Ajouter un tier : Nous débutons par la création d'une requête de type "POST", incluant ses différents cas de test. Ensuite, nous configurons l'Endpoint avec l'adresse "<http://localhost:31080/solife/services/api>" et la ressource "/thirdparties". Les détails et les coordonnées du nouveau tiers à ajouter sont spécifiés comme paramètres. Enfin, la requête est exécutée et les résultats obtenus sont analysés.

La figure ci-dessous montre le test réalisé pour l'ajout d'un tiers à l'aide de ReadyAPI.

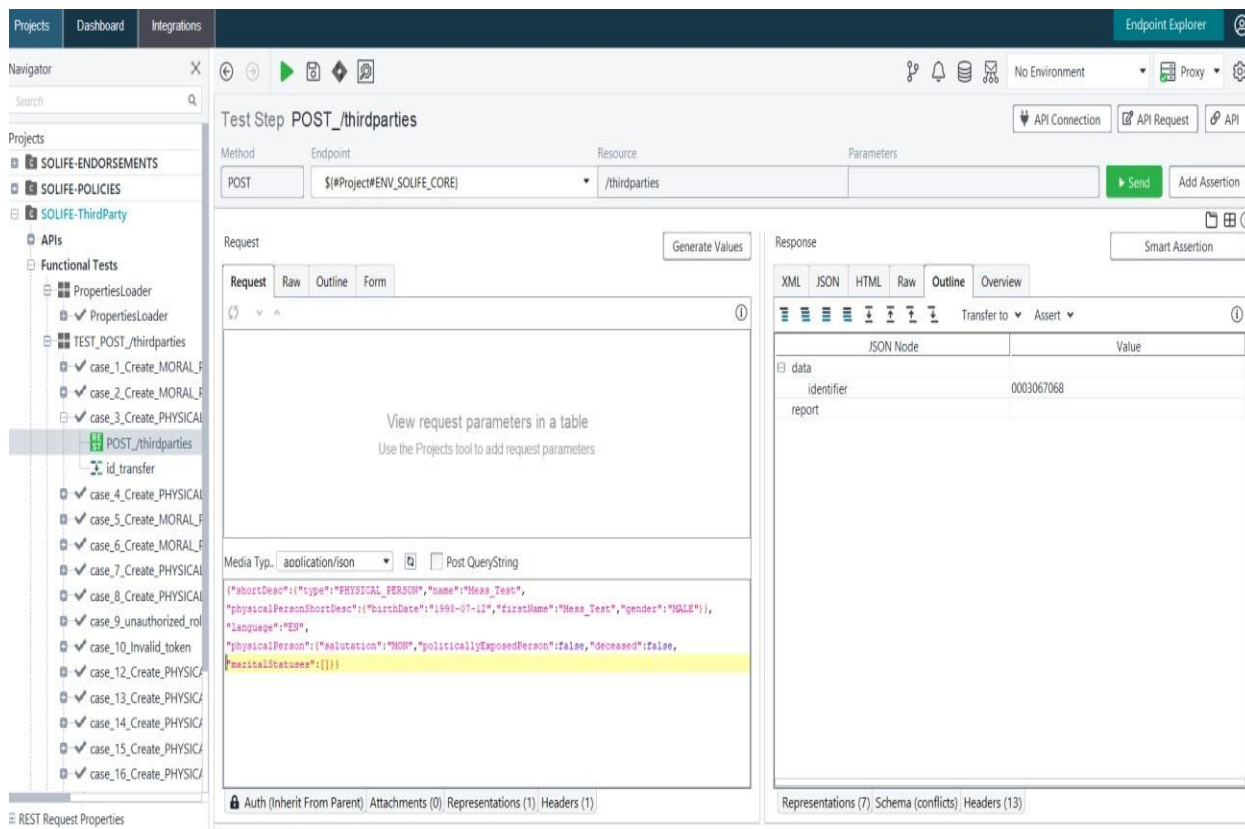


Figure 55. Test automatisé API ajout un tiers via ReadyApi

Mettre à jour un tiers : Nous débutons par la création d'une requête de type "PUT", accompagnée de ses différents cas de test. Ensuite, la configuration est réalisée en définissant l'Endpoint "<http://localhost:31080/solife/services/api>" et la ressource `"/thirdparties/id"`.

L'identifiant du tiers à modifier ainsi que les nouvelles coordonnées à mettre à jour sont spécifiés comme paramètres. Enfin, la requête est exécutée et les résultats obtenus sont vérifiés.

La figure ci-dessous illustre le test réalisé pour la mise à jour d'un tiers à l'aide de ReadyAPI.

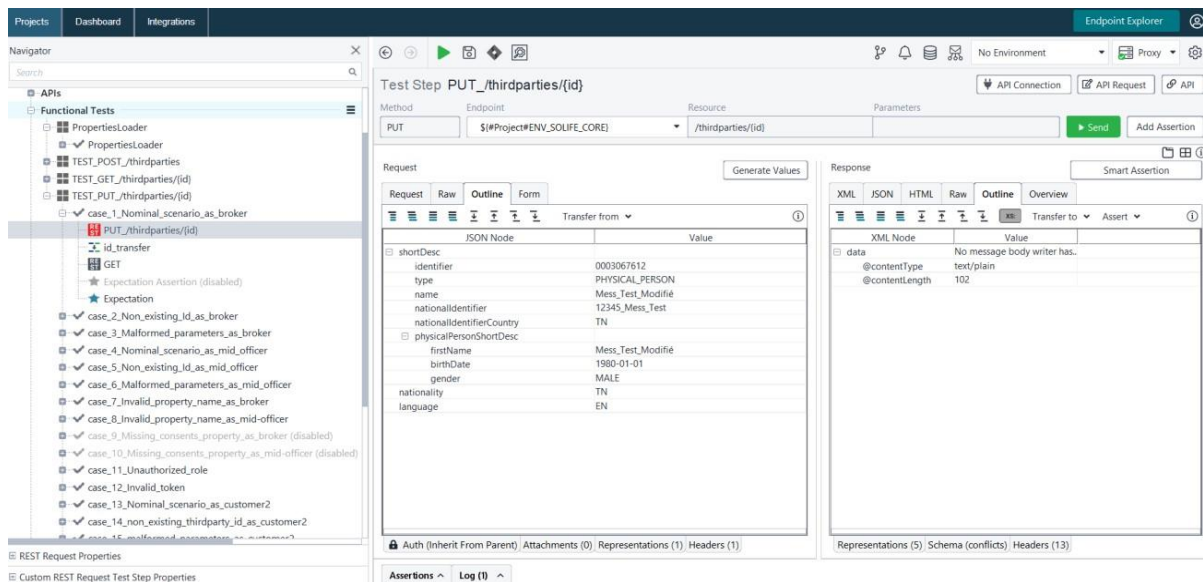


Figure 56. Test automatisé API mettre à jour un tier via ReadyApi

6.3.6 Génération des rapports des tests automatiques sprint 3

Après l'exécution des tests automatisés déclenchés sur ReadyAPI, les rapports détaillés sont générés et visualisables sur Allure, comme illustré dans la figure suivante.

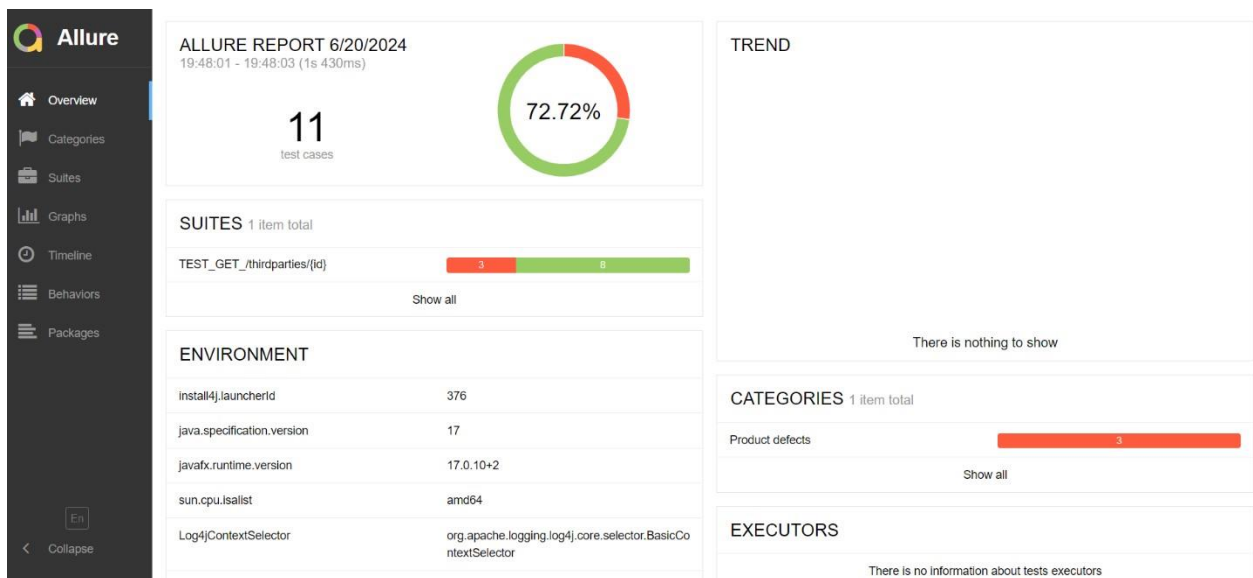


Figure 57. Génération rapport de test consulter un tier

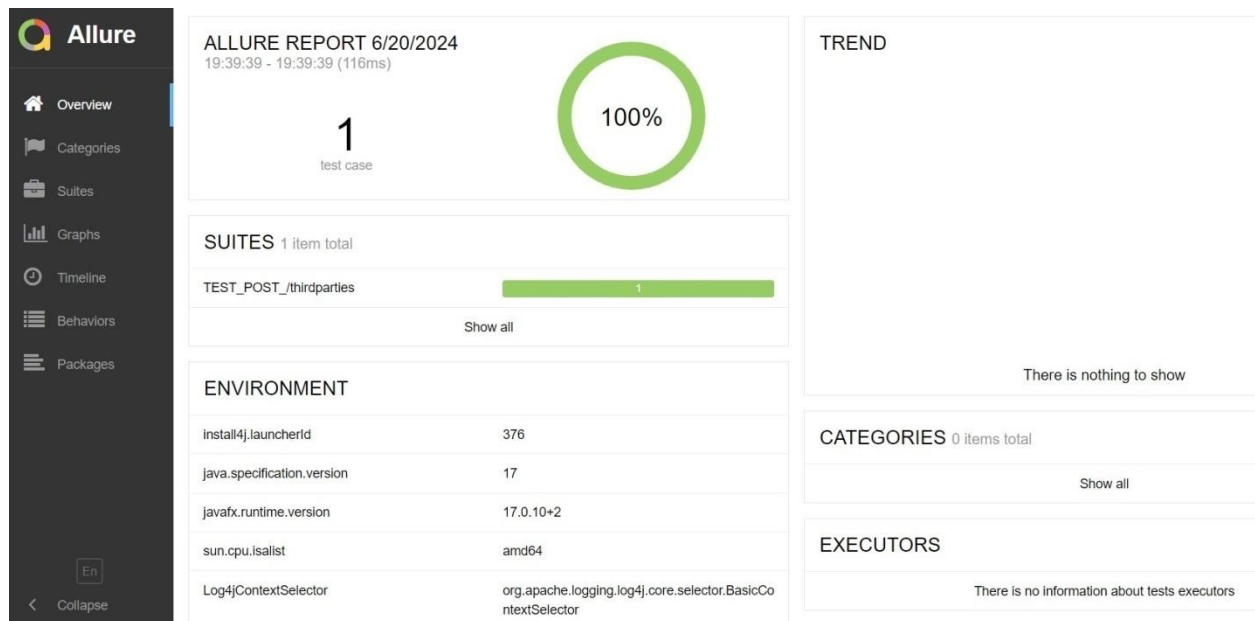


Figure 58. Génération rapport de test ajout un tier

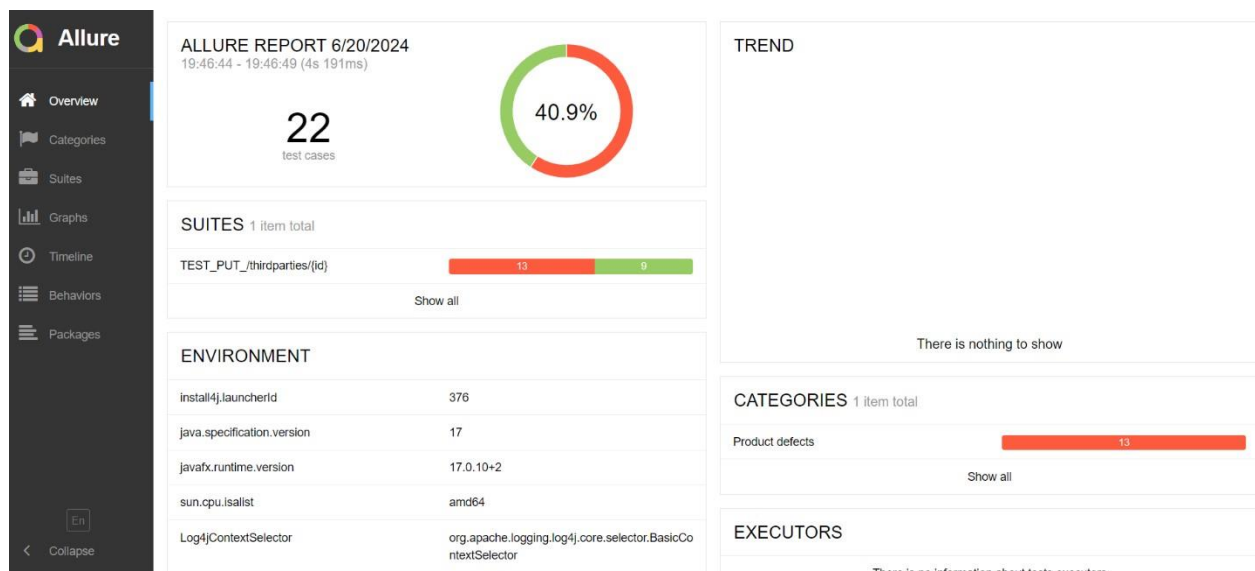


Figure 59. Génération rapport de test mettre à jour d'un tier

6.4 Intégration Continue CI et déploiement CD

6.4.1 Pipelines Jenkins et configuration CI/CD

L'intégration continue est une méthode où les développeurs intègrent fréquemment leurs nouvelles portions de code au projet principal. Ce code est ensuite stocké dans un dépôt central partagé et soumis à des tests et vérifications automatiques constants. Ce processus aide l'équipe à repérer rapidement les erreurs potentielles après chaque modification, en recevant des notifications immédiates en cas de problème. Cela leur permet d'apporter les corrections nécessaires pour garantir la stabilité du logiciel. L'intégration continue automatise la construction, l'empaquetage et les tests du code.

6.4.2 Livraison continue et déploiement continu CD

La livraison et le déploiement continu représentent la seconde étape du pipeline CI/CD et viennent compléter le processus d'intégration continue. Une fois la construction du code terminée, les changements sont automatiquement mis à disposition dans un environnement de test ou, parfois, directement en production.

Livraison Continue : Cette méthode vise à rendre le logiciel disponible pour un déploiement rapide et fréquent, en générant des versions prêtes à être lancées à tout moment. Cela signifie que chaque changement dans le code peut être mis en production, bien qu'une action manuelle soit parfois nécessaire pour le faire.

Déploiement Continu: Cette approche va encore plus loin en automatisant complètement le passage en production. Chaque modification validée par les tests automatiques est directement publiée pour les utilisateurs finaux, sans intervention humaine. Cela assure une disponibilité instantanée des mises à jour validées.

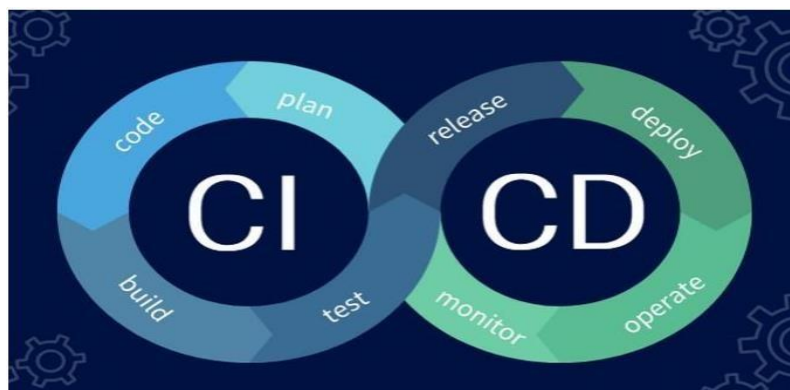


Figure 60. Processus d'intégration continue et déploiement

➤ Outils d'intégration continue et de livraison continue : Jenkins

Jenkins est une plateforme open source essentielle dans les pipelines CI/CD, largement adoptée pour son extensibilité et sa flexibilité. Grâce à son vaste écosystème de plugins, il permet de personnaliser et d'automatiser les processus avec une grande efficacité. Ce serveur d'automatisation se distingue par sa capacité à intégrer différents outils, ce qui en fait un choix privilégié pour de nombreux projets. Après analyse de critères tels que l'accessibilité, la variété des extensions et la simplicité d'utilisation, Jenkins a été sélectionné pour notre projet en raison de sa capacité à répondre parfaitement à nos besoins d'intégration et de déploiement.

➤ Outil de test de qualité de code : SonarQube

SonarQube est un outil puissant d'analyse de code utilisé pour détecter et corriger les problèmes de qualité logicielle. Sa flexibilité, ses fonctionnalités avancées et ses rapports détaillés le placent parmi les leaders dans ce domaine. De plus, SonarQube bénéficie d'une communauté active et d'un support solide, qui renforcent son efficacité dans l'amélioration continue du code. Après avoir étudié ses capacités, nous l'avons choisi comme solution principale pour l'analyse de code dans le cadre de notre projet, en raison de sa couverture complète et de son intégration fluide dans les pipelines CI/CD.

➤ Outil de gestion de référentiel : Nexus

Sonatype Nexus est un gestionnaire de référentiel fiable, idéal pour organiser, stocker et distribuer les artefacts nécessaires au développement logiciel. Il prend en charge un large éventail de technologies et s'intègre aisément avec les outils de build modernes. En considérant ses nombreuses fonctionnalités et sa capacité à évoluer avec les technologies actuelles, Nexus s'est imposé comme un choix stratégique pour notre projet, permettant une gestion efficace des dépendances et des livrables.

➤ Récapitulation des choix des outils

Pour résumer nos choix d'outils et de technologies pour ce projet, nous avons répertorié les outils sélectionnés pour chaque fonctionnalité dans le tableau suivant :

Fonctionnalité	Outil
Outil de gestion des versions	GitLab
Serveur d'automatisation CI/CD	Jenkins
Outil de revue de code	SonarQube
Outil de gestion de dépôt	SonarType Nexus

6.4.3 Réalisation de CI/CD

6.4.3.1 Création et Exécution d'un Pipeline Jenkins

La création d'un pipeline dans Jenkins consiste à définir et automatiser les étapes de construction, de test, et de déploiement d'un projet logiciel. Voici les étapes générales de création d'un pipeline, suivies de l'exécution détaillée dans le cas de notre projet.

1. Création du Pipeline:

- **Définition du Jenkinsfile** : Le pipeline est d'abord défini dans un fichier appelé **Jenkinsfile**, qui décrit les différentes étapes à suivre dans le processus. Ce fichier peut être écrit en langage de script Groovy et contient des instructions pour Jenkins sur les tâches à accomplir à chaque étape.
- **Configuration des outils** : Des outils nécessaires au pipeline (par exemple, JDK, Maven, Gradle) sont spécifiés dans le Jenkinsfile ou dans la configuration de Jenkins. Ces outils permettent de compiler et de déployer le projet.
- **Définition des Étapes (Stages)** : Le pipeline est structuré en plusieurs étapes logiques, appelées **stages**, qui regroupent les actions à réaliser. Par exemple, une étape peut être dédiée à la récupération du code source, une autre à l'exécution des tests, et ainsi de suite.

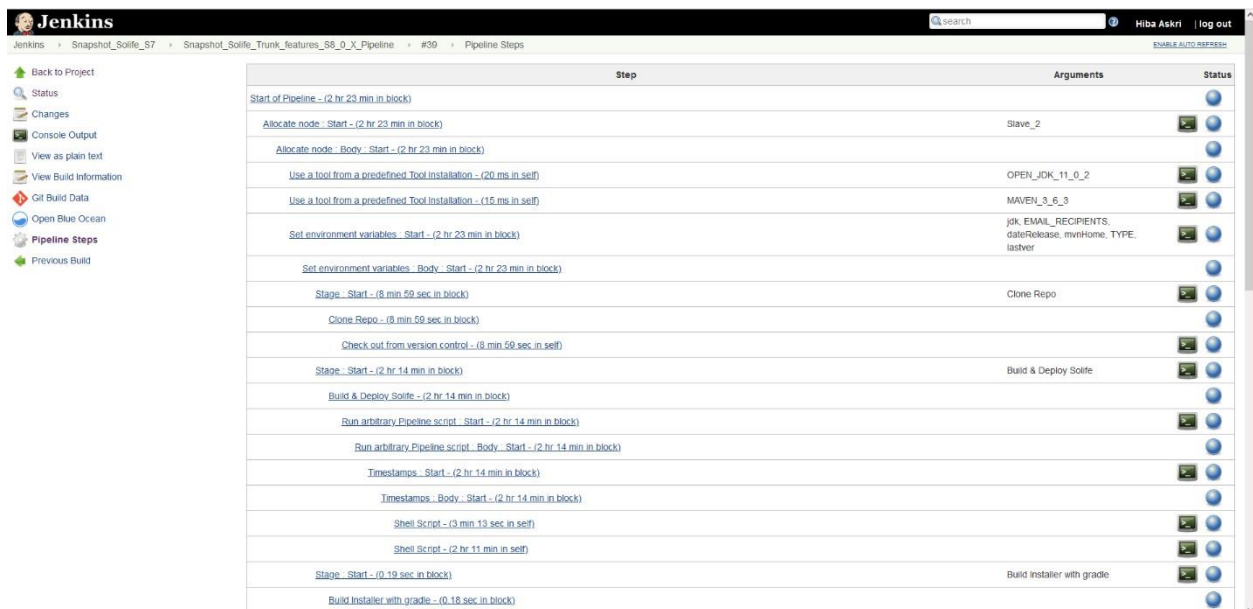
2. **Exécution du Pipeline** : Une fois le pipeline créé, il est exécuté automatiquement chaque fois que des changements sont détectés dans le code source. Jenkins suit alors les étapes définies dans le Jenkinsfile pour automatiser le processus de développement.

Dans l'exemple du pipeline "Snapshot_Solife_Trunk_features_S8_0_X_Pipeline", les étapes suivantes sont exécutées :

- **Start of Pipeline** : Le pipeline démarre, et la durée totale d'exécution est affichée (par exemple, 2 heures et 23 minutes d'attente).
- **Allocate Node** : Jenkins alloue un nœud spécifique pour exécuter le pipeline (ici "Slave_2").
- **Use a Tool from a Predefined Tool Installation** : Jenkins utilise des outils préinstallés tels que **OPEN_JDK_11_0_2** et **MAVEN_3_6_3** pour la compilation et le déploiement.
- **Set Environment Variables** : Des variables d'environnement sont définies pour configurer l'exécution, comme le JDK, les récipiendaires des emails, la version du projet, etc.
- **Clone Repo et Checkout from Version Control** : Le code source est récupéré depuis le dépôt Git pour s'assurer que la version la plus récente est utilisée.
- **Build&DeploySolife** : L'étape de compilation et de déploiement du projet "Solife" est exécutée, ce qui représente une part importante du pipeline. ○

- **Run Arbitrary Pipeline Script** : Un script personnalisé est exécuté pour accomplir des tâches spécifiques au projet.
- **Timestamps et Shell Script** : Des horodatages sont générés pour le suivi et des commandes shell sont exécutées pour effectuer des configurations ou vérifications supplémentaires.
- **Build Installer with Gradle** : Finalement, l'installateur du projet est construit à l'aide de Gradle pour préparer le déploiement du projet.

Ainsi, le pipeline Jenkins permet d'automatiser l'ensemble du processus de développement, de la récupération du code à son déploiement, en passant par la compilation et l'exécution de tests.



Step	Arguments	Status
Start of Pipeline - (2 hr 23 min in block)		
Allocate node - Start - (2 hr 23 min in block)	Slave_2	
Allocate node - Body - Start - (2 hr 23 min in block)		
Use a tool from a predefined Tool Installation - (20 ms in self)	OPEN_JDK_11_0_2	
Use a tool from a predefined Tool Installation - (15 ms in self)	MAVEN_3_6_3	
Set environment variables - Start - (2 hr 23 min in block)	jdk, EMAIL_RECIPIENTS, dateRelease, myHome, TYPE, lastver	
Set environment variables - Body - Start - (2 hr 23 min in block)		
Stage - Start - (8 min 59 sec in block)	Clone Repo	
Clone Repo - (8 min 59 sec in block)		
Check out from version control - (8 min 59 sec in self)		
Stage - Start - (2 hr 14 min in block)	Build & Deploy Solite	
Build & Deploy Solite - (2 hr 14 min in block)		
Run arbitrary Pipeline script - Start - (2 hr 14 min in block)		
Run arbitrary Pipeline script - Body - Start - (2 hr 14 min in block)		
Timestamps - Start - (2 hr 14 min in block)		
Timestamps - Body - Start - (2 hr 14 min in block)		
Shell Script - (3 min 13 sec in self)		
Shell Script - (2 hr 11 min in self)		
Stage - Start - (0 19 sec in block)	Build installer with gradle	
Build installer with gradle - (0 18 sec in block)		

Figure 61. Exécution du Pipeline

6.4.3.2 Configuration de la chaîne de déploiement

a) Mise en place d'un pipeline de déploiement

La Figure suivante présente le pipeline de déploiement quotidien du dernier snapshot généré dans Nexus

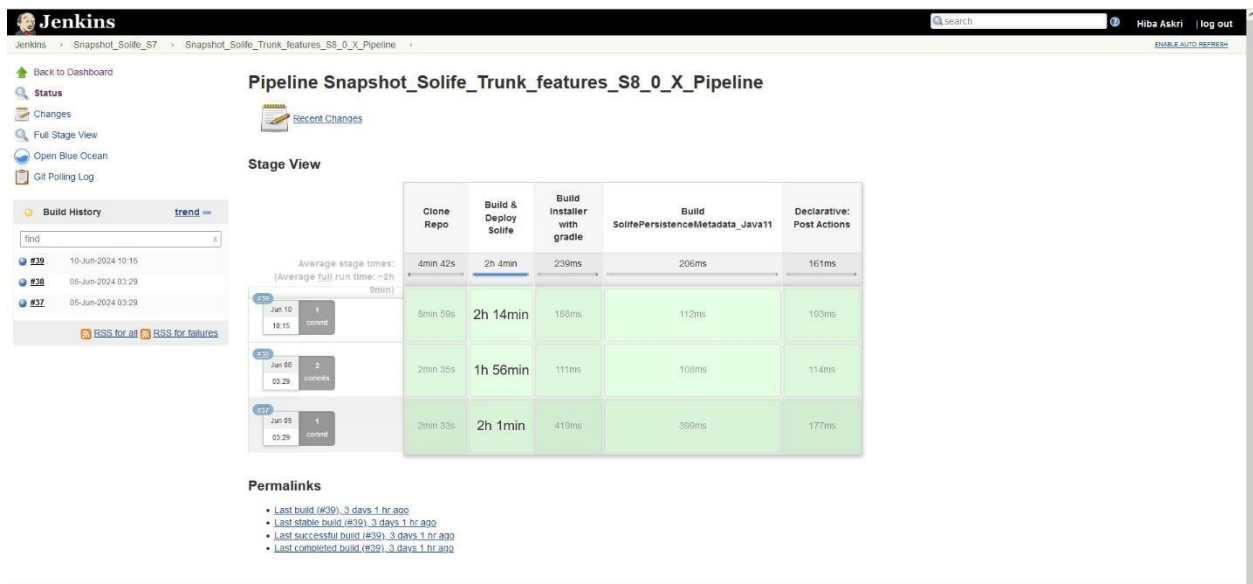


Figure 62. Vue pipeline

b) Processus de construction et de déploiement du pipeline

Le pipeline commence par le clonage du dépôt de code source depuis un serveur distant vers l'environnement de construction, garantissant ainsi que la dernière version du code est utilisée pour les étapes suivantes. Ensuite, une compilation et un déploiement de l'application Solife sont effectués, comprenant la gestion des dépendances et l'exécution des tests pour s'assurer que l'application est prête à être déployée. Un installateur pour le projet est ensuite généré avec Gradle, permettant de créer un package ou un exécutable pour la distribution. Parallèlement, un sous module du projet, SolifePersistenceMetadata, est compilé en utilisant Java 11, avec la lecture du fichier pom.xml, ce qui indique l'utilisation de Maven pour la gestion des dépendances et de la configuration. Une notification par email est ensuite envoyée à l'équipe, afin de communiquer le statut de la construction, qu'il soit réussi ou échoué. Une étape de construction générale peut également être effectuée pour vérifier ou documenter l'état final. Enfin, un autre build pour le module SolifePersistenceMetadata_Java11 peut être déclenché, indiquant qu'un processus supplémentaire est nécessaire suite aux étapes précédentes.

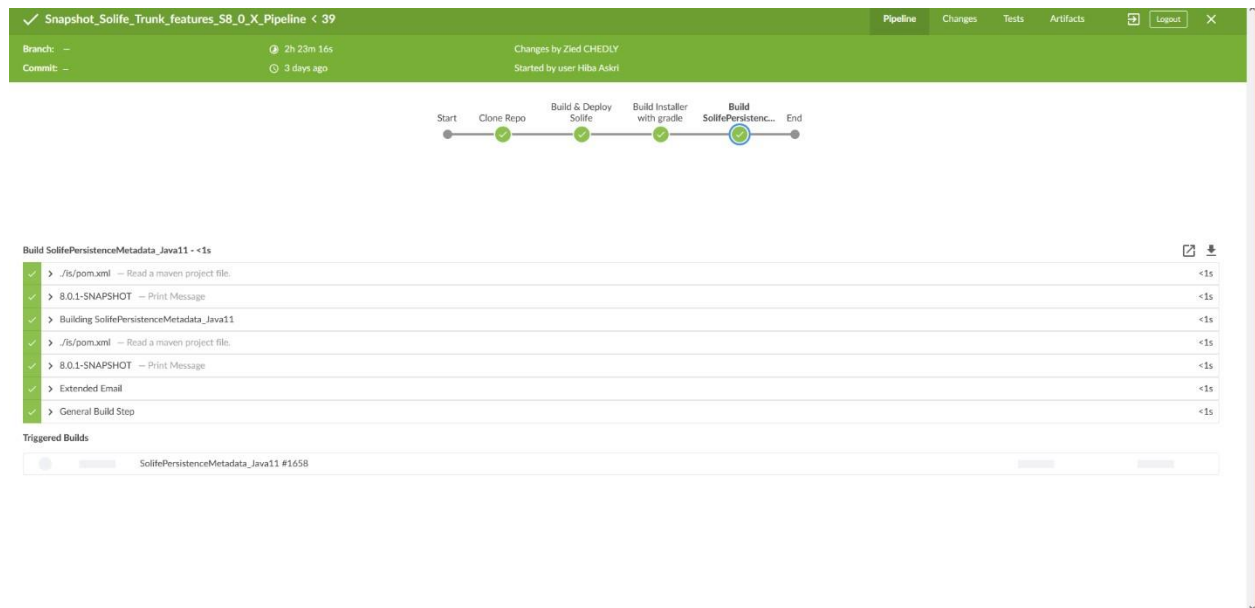


Figure 63. Processus déploiement

Conclusion

Tout au long de ce chapitre nous avons fourni une vue détaillée sur les étapes de réalisation du dernier sprint commençant par la gestion du backlog et la phase de conception jusqu'à l'implémentation des tests automatisés, l'intégration continue et le déploiement continu : la génération de la release.

Conclusion générale

Ce rapport présente une synthèse de mon stage de fin d'études, réalisé en alternance au sein de l'entreprise Vermeg. Cette expérience a été décisive pour mon insertion dans le monde professionnel, me permettant de mettre en pratique les connaissances acquises durant mes études.

Mon stage a principalement porté sur l'étude, le développement et la mise en œuvre de web services REST dédiés à la gestion des clauses bénéficiaires dans les produits d'assurance vie.

Ce rapport détaille les différentes étapes ayant mené à la réalisation de cette solution, en réponse aux besoins spécifiques de certains clients de Vermeg. Le projet a commencé par une analyse approfondie du contexte général, suivie d'une présentation de l'entreprise d'accueil. Une phase clé a été l'étude détaillée de l'existant, fournissant des bases solides pour la conception de la solution. Nous avons également exploré l'utilisation de divers outils et technologies afin de garantir l'efficacité du projet.

La phase suivante a consisté à définir l'architecture du projet ainsi que le planning de travail, en prenant en compte les priorités identifiées. Chaque sprint a été précédé d'une étude conceptuelle approfondie, incluant la création de diagrammes et la mise en place de tests pour valider les résultats obtenus à chaque étape.

En conclusion, ce projet représente une initiative prometteuse. Bien que les objectifs fixés aient été globalement atteints, le projet reste perfectible, comme tout projet humain. La solution développée est évolutive et ouvre la voie à des extensions futures, telles que :

- L'élargissement de l'APIsation à d'autres fonctionnalités pour simplifier encore davantage la gestion des produits Solife.
- L'ajout de nouveaux web services pour la gestion des configurations sur Solife.

Ce projet a permis de concevoir une solution technique pertinente et a posé les bases de futures améliorations. Il contribue également à renforcer la position de Vermeg sur le marché de l'assurance vie.

Webographie

- [1] Vermeg : <https://www.vermeg.com/>
- [2] Java : <https://www.futura-sciences.com/tech/definitions/informatique-java-485/>
- [3] IntelliJ IDEA : https://fr.vikidia.org/wiki/IntelliJ_IDEA [4] Postman :
<https://www.postman.com/>
- [5] ReadyAPI : <https://smartbear.com/product/ready-api/overview/>
- [6] Allure : <https://allure.qatools.ru/>
- [7] Keycloak : <https://www.keycloak.org/>



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889